Date Published: April 01, 2026

*Visit the Zebra Product Support page for the most up-to-date version of this documentation.*

# Mapp functions

## Synopsis

The functions prefixed with Mapp make up the Application module. The Application module allows you to initialize and control the execution environment for an Aurora Imaging Library application. The module provides, among other things, integrated debugging services, a high-performance timer for benchmarking, portable threads and event handling, and automatic system resource compensation for processing and memory operations.

## Functions

MappAlloc
MappAllocDefault
MappCloseConnection
MappControl
MappControlMp
MappFileOperation
MappFree
MappFreeDefault
MappGetError
MappGetHookInfo
MappHookFunction
MappInquire
MappInquireConnection
MappInquireMp
MappOpenConnection
MappTimer
MappTrace

# MappAlloc

## Synopsis

Allocate an application context.

## Syntax

```
AIL_ID MappAlloc(
    AIL_CONST_TEXT_PTR ServerDescription, //in
    AIL_INT64 InitFlag,                   //in
    AIL_ID *ContextAppIdPtr               //out
)
```

## Description

This function allocates an Aurora Imaging Library application context. An application context must be allocated prior to using any other function.

In multi-thread environments, an application context is allocated in the main thread and is shared by all threads. Mapp...() function calls from any thread apply to all threads, unless specifically localized to that thread (typically by specifying M_THREAD_CURRENT when calling the function).

In addition, a default 2D graphics context is also allocated upon allocation of an application context. You can use this default 2D graphics context in graphic function calls by specifying M_DEFAULT wherever a 2D graphics context identifier is required. In multi-thread applications, a default 2D graphics context is allocated for each thread to avoid inter-thread interference.

When allocating a new application context that will be used in a Distributed Aurora Imaging Library monitoring configuration (either the monitoring application or publishing application), you should specify the cluster manager. The cluster manager is a server that prevents duplicate application context identifiers by supplying each new application context with a range of allowable identifiers to choose from.

After allocating the application context, you should check if the operation was successful, using MappGetError() or by verifying that the application context identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the application context is no longer required, release it using MappFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the application context's lifetime and you must not manually free it.

## Parameters

ServerDescription

Specifies the computer name or IP address of the cluster manager. When not in a Distributed Aurora Imaging Library environment, set this parameter to M_NULL.

| For specifying the cluster manager | |
|---|---|
| Value | Description |

| | |
|---|---|
| AIL_TEXT("M_DEFAULT") 1 | Specifies to use the cluster manager specified in the Aurora Imaging Configurator utility (**Cluster Manager** pane, accessible from the **Distributed Aurora Imaging Library** item).<br><br>> This only applies when allocating an application within a Distributed Aurora Imaging Library monitoring configuration. |
| M_NULL | Specifies that no cluster manager will be used. This applies to applications in the controlling configuration and any application not part of a Distributed Aurora Imaging Library configuration. |
| AIL_TEXT("*ClusterManagerName*") 1 | Specifies the computer name or IP address of the cluster manager.<br><br>Replace "ClusterManagerName" with a string specifying the computer name or IP address of the server used as the cluster manager; both IPv4 and IPv6 addresses are supported.<br><br>> This only applies when allocating an application within a Distributed Aurora Imaging Library monitoring configuration. |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

InitFlag

Specifies the type of initialization to perform on the application context. This parameter should be set to one of the following values:

| For specifying the type of initialization | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the default initialization. During the allocation of the application context, reported error messages are displayed. |
| M_QUIET + | Specifies to suppress the displaying of error messages during the allocation of the application context. Note that this value can be combined with M_TRACE_LOG_DISABLE. |
| M_TRACE_LOG_DISABLE + | Specifies to disable the creation of trace logs by any external program, such as Aurora Imaging Profiler or the Aurora Imaging Configurator utility (Interactive Troubleshooting). Calling MappControl() with M_TRACE set to M_LOG_ENABLE will override M_TRACE_LOG_DISABLE from the point in the code that the function appears.<br><br>Disabling trace logs with M_TRACE_LOG_DISABLE helps to protect intellectual property against reverse engineering because no external program will be able to generate a trace for this application. Note that this value can be combined with M_QUIET. |

Combination value for M_QUIET; M_TRACE_LOG_DISABLE.

You can add the following value to the above-mentioned values to set which hardware-acceleration display mode to use.

By default, the mode used is the one set using the Aurora Imaging Configurator utility.

| For specifying the hardware-acceleration display mode | |
|---|---|
| Combination value | Description |

| | |
|---|---|
| M_X11_ACCELERATION(<br>AIL_INT Status<br>) | [This is only applicable to Linux]<br><br>Specifies whether Aurora Imaging Library can use X11 acceleration for display purposes. This determines the hardware acceleration display mode. |
| | **Parameters** |
| | Status INQ<br><br>This parameter specifies whether X11 acceleration can be used. You can set this parameter to one of the following: |
| | 0 — Specifies that X11 acceleration cannot be used. In this case, the standard hardware acceleration mode is used for display. Rendering might be slow. |
| | 1 — Specifies that X11 acceleration can be used. In this case, X11 hardware acceleration mode is used for display. |

Combination value for M_QUIET; M_TRACE_LOG_DISABLE.

You can add the following value to the above-mentioned values to specify a cluster node value without using a cluster manager.

| For specifying a cluster node value | |
|---|---|
| Combination value | Description |
| M_CLUSTER_NODE(<br>AIL_INT ClusterNodeValue<br>) | Specifies a cluster node value when none is specified with a cluster manager. A Distributed Aurora Imaging Library publishing application requires a cluster node value that identifies the publishing application. If you have not specified a cluster manager for this application, ensure that each publishing application in the cluster has a unique, non-zero value.<br><br>> Note that a cluster manager automatically assigns a cluster node value to an application. It is the preferred method of managing cluster node values. |
| | **Parameters** |
| | ClusterNodeValue<br><br>Specifies the cluster node value. |
| | *Value* > 0 — Specifies the cluster node value. |

ContextAppIdPtr

Specifies the address of the variable in which to write the application context identifier or specifies the data type that the function should use to return the application context identifier.

| For retrieving the identifier or specifying how to return it | |
|---|---|
| Value | Description |

| | |
|---|---|
| M_NULL | Specifies that you will use this function's return value to obtain the identifier of the allocated application context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned. |
| M_UNIQUE_ID | Specifies that you will use this function's return value to obtain the identifier of the allocated application context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_APP_ID* is returned instead of a standard Aurora Imaging Library identifier.<br><br>> This setting is only available when using C++11 (or later).<br><br>An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the application context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application. |
| *Address in which to write the identifier* | Specifies the address of an *AIL_ID* in which to write the identifier of the allocated application context.<br><br>If allocation fails, **M_NULL** is written as the identifier. |

## Return value

The returned value is the application context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_APP_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

## Remark

- If you are creating a DLL that includes a call to **MappAlloc**(), ensure that the call is not made from the `DllMain()` function, because **MappAlloc**() might load a required DLL and you cannot load a DLL from `DllMain()`. If necessary, call **MappAlloc**() from an initialization function in your DLL instead.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappAllocDefault

## Synopsis

Allocate default Aurora Imaging Library objects.

## Syntax

```
void MappAllocDefault(
    AIL_INT InitFlag,        //in
    AIL_ID *ContextAppIdPtr, //out
    AIL_ID *SysIdPtr,        //out
    AIL_ID *DispIdPtr,       //out
    AIL_ID *DigIdPtr,        //out
    AIL_ID *ImageBufIdPtr    //out
)
```

## Description

**MappAllocDefault**() is implemented as a macro. This macro can allocate and set up the requested objects using the defaults specified during allocation and using the Aurora Imaging Configurator utility. This macro can allocate and initialize an application context, and allocate the default system. On this system, **MappAllocDefault**() can allocate the default digitizer and display, and allocate and clear a default displayable image buffer.

The installation customizes the default setup. After installation, if you want to review or change the default settings of **MappAllocDefault**(), use the Aurora Imaging Configurator utility.

Note, if allocating the default digitizer and the default DCF, selected using the Aurora Imaging Configurator utility, is for a 3-band color (RGB) camera, then the default image buffer will be 3-band if allocated; otherwise, it will be 1-band if allocated. The size of a default image buffer is 640x480, in pixels.

Note, upon execution of this function, a default 2D graphics context is automatically allocated. This default 2D graphics context can be used in function calls by specifying M_DEFAULT wherever a 2D graphics context identifier is required.

When you have finished using the allocated objects, you must free them using MappFreeDefault(). With multiple threads, you must free the allocated application context (using MappFreeDefault()) in the thread in which it was allocated.

## Parameters

InitFlag

Specifies the type of initialization setup to perform and is used principally to initialize the default system. This parameter can be set to one of the following:

| For specifying the type of initialization setup | |
|---|---|
| Value | Description |
| M_DEFAULT | Same as M_COMPLETE. |

| | |
|---|---|
| M_COMPLETE + | Specifies to initialize the system completely; the system is initialized to its default state and any required resident software is downloaded. At least one complete initialization is necessary after you power-up your system. |

Combination value for the values listed in For specifying the type of initialization setup.

You can add the following value to the above-mentioned value to specify a cluster node value without using a cluster manager.

| For specifying a cluster node value | |
|---|---|
| Combination value | Description |
| M_CLUSTER_NODE( AIL_INT ClusterNodeValue ) | Specifies a cluster node value when none is specified with a cluster manager. A Distributed Aurora Imaging Library publishing application requires a cluster node value that identifies the publishing application. If you have not specified a cluster manager for this application, ensure that each publishing application in the cluster has a unique, non-zero value.<br>> Note that a cluster manager automatically assigns a cluster node value to an application. It is the recommended method of managing cluster node values. |
| | **Parameters** |
| | ClusterNodeValue<br>Specifies the cluster node value. |
| | *Value* > 0 — Specifies the cluster node value. |

ContextAppIdPtr

Specifies the address of the variable in which to write the application context identifier. Upon execution of this function, the application context is allocated and its identifier returned. Instead of using **MappAllocDefault**(), you can use MappAlloc() to allocate an application context. Note, an application context must be allocated to allocate any other object.

SysIdPtr

Specifies the address of the variable in which to write the system identifier. Upon execution of this function, the default system is allocated and its identifier returned. Instead of using **MappAllocDefault**(), you can use MsysAlloc() to allocate a system. MappAlloc() will also allocate a default Host system. Note, a system must be allocated to allocate any other objects on it (display, digitizer or data buffers).

DispIdPtr

Specifies the address of the variable in which to write the identifier of the default display. If this parameter is set to **M_NULL**, a display is not allocated; otherwise, the default display is allocated and its identifier returned.

DigIdPtr

Specifies the address of the variable in which to write the identifier of the default digitizer. If this parameter is set to **M_NULL**, a digitizer is not allocated; otherwise, the default digitizer is allocated and its identifier returned.

ImageBufIdPtr

Specifies the address of the variable in which to write the identifier of the default image buffer. If this parameter is set to **M_NULL**, an image buffer is not allocated; otherwise, the default image buffer is allocated and its identifier returned. It is then cleared and displayed on the system's display screen.

## Examples

For example, a typical setup for a Zebra board in its power-up state with one input device and one default image buffer (full-screen size) selected on the default display is:

```
MappAllocDefault(M_COMPLETE, &Application, &System, &Display, &Digitizer, &ImageBuffer);
```

If, for example, you don't need to acquire data from the camera but want to perform the rest of the above setup, you would make the following call:

```
MappAllocDefault(M_COMPLETE, &Application, &System, &Display, M_NULL, &ImageBuffer);
```

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappCloseConnection

## Synopsis

Closes a connection to a remote Aurora Imaging Library application.

## Syntax

```
void MappCloseConnection(
    AIL_ID RemoteContextAppId //in
)
```

## Description

Closes the connection opened by MappOpenConnection(). All objects allocated from this node must have been freed, using M...Free(), before disconnecting.

## Parameter

RemoteContextAppId

Specifies the identifier of the remote application context from which to disconnect. This application identifier must have been returned by a previous call to MappOpenConnection().

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappControl

## Synopsis

Control an application environment setting.

## Syntax

```
void MappControl(
    AIL_ID ContextAppId,   //out
    AIL_INT64 ControlType, //in
    AIL_INT ControlValue   //in
)
```

## Description

This function controls the settings of an Aurora Imaging Library application environment. These settings include printing error messages to the screen, parameter checking, logging trace information, as well as on-board memory and processing compensation modes. This function can also clear current and/or global errors.

In multi-thread environments, an **MappControl**() call applies to all application threads running Aurora Imaging Library, unless otherwise stated (or specifically limited to the calling thread by adding M_THREAD_CURRENT to the ControlType parameter). When you override settings for a specific thread, a subsequent call to change those settings from any other thread will not change the settings for the current thread. For example, if you enable trace printing for a particular thread, that thread will ignore calls from other threads that try to change trace printing setting.

## Parameters

ContextAppId

Specifies the identifier of the application context to use.

| For specifying the application context | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application context identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

ControlType

Specifies the type of application environment setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

## Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For managing error functionality
- For specifying general application environment settings

The following ControlType and corresponding ControlValue parameter settings are used to manage error functionality in the application.

| For managing error functionality | |
|---|---|
| ControlType / ControlValue | Description |
| M_CLEAR_ERROR | Clears current and/or global errors. |
| M_DEFAULT | Specifies to clear both current and global errors. |
| M_CURRENT | Specifies to clear only current errors. |
| M_GLOBAL | Specifies to clear only global errors. |
| M_ERROR + | Sets whether the printing of error messages to screen is enabled.<br>Note that, within a user-defined function, this setting is independent of the main application. If you enable or disable printing of error messages within a callee function, that setting will not be retained when the callee function ends. INQ |
| M_DEFAULT | Same as M_PRINT_ENABLE, unless M_THREAD_CURRENT is specified, in which case this setting specifies that the current thread uses the setting of the application environment. |
| M_PRINT_DISABLE | Specifies not to print error messages. If error printing is disabled, you can still check for error, using MappGetError(). |
| M_PRINT_ENABLE | Specifies to print error messages. |
| M_THROW_EXCEPTION | Specifies that when an error occurs, a .NET or Python exception will be thrown instead of a message box being displayed.<br>> M_THROW_EXCEPTION is only valid when working in a .NET or Python environment. |
| M_ERROR_HOOKS | Sets whether errors generated in the current thread will trigger the user-defined function that you specified using MappHookFunction() (with M_ERROR_CURRENT or M_ERROR_FATAL). INQ |

| | | |
|---|---|---|
| | M_DISABLE | Specifies to not trigger the user-defined function (hooked using MappHookFunction()) when an error event occurs.<br><br>This ignores the functionality of MappHookFunction() with regards to error event types, until the functionality is restored using M_ERROR_HOOKS set to M_ENABLE.<br><br>> If an error of the appropriate type is generated in another thread, the user-defined function will still be triggered. |
| | M_ENABLE | Specifies to trigger the user-defined function when an error event occurs, as specified in MappHookFunction().<br><br>This is the default value. |

The following ControlType and corresponding ControlValue parameter settings are used to control the application environment.

| For specifying general application environment settings | | |
|---|---|---|
| ControlType | | Description |
| | ControlValue | |
| M_COMPONENT_AUTO_RESELECT + | | Sets whether image buffer components are automatically reselected to 2D displays when they are reallocated. A component is considered reallocated when a single function internally frees the component and allocates a new similar component in the same container.<br><br>This setting does not affect containers shown in 3D displays. INQ |
| | M_DEFAULT | Same as M_COMPONENT_AUTO_RESELECT_ENABLE, unless M_THREAD_CURRENT was specified, in which case this setting specifies that the current thread uses the setting of the application environment.<br><br>Same as M_COMPONENT_AUTO_RESELECT_ENABLE. |
| | M_COMPONENT_AUTO_RESELECT_DISABLE | Specifies not to automatically reselect image buffer components to 2D displays. If a component selected to a 2D display is reallocated, the 2D display will close. |
| | M_COMPONENT_AUTO_RESELECT_ENABLE | Specifies to automatically reselect image buffer components to 2D displays when they are reallocated.<br><br>For example, this is useful if you want to display a single component from a container that you are grabbing into and the layout of the container might change between grabs (in which case, the components of the container are freed and reallocated). |

| | |
|---|---|
| M_DAIL_CONNECTION | Specifies the Distributed Aurora Imaging Library monitoring configuration global permission level.<br><br>This allows or disallows another application, the monitoring application, to provisionally access specified objects in the current application, the publishing application. Each object in the publishing application must individually allow either read only, or read/write access by calling MobjControl() with M_DAIL_PUBLISH set to either M_READ_ONLY or M_READ_WRITE, respectively. Any object not specified as read only or read/write is not accessible by the monitoring application. INQ |
| M_DAIL_CONTROL | Specifies that objects in the current application can be accessed with either read only or read/write permission, depending on how that object was specified with MobjControl(). |
| M_DAIL_MONITOR | Specifies that objects in the current application can be accessed only with read only permission, regardless of how that object was specified with MobjControl(). |
| M_DISABLE | Specifies that Distributed Aurora Imaging Library monitoring configuration is disabled in the current application. Once disabled, a monitoring application cannot access any objects in the current application. |
| M_DAIL_CONNECTION_PORT | Specifies the listening port for a publishing application in the Distributed Aurora Imaging Library monitoring configuration. If the publishing application has specified a listening port, the monitoring application connecting to the publishing application must specify that listening port. If the publishing application does not specify a listening port, the default listening port, set using the Aurora Imaging Configurator utility, is used.<br><br>M_DAIL_CONNECTION_PORT only applies when the current application is a publishing application in a Distributed Aurora Imaging Library monitoring configuration. INQ |
| *Listening port* | Specifies the listening port (an integer between 0 and 65535) that the current application will use to listen for connections from a monitoring application. |
| M_MEMORY + | Sets whether on-board memory compensation on the Host is enabled. On-board memory compensation allows memory to be allocated on Host when a system has insufficient memory to perform that memory allocation. INQ |
| M_DEFAULT | Same as M_COMPENSATION_ENABLE, unless M_THREAD_CURRENT is specified, in which case this setting specifies that the current thread uses the setting of the application environment. |
| M_COMPENSATION_DISABLE | Specifies that on-board memory compensation is disabled. If there is insufficient memory available for the allocation, an error is generated. |
| M_COMPENSATION_ENABLE | Specifies that on-board memory compensation is enabled. |
| M_PARAMETER + | Sets whether the checking of parameters is enabled. INQ |

| | | |
|---|---|---|
| | M_DEFAULT | Same as M_CHECK_ENABLE, unless M_THREAD_CURRENT is specified, in which case this setting specifies that the current thread uses the setting of the application environment. |
| | M_CHECK_DISABLE | Specifies that checking of parameters is disabled. Note, if parameter checking is disabled (for example, to accelerate an application), unpredictable behavior can be expected when passing invalid parameters to a function. |
| | M_CHECK_ENABLE | Specifies that checking of parameters is enabled. |
| M_PROCESSING + | | Sets whether the Host should compensate for processes that cannot be performed on-board due to board-specific limitations. INQ |
| | M_DEFAULT | Same as M_COMPENSATION_ENABLE, unless M_THREAD_CURRENT is specified, in which case this setting specifies that the current thread uses the setting of the application environment. |
| | M_COMPENSATION_DISABLE | Specifies that on-board processing compensation is disabled. If an operation cannot be performed by the system's on-board processor, an error is generated. |
| | M_COMPENSATION_ENABLE | Specifies that on-board processing compensation is enabled. If an operation cannot be performed by the system's on-board processor, it will be performed by the Host. |
| M_TRACE + | | Sets the trace setting, which controls when a trace log is generated.<br><br>The trace log is a binary file with a *.mtrace* extension and is readable only with Aurora Imaging Profiler.<br><br>This control type is synchronous. INQ |
| | M_DEFAULT | Specifies the default trace setting, which either enables or disables generating a trace log, based on whether Aurora Imaging Profiler (or the Interactive log) was used to start generating the log.<br><br>If Aurora Imaging Profiler (or the Interactive log) started generating the trace log, this control value is the same as M_LOG_ENABLE.<br><br>If a function started generating the trace log, this control value is the same as M_LOG_DISABLE. |
| | M_LOG_DISABLE | Specifies the log disable setting, which stops generating the trace log. |
| | M_LOG_ENABLE | Specifies the log enable setting, which starts generating the trace log. |

| | | |
|---|---|---|
| M_TRACE_SAVE_TO_FILE | | Transfers gathered trace data to file depending on how the trace log is generated.<br><br>When generating a trace log to memory with Aurora Imaging Profiler, this control type transfers all gathered trace data currently in the temporary circular trace log buffer in chronological order to an *.mtrace* file with the specified path and/or file name. The resulting *.mtrace* file only contains the most recent trace data present in the circular buffer.<br><br>When generating a trace log to file, this control type forces all outstanding writes to be performed to the *.mtrace* file in the default file path. When generating the trace log using Aurora Imaging Profiler, the default file path is *%PROGRAMDATA%\Aurora Imaging Library\11\Profiler*; otherwise, the default file path is %tmp%. The file will contain all trace data from the moment the trace started gathering data until the moment when this control is called. In this case, ControlValue must be set to M_NULL. |
| | M_NULL | Specifies to use the default path and default file name. |
| | AIL_TEXT("*FileName*") 1 | Specifies the file name, but uses the default path. For instance, AIL_TEXT("Test.mtrace").<br><br>Note, if using a C compiler, you must pass the value as AIL_PTR_TO_INT(AIL_TEXT("*FileName*")). |
| | AIL_TEXT("*Path*") 1 | Specifies the path, but uses the default file name. For instance, AIL_TEXT("D:\\MyFolder").<br><br>Note, if using a C compiler, you must pass the value as AIL_PTR_TO_INT(AIL_TEXT("*Path*")).<br><br>Note that every folder in the path must already exist. This control only uses existing folders. |
| | AIL_TEXT("*PathAndFileName*") 1 | Specifies the path and the file name. For instance, AIL_TEXT("D:\\MyFolder\\Test.mtrace").<br><br>Note, if using a C compiler, you must pass the value as AIL_PTR_TO_INT(AIL_TEXT("*PathAndFileName*")).<br><br>Note that every folder in the path must already exist. This control only uses existing folders. |

| | |
|---|---|
| M_WEB_CONNECTION | Sets whether Aurora Imaging Web Library server functionality is enabled.<br><br>When Aurora Imaging Web Library server functionality is enabled, your application is a server application. You can publish some types of objects using MobjControl() with M_WEB_PUBLISH. You can access these objects over the network using an Aurora Imaging Web Library client application (for example, running in a web browser).<br><br>For more information about Aurora Imaging Web Library, including how to create a client application, see Chapter 58: Using Aurora Imaging Web Library to monitor your application.<br><br>Aurora Imaging Web Library servers are intended to be run on a segmented LAN that has no internet/WAN access; servers offer no internal security mechanisms. When Aurora Imaging Web Library is enabled, you should typically ensure that your network administrator configures your network to prevent incoming connections from the internet/WAN to the computer running your application (and from any other devices on your network that do not need access). If this is not possible, at a minimum you should block incoming connections from the internet to the listening port of the server. You can inquire which port is used for this purpose, using MappInquire() with M_WEB_CONNECTION_PORT. INQ |
| M_DISABLE | Specifies that Aurora Imaging Web Library server functionality is disabled in the current application. |
| M_ENABLE | Specifies that Aurora Imaging Web Library server functionality is enabled in the current application. |
| M_WEB_CONNECTION_PORT | Sets the listening port used for Aurora Imaging Web Library server functionality. If you specify a listening other than the default, your Aurora Imaging Web Library client application must specify that port when establishing a connection. If the server application does not specify a listening port, the default listening port 7861 is used.<br><br>For increased security, you should ensure that remote computers cannot access this port from the internet/WAN. INQ |
| 0 <= *Value* <=65535 | Specifies the port number.<br><br>> This port must not be used by any other application (including third-party applications) running on the local computer.<br><br>The default value is 7861. |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Combination value for M_ERROR; M_COMPONENT_AUTO_RESELECT; M_MEMORY; M_PARAMETER; M_PROCESSING; M_TRACE.

You can add the following value to the above-mentioned values to set whether the control should be limited to the current thread.

| For limiting effects to the current thread | |
|---|---|
| Combination value | Description |

| | |
|---|---|
| M_THREAD_CURRENT | Limits the effect of the control type setting to the current thread.<br><br>> Note that when this value is added to M_TRACE, M_TRACE set to M_DEFAULT will adopt the application's current global trace state (enabled or disabled). |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappControlMp

## Synopsis

Control a multi-core processing application environment setting.

## Syntax

```
void MappControlMp(
    AIL_ID ContextAppId,   //out
    AIL_INT64 ControlType, //in
    AIL_INT64 TypeFlag,    //in
    AIL_INT64 TypeValue,   //in
    void *ValuePtr         //in
)
```

## Description

This function controls the settings of an Aurora Imaging Library multi-core processing application environment. It establishes whether multi-core processing can be used to execute certain parts of functions. It also sets how multi-core processing is performed.

In multi-thread environments, an **MappControlMp**() call applies to all application threads running Aurora Imaging Library, unless the specific setting was overridden for a specific thread using MthrControlMp().

## Parameters

ContextAppId

Specifies the identifier of the application context to use.

| For specifying the application context | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application context identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

ControlType

Specifies the type of multi-core processing application environment setting to control.

See the Parameter associations section for possible values that can be specified.

TypeFlag

Specifies additional information regarding the control setting. This parameter must be set to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

TypeValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

ValuePtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_UINT64 [optionally, in C++: a reference to a constant std::vector<AIL_UINT64> ]

Specifies the address which contains more information about the setting's new value. Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

## Parameter associations

The table below lists possible values for the **ControlType**, **TypeFlag**, **TypeValue**, and **ValuePtr** parameters.

- For specifying multi-core processing application environment settings

Set unused parameters to **M_NULL**.

Note that TypeValue and TypeFlag should be set to **M_DEFAULT** if they are not used.

| For specifying multi-core processing application environment settings | |
|---|---|
| ControlType | Description |
| TypeFlag | |
| TypeValue | |
| ValuePtr | |
| M_CORE_AFFINITY_MASK | Sets the core affinity bit-mask, which indicates on which CPU core(s) (processor(s)) to run the multi-core processing part of Aurora Imaging Library functions. Essentially, the core affinity mask defines the preferred processing CPU core(s) to use for multi-core processing.<br><br>You can override this setting for a specific thread using MthrControlMp() with M_CORE_AFFINITY_MASK.<br><br>The number of CPU cores actually used for multi-core processing will be affected by M_CORE_MAX (or M_CORE_MAX_FOR_COPY for copy type functions such as MbufCopy()) and M_CORE_AFFINITY_MASK. Typically, the number of cores used for multi-processing will be the lesser of the two values.<br><br>• If M_CORE_MAX (or M_CORE_MAX_FOR_COPY) specifies a larger number than the number of CPU cores enabled with M_CORE_AFFINITY_MASK, only the CPU cores enabled with the core affinity mask will be used for multi-core processing.<br><br>• If M_CORE_MAX (or M_CORE_MAX_FOR_COPY) specifies a smaller number than the number of CPU cores enabled with M_CORE_AFFINITY_MASK, Aurora Imaging Library will restrict multi-core processing to the number of CPU cores specified with M_CORE_MAX (or M_CORE_MAX_FOR_COPY), but only from those enabled with the core affinity mask.<br><br>To establish which CPU cores are assigned to the process running your application, call MappInquireMp() with M_CORE_AFFINITY_MASK_PROCESS. |
| **TypeValue** | Specifies how to define the mask. |

| | | | |
|---|---|---|---|
| | | M_DEFAULT | Specifies to automatically select a value using the current computer setup. The selected value is typically M_ALL. |
| | | M_ALL | Specifies to use all CPU cores available to the process running the application, as per the operating system. |
| | | M_USER_DEFINED | Specifies to use a user-defined core affinity bit-mask. |
| | **ValuePtr** | | Data type info: Data type: *array of type AIL_UINT64 [optionally, in C++: a reference to a constant std::vector<AIL_UINT64> ]*<br><br>Specifies the core affinity bit-mask to use when defining an M_USER_DEFINED mask. INQ |
| | | M_NULL | Specifies that a user-defined mask is not used. |
| | | *Non-zero initialized bit-mask array* | Specifies a non-zero initialized bit-mask array.<br><br>The first element of the array represents the first 64 CPU cores. The least-significant bit of the first element represents CPU core 0. The most-significant bit of the first element represents CPU core 63. The least-significant bit of the second element represents CPU core 64 and so on. CPU cores can be used for multi-core processing if their corresponding bit is enabled (1). If their corresponding bit is disabled (0), multi-core processing is not allowed to occur on those CPU cores. CPU cores always have the same indices, as long as the hardware in your computer and the operating system does not change.<br><br>A normal core affinity bit-mask should have at least one bit enabled so that at least one CPU core is enabled for processing. A core affinity bit-mask whose bits are all set to zero is therefore a special case and represents the default setting of all CPU cores being enabled for processing. |

| | | |
|---|---|---|
| M_CORE_MAX | | Sets the maximum number of CPU cores to use to process the multi-core processing part of each function, when multi-core processing is enabled. You can enable multi-core processing using the Aurora Imaging Configurator utility, **MappControlMp**() with M_MP_USE, or MthrControlMp() with M_MP_USE.<br><br>Note that this control type overrides the value set with the Aurora Imaging Configurator utility.<br><br>To set a different number of CPU cores for the multi-core processing part of each function on a specific thread, use MthrControlMp() with M_CORE_MAX.<br><br>The number of CPU cores actually used for multi-core processing will be affected by M_CORE_MAX (or M_CORE_MAX_FOR_COPY for copy type functions such as MbufCopy()) and M_CORE_AFFINITY_MASK. Typically, the number of cores used for multi-processing will be the lesser of the two values.<br><br>• If M_CORE_MAX (or M_CORE_MAX_FOR_COPY) specifies a larger number than the number of CPU cores enabled with M_CORE_AFFINITY_MASK, only the CPU cores enabled with the core affinity mask will be used for multi-core processing.<br>• If M_CORE_MAX (or M_CORE_MAX_FOR_COPY) specifies a smaller number than the number of CPU cores enabled with M_CORE_AFFINITY_MASK, Aurora Imaging Library will restrict multi-core processing to the number of CPU cores specified with M_CORE_MAX (or M_CORE_MAX_FOR_COPY), but only from those enabled with the core affinity mask.<br><br>The effective number of CPU cores available to Aurora Imaging Library is limited by the number of CPU cores installed in your computer and any limits imposed by the operating system. To establish the number of CPU cores assigned to the process running your application, call MappInquireMp() with M_CORE_NUM_PROCESS.<br><br>Note that the first call to MappAlloc() or MappAllocDefault() determines the number of CPU cores available from the operating system. This information is stored in Aurora Imaging Library and not updated dynamically. Changing the number of processors available at the operating-system level, after your application is allocated, can result in erratic and unpredictable behavior. |
| | **TypeValue** | Specifies one of the following.INQ |
| | M_DEFAULT | Specifies to use the default value. This value is set using the Aurora Imaging Configurator utility. |
| | 1 <= *Value* <= 65535 | Specifies the maximum number of CPU cores to use. To use only one CPU core and disable multi-core processing, set the value to 1 or use the M_MP_USE control type.<br><br>> Note that specifying the number of cores to use as 1 is effectively the equivalent of setting M_MP_USE to M_DISABLE. |

| | | |
|---|---|---|
| M_CORE_MAX_FOR_COPY | | Sets the maximum number of CPU cores to use to process the multi-core processing part of copy type functions, such as MbufCopy(), when multi-core processing is enabled. Multi-core processing can be enabled using the Aurora Imaging Configurator utility, **MappControlMp**() with M_MP_USE, or MthrControlMp() with M_MP_USE.<br><br>Note that this control type overrides the value set with the Aurora Imaging Configurator utility.<br><br>To set a different number of CPU cores for the multi-core processing part of copy type functions on a specific thread, use MthrControlMp() with M_CORE_MAX_FOR_COPY.<br><br>The number of cores set with M_CORE_MAX_FOR_COPY only affects copy type functions, and supercedes the number of cores set with M_CORE_MAX. For instance, if M_CORE_MAX specifies to use 4 cores, then all functions will use 4 cores. If afterwards M_CORE_MAX_FOR_COPY specifies 2 cores, all copy type functions will use 2 cores, while the remaining functions will still use 4.<br><br>The number of CPU cores actually used for multi-core processing will be affected by M_CORE_MAX (or M_CORE_MAX_FOR_COPY for copy type functions such as MbufCopy()) and M_CORE_AFFINITY_MASK. Typically, the number of cores used for multi-processing will be the lesser of the two values.<br><br>• If M_CORE_MAX (or M_CORE_MAX_FOR_COPY) specifies a larger number than the number of CPU cores enabled with M_CORE_AFFINITY_MASK, only the CPU cores enabled with the core affinity mask will be used for multi-core processing.<br><br>• If M_CORE_MAX (or M_CORE_MAX_FOR_COPY) specifies a smaller number than the number of CPU cores enabled with M_CORE_AFFINITY_MASK, Aurora Imaging Library will restrict multi-core processing to the number of CPU cores specified with M_CORE_MAX (or M_CORE_MAX_FOR_COPY), but only from those enabled with the core affinity mask.<br><br>The effective number of CPU cores available to Aurora Imaging Library is limited by the number of CPU cores installed in your computer and any limits imposed by the operating system. To establish the number of CPU cores assigned to the process running your application, call MappInquireMp() with M_CORE_NUM_PROCESS.<br><br>Note that the first call to MappAlloc() or MappAllocDefault() determines the number of CPU cores available from the operating system. This information is stored in Aurora Imaging Library and not updated dynamically. Changing the number of processors available at the operating-system level, after your application is allocated, can result in erratic and unpredictable behavior. |
| | **TypeValue** | Specifies one of the following.INQ |
| | M_DEFAULT | Specifies to use the default value. This value is set using the Aurora Imaging Configurator utility. |
| | M_FOLLOW_CORE_MAX | Specifies to use the current value of M_CORE_MAX. |
| | 1 <= *Value* <= 65535 | Specifies the maximum number of CPU cores to use. To use only one CPU core and disable multi-core processing for copy type functions, set the value to 1. |

| | | |
|---|---|---|
| M_CORE_SHARING | | Sets whether multi-core processing can use multiple logical cores per physical CPU core, when hyper-threading is enabled and supported.<br><br>An Intel processor's hyper-threading technology allows each of its physical CPU cores to be represented by multiple logical CPU cores, typically improving parallelization of computations. However, depending on the processing operation, this might reduce the processing speed. If you disable M_CORE_SHARING, Aurora Imaging Library multi-core processing executes as if hyper-threading was disabled and restricts multi-core processing to one logical core per physical CPU core, minimizing logical core interactions.<br><br>Note that a thread might not have exclusive access to a CPU core; other processes or threads might still use the other logical cores of a physical CPU core and might impede multi-core processing restricted this way.<br><br>This control type only affects how the multi-core processing part of functions is performed.<br><br>You can override this setting for a specific thread using MthrControlMp() with M_CORE_SHARING. |
| | **TypeValue** | Specifies one of the following.INQ |
| | M_DEFAULT | Specifies to use the default value. This default value is initially M_DISABLE, but can be changed using the Aurora Imaging Configurator utility. To change the default value, select the **Core Sharing** option in the **Basic Settings** pane, accessible from the **Multi-Processing** item in the Aurora Imaging Configurator utility. |
| | M_DISABLE | Specifies to use only one logical core per physical CPU core, if hyper-threading is enabled. |
| | M_ENABLE | Specifies to use all the logical cores of a physical CPU core, if hyper-threading is enabled. |
| M_MP_PRIORITY | | Sets the processing priority of the multi-core processing part of functions.<br><br>You can override this setting for a specific thread using MthrControlMp() with M_MP_PRIORITY. |
| | **TypeValue** | Specifies one of the following.INQ |
| | M_DEFAULT | Same as M_NORMAL. |
| | M_ABOVE_NORMAL | Specifies that the multi-core processing part of functions will be executed with above normal priority. |
| | M_BELOW_NORMAL | Specifies that the multi-core processing part of functions will be executed with below normal priority. |
| | M_HIGHEST | Specifies that the multi-core processing part of functions will be executed with high priority. Only M_TIME_CRITICAL assigns a higher priority. |

| | | | |
|---|---|---|---|
| | | M_IDLE | Specifies that the multi-core processing part of functions will be executed with idle priority. All idle priority processing objects are only executed when a CPU core is idle. |
| | | M_LOWEST | Specifies that the multi-core processing part of functions will be executed with low priority. |
| | | M_NORMAL | Specifies that the multi-core processing part of functions will be executed with normal priority. |
| | | M_TIME_CRITICAL | Specifies that the multi-core processing part of functions will be executed with time critical priority. Time critical processing objects have highest priority. |
| M_MP_USE | | | Sets whether multi-core processing can be used to execute certain parts of functions.<br><br>To enable or disable multi-core processing for a specific thread, use MthrControlMp() with M_MP_USE.<br><br>Note that the Aurora Imaging Configurator utility can override and disable all multi-core processing. To inquire whether the Aurora Imaging Configurator utility has disabled multi-core processing, use MappInquireMp() with M_MP_FORCED_DISABLE. |
| | **TypeValue** | | Specifies one of the following.INQ |
| | | M_DEFAULT | Specifies to use the default value. This value is set using the Aurora Imaging Configurator utility. |
| | | M_DISABLE | Specifies that multi-core processing cannot be used. |
| | | M_ENABLE | Specifies that multi-core processing can be used. |
| M_MP_USE_PERFORMANCE_LEVEL | | | Sets which cores a multi-core processing application can use on a heterogeneous architecture, according to their performance level.<br><br>Use MappInquireMp() with M_MP_NB_PERFORMANCE_LEVEL to get the number of core performance levels. |
| | **TypeFlag** | | Specifies one of the following. |
| | | M_ALL | Specifies to use all CPU cores, regardless of performance level. |
| | | 1 <= *Value* <= 5 | Specifies the performance level.<br><br>> Note that Level 1 corresponds to the fastest cores. |
| | **TypeValue** | | Specifies one of the following.INQ |

| | | | |
|---|---|---|---|
| | | M_DEFAULT | Specifies to use the default value. This value is set using the Aurora Imaging Configurator utility. |
| | | M_DISABLE | Specifies that multi-core processing cannot be used on cores with the specified performance level. |
| | | M_ENABLE | Specifies that multi-core processing can be used on cores with the specified performance level. |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll, ailmp11.dll. |

# MappFileOperation

## Synopsis

Perform a file operation or file search on a local or a remote computer.

## Syntax

```
void MappFileOperation(
    AIL_ID Comp1ContextAppId,         //in
    AIL_CONST_TEXT_PTR Comp1FileName, //in
    AIL_ID Comp2ContextAppId,         //in
    AIL_CONST_TEXT_PTR Comp2FileName, //in
    AIL_INT64 Operation,              //in
    AIL_INT64 OperationFlag,          //in
    void *OperationDataPtr            //out
)
```

## Description

This function can copy or move a file from one computer to another, including your local computer as either source or destination. This function can also remotely delete a file, create or delete a directory, search a directory for files with a specific file name (wildcards are supported), or execute a program on another computer. When executing a program, you can choose to wait for the remotely-run program to finish before proceeding with the local program, or start the remote program and immediately continue with the local program.

This function is typically used to perform operations on remote computers, but also works on the local computer.

When using this function to manipulate files (for example, moving, copying, or executing them), you must set the Comp1FileName parameter (and optionally the Comp2FileName parameter) to a file name or full file path. If you specify only a file name or partial file path (without drive letter), the working directory of the application is assumed as the location of the file (or as the prefix of the partial file path).

When using this function to search for files (M_FILE_NAME_FIND and M_FILE_NAME_FIND + M_NB_ELEMENTS), you must specify a file name search pattern. The search might find more than one matching file. You must specify which file name to return by setting the OperationFlag parameter to the file's index in the search results. You can retrieve the total number of search results using M_FILE_NAME_FIND + M_NB_ELEMENTS.

## Parameters

Comp1ContextAppId

Specifies the identifier of the Aurora Imaging Library application context to use. The computer on which this application context has been allocated is referred to as either computer 1, or the source computer.

| For specifying the application context on Computer 1 | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the application context of the current process. |

| | |
|---|---|
| *Application context identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

Comp1FileName

Specifies the file on the target or source computer to use for the file operation, or the file name search pattern to use.

| For specifying the file on Computer 1 or the file name search pattern to find files on Computer 1 | |
|---|---|
| Value | Description |
| AIL_TEXT("*FileName*") [1] | Specifies the file name, and optionally the full file path drive and directory, of the file on the target or source computer to use for the operation. If you specify only the file name, the file is assumed to be in the working directory of the application. If the file is elsewhere on the computer, a path is required. For example, "*myFile*" or "*C:\myDirectory\myFile*".<br><br>The file name is always local to computer 1.<br><br>Optionally, the file name and/or path can be followed by executable arguments (command-line parameters) for the M_FILE_EXECUTE and M_FILE_DISPATCH operations. For example, "*C:\myDirectory\myProgram -c -r*". |
| AIL_TEXT("*FileNameSearchPattern*") [1] | Specifies the file name search pattern to use. This must include the path to the root folder of the search, and can also include the asterisk (*) and question mark (?) wildcard characters. An asterisk (*) matches 0 or more characters, while a question mark (?) matches exactly 1 character.<br><br>For example, the search string "*C:\DirectoryToSearch\Year_188?*Night*.bmp*" specifies to search only the immediate subdirectories of *C:\DirectoryToSearch* that have the name *Year188* plus exactly one additional character. Examples of files that would be found with this string include:<br><br>• *C:\DirectoryToSearch\Year_1889\The_Starry_Night.bmp*.<br>• *C:\DirectoryToSearch\Year_1888\Starry_Night_Over_the_Rhone.bmp*.<br>• *C:\DirectoryToSearch\Year_188W\Night.bmp*.<br><br>> The string must not end in a backslash (\). To find all file names in a directory, follow the path with an asterisk (*) (for example, "*C:\DirectoryToSearch**").<br><br>The file name search pattern is always local to computer 1. |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Comp2ContextAppId

Specifies the identifier of the application context that determines where to copy or move the file specified using Comp1FileName, during an M_FILE_COPY, M_FILE_COPY_AIL_DLL, M_FILE_COPY_AIL_USER_DLL, or M_FILE_MOVE operation. The computer on which this application context has been allocated is referred to as either computer 2, or the destination computer.

> Note that computer 2 can be the same as computer 1. In other words, the source and destination of the copied file can be different paths on the same computer.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Comp2FileName

Specifies the directory (and optionally a new file name) on computer 2 to which to copy the file specified using Comp1FileName. If this parameter is set to a file name and not a full path, the file is copied or moved to the working directory of the destination application.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the specified file, or the type of information to return from the search.

See the Parameter associations section for possible values that can be specified.

OperationFlag

Specifies additional settings for the operation or search. Set this parameter to **M_DEFAULT** if not used.

OperationDataPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_INT
- M_NULL

Specifies the address of the user data that is used by some operations. Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be returned.

## Parameter associations

The tables below list possible values for the **Comp2ContextAppId**, **Comp2FileName**, and **Operation** parameters and possible values returned to the **OperationDataPtr** parameter.

- For specifying the file operation
- For specifying to perform a file search

The following Operation and corresponding Comp2ContextAppId and Comp2FileName parameter settings can be specified to perform a file operation. For these settings, you must set Comp1FileName to an explicit file name, partial path, or full path.

Set unused parameters to **M_NULL**, except for OperationFlag which must be set to **M_DEFAULT**.

| For specifying the file operation | |
|---|---|
| Operation | Description |
| Comp2ContextAppId | |
| Comp2FileName | |

| | |
|---|---|
| OperationDataPtr<br>- Possible values returned | |
| M_FILE_COPY | Specifies to copy the file from the source computer to the destination computer. |
| **Comp2ContextAppId** | Specifies the application context that determines where to copy the file specified using Comp1FileName. The computer that will receive the copied file is referred to as the destination computer. |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application Context Identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |
| **Comp2FileName** | Specifies the directory, and optionally a new file name, on the destination computer in which to copy the file specified using Comp1FileName. |
| AIL_TEXT("*Drive\Path\[FileName]*") 1 | Specifies the drive, directory, and optionally the new file name, on the destination computer in which the file specified using Comp1FileName will be copied. For example, "*C:\myDirectory*" or "*C:\myDirectory\myNewFileName*".<br><br>When specifying a file, the file name is always local to computer 2.<br><br>When specifying a directory, the string must end in a directory separator (for example, "*C:\myDirectory*").<br><br>If no destination file name is specified, the source file name is used. |
| **OperationDataPtr** | Data type info: Data type: *Set this parameter to M_NULL* |
| M_FILE_COPY_AIL_DLL | Specifies to attempt to copy a file from the source computer to the runtime library subfolder of the installation folder on the destination computer; the specified application might not have write-access to this folder (in which case the operation fails). This is primarily useful for copying a compiled library that contains custom functions, created using the Function Development module (Mfunc...()).<br><br>You can check whether the file already exists in this folder using M_FILE_EXISTS_AIL_DLL. |
| | *Operating system specific* |
| | [This is only applicable to Windows]<br><br>This is the *DLL* subfolder of your installation folder (for example, *C:\Program Files\Aurora Imaging Library\<version number>\Library\DLL*).<br><br>This folder might not be write-accessible if the destination application, specified by the Comp2ContextAppId parameter, was launched without administrative privileges. To copy the DLL to a folder that does not require administrative privileges (but where it can still be loaded by the application), use M_FILE_COPY_AIL_USER_DLL instead. |

| | |
|---|---|
| | [This is only applicable to Linux]<br><br>This is the *library* subfolder of your installation folder (for example, */opt/aurora_imaging_library/<version>/library*).<br><br>This folder might not be write-accessible if the destination application, specified by the Comp2ContextAppId parameter, was launched by a user who is not part of the permissions group.<br><br>Before accessing the copied DSO, you must update the runtime library bindings using ldconfig. |
| **Comp2ContextAppId** | Specifies the application context that determines where to copy the file specified using Comp1FileName. The computer that will receive the copied file is referred to as the destination computer. |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application Context Identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |
| **Comp2FileName** | Specifies to rename the DLL copied to Computer 2. The drive and path are determined based on the location of the DLLs on Computer 2.<br><br>To keep the file name specified in Comp1FileName, set this parameter to **M_NULL**. |
| AIL_TEXT("*FileName*") 1 | Specifies a new file name for the copied DLL. |
| **OperationDataPtr** | Data type info: Data type: *Set this parameter to M_NULL* |
| M_FILE_COPY_AIL_USER_DLL | Specifies to copy a file from the source computer to the user runtime library folder on the destination computer; the specified application typically has write-access to this folder (under Windows). This is primarily useful for copying a compiled library that contains custom functions, created using the Function Development module (Mfunc...()).<br><br>You can check whether the file already exists in this folder using M_FILE_EXISTS_AIL_USER_DLL. |
| | *Operating system specific* |
| | [This is only applicable to Windows]<br><br>The path to this folder is *C:\Users\Public\Documents\Aurora Imaging Library\<version number>\Library\UserDLL*. |
| | [This is only applicable to Linux]<br><br>This setting is the same as M_FILE_COPY_AIL_DLL. The folder is not guaranteed to be write-accessible.<br><br>Before accessing the copied DSO, you must update the runtime library bindings using ldconfig. |

| | | |
|---|---|---|
| | **Comp2ContextAppId** | Specifies the application context that determines where to copy the file specified using Comp1FileName. The computer that will receive the copied file is referred to as the destination computer. |
| | M_DEFAULT | Specifies the application context of the current process. |
| | *Application Context Identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |
| | **Comp2FileName** | Specifies to rename the DLL copied to Computer 2. The drive and path are determined based on the location of the DLLs on Computer 2.<br><br>To keep the file name specified in Comp1FileName, set this parameter to **M_NULL**. |
| | AIL_TEXT("*FileName*") 1 | Specifies a new file name for the copied DLL. |
| | **OperationDataPtr** | Data type info: Data type: *Set this parameter to M_NULL* |
| M_FILE_DELETE | | Specifies to delete the specified file on Computer 1. |
| | **OperationDataPtr** | Data type info: Data type: *Set this parameter to M_NULL* |
| M_FILE_DELETE_DIR | | Specifies to delete a directory on Computer 1. The directory must already be empty, unless OperationFlag is set to M_RECURSIVE. |
| | **OperationFlag** | Specifies one of the following. |
| | M_DEFAULT | Specifies the default behavior. The directory is only deleted if it is empty. |
| | M_RECURSIVE | Specifies to delete the directory and its contents. |
| | **OperationDataPtr** | Data type info: Data type: *Set this parameter to M_NULL* |
| M_FILE_DISPATCH | | Specifies to execute the specified program on Computer 1 without waiting for the specified program to terminate on the source computer. When set to M_FILE_DISPATCH, Computer 1 cannot return an exit code.<br><br>Note that when set to M_FILE_DISPATCH, the Comp1FileName parameter can include executable arguments (command-line parameters).<br><br>The capability to execute programs on a remote computer has security implications. You are responsible for ensuring that the program executed by the remote Distributed Aurora Imaging Library application is in fact the program that you requested. **MappFileOperation**() cannot execute a program on a remote computer unless that computer is running a connected Distributed Aurora Imaging Library application. |
| | **OperationDataPtr** | Data type info: Data type: *Set this parameter to M_NULL* |

| | |
|---|---|
| M_FILE_EXECUTE | Specifies to execute the specified program on Computer 1. The computer initiating the operation will wait for the program to terminate. Optionally, you can retrieve the exit code of the program using OperationDataPtr.<br><br>Note that when set to M_FILE_EXECUTE, the Comp1FileName parameter can include executable arguments (command-line parameters).<br><br>The capability to execute programs on a remote computer has security implications. You are responsible for ensuring that the program executed by the remote Distributed Aurora Imaging Library application is in fact the program that you requested. **MappFileOperation**() cannot execute a program on a remote computer unless that computer is running a connected Distributed Aurora Imaging Library application. |
| **OperationDataPtr** | Data type info: Data type: *Set this parameter to M_NULL* |
| M_FILE_EXISTS | Specifies to determine whether the file specified by Comp1FileName exists on computer 1. |
| **OperationDataPtr** | Data type info: Data type: *address of an AIL_INT*<br><br>Returns one of the following. |
| M_NO | Specifies that the file does not exist. |
| M_YES | Specifies that the file does exist. |
| M_FILE_EXISTS_AIL_DLL | Specifies to determine whether the file specified by Comp1FileName exists in the runtime library subfolder of the installation folder on computer 1; the specified application might not have write-access to this folder. |
| | *Operating system specific* |
| | [This is only applicable to Windows]<br><br>This is the *DLL* subfolder of your installation folder (for example, *C:\Program Files\Aurora Imaging Library\<version number>\Library\DLL*). |
| | [This is only applicable to Linux]<br><br>This is the *library* subfolder of your installation folder (for example, */opt/aurora_imaging_library/<version>/library*). |
| **OperationDataPtr** | Data type info: Data type: *address of an AIL_INT*<br><br>Returns one of the following. |
| M_NO | Specifies that the file does not exist. |
| M_YES | Specifies that the file does exist. |
| M_FILE_EXISTS_AIL_USER_DLL | Specifies to determine whether the file specified by Comp1FileName exists in the user runtime library folder on computer 1; the specified application typically has write-access to this folder (under Windows). |

| | | |
|---|---|---|
| | | *Operating system specific* |
| | | [This is only applicable to Windows]<br><br>The path to this folder is *C:\Users\Public\Documents\Aurora Imaging Library\<version number>\Library\UserDLL*. |
| | | [This is only applicable to Linux]<br><br>This is the same as M_FILE_EXISTS_AIL_DLL. |
| | **OperationDataPtr** | Data type info: Data type: *address of an AIL_INT*<br><br>Returns one of the following. |
| | M_NO | Specifies that the file does not exist. |
| | M_YES | Specifies that the file does exist. |
| M_FILE_MAKE_DIR | | Specifies to create a directory on Computer 1. |
| | **OperationDataPtr** | Data type info: Data type: *Set this parameter to M_NULL* |
| M_FILE_MOVE | | Specifies to move the file from the source computer to the destination computer.<br><br>The file is only deleted from the source computer after it is successfully copied to the destination computer. |
| | **Comp2ContextAppId** | Specifies the application context that determines where to move the file specified using Comp1FileName. The computer that will receive the moved file is referred to as the destination computer. |
| | M_DEFAULT | Specifies the application context of the current process. |
| | *Application Context Identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |
| | **Comp2FileName** | Specifies the directory, and optionally a new file name, on the destination computer in which to move the file specified using Comp1FileName. |
| | AIL_TEXT("*Drive\Path\[FileName]*") 1 | Specifies the drive, directory, and optionally the new file name, on the destination computer in which the file specified using Comp1FileName will be moved. For example, "*C:\myDirectory*" or "*C:\myDirectory\myNewFileName*".<br><br>When specifying a file, the file name is always local to computer 2.<br><br>When specifying a directory, the string must end in a directory separator (for example, "*C:\myDirectory*").<br><br>If no destination file name is specified, the source file name is used. |

| | | |
|---|---|---|
| | **OperationDataPtr** | Data type info: Data type: *Set this parameter to M_NULL* |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

The following Operation parameter settings can be specified to perform a file search. For these settings, you must set Comp1FileName to a file name search pattern.

Set unused parameters to **M_NULL**, except for OperationFlag which must be set to **M_DEFAULT**.

| For specifying to perform a file search | | |
|---|---|---|
| Operation | | Description |
| | Comp2ContextAppId | |
| | Comp2FileName | |
| | OperationDataPtr<br>- Possible values returned | |
| M_FILE_NAME_FIND + | | Specifies to determine the name of a file found with the search pattern specified by the Comp1FileName parameter. Since the search might find more than one file, you must specify which file name to return by setting the OperationFlag parameter to the file's index in the search results. You can determine the total number of files that match the search pattern using M_FILE_NAME_FIND + M_NB_ELEMENTS.<br><br>The search is not recursive, and does not include the names of folders that match the search pattern. Only the file name is output, and not the full path.<br><br>Search results are not cached. If a file is added or deleted between calls to **MappFileOperation**(), the indexing of the search results might change. Therefore, if you want to iterate through the files found by a search and delete some of them, you should first iteratively call **MappFileOperation**() with M_FILE_NAME_FIND to store all of the search results in an array (or other data structure), and then delete the files by iterating through the stored results. |
| | **OperationFlag** | Specifies the following. |
| | 0 <= *Value* < M_FILE_NAME_FIND + M_NB_ELEMENTS | Specifies which file name to return, by index in the search results. |
| | **OperationDataPtr** | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* |

| | |
|---|---|
| M_FOLDER_NAME_FIND + | Specifies to determine the name of a folder found with the search pattern specified by the Comp1FileName parameter. Since the search might find more than one folder, you must specify which folder name to return by setting the OperationFlag parameter to the folder's index in the search results. You can determine the total number of folders that match the search pattern using M_FOLDER_NAME_FIND + M_NB_ELEMENTS.<br><br>The search is not recursive, and does not include the names of files that match the search pattern. Only the folder name is output, and not the full path.<br><br>Search results are not cached. If a folder is added or deleted between calls to **MappFileOperation**(), the indexing of the search results might change. Therefore, if you want to iterate through the folders found by a search and delete some of them, you should first iteratively call **MappFileOperation**() with M_FILE_NAME_FIND to store all of the search results in an array (or other data structure), and then delete the folders by iterating through the stored results. |
| **OperationFlag** | Specifies the following. |
| 0 <= *Value* < M_FOLDER_NAME_FIND + M_NB_ELEMENTS | Specifies which folder name to return, by index in the search results. |
| **OperationDataPtr** | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* |

Combination value for M_FILE_NAME_FIND; M_FOLDER_NAME_FIND.

You can add the following value to the above-mentioned values to get the string's length.

| For getting the string size | |
|---|---|
| Operation | Description |
| Comp2ContextAppId | |
| Comp2FileName | |
| OperationDataPtr<br>- Possible values returned | |
| M_STRING_SIZE | Retrieves the length of the string, including the terminating null character ("\0"). |
| **OperationDataPtr** | Data type info: Data type: *address of an AIL_INT* |

Combination value for M_FILE_NAME_FIND; M_FOLDER_NAME_FIND.

You can add the following value to the above-mentioned values to get the number of elements found with the search pattern specified by the Comp1FileName parameter.

| For getting the number of elements found with the search pattern | |
|---|---|
| Operation | Description |
| Comp2ContextAppId | |
| Comp2FileName | |

| | |
|---|---|
| OperationDataPtr - Possible values returned | |
| M_NB_ELEMENTS | Retrieves the number of elements found with the search pattern specified by the Comp1FileName parameter.<br><br>The search is not recursive, and the result will only include search pattern matches for the specified element type. |
| **OperationDataPtr** | Data type info: Data type: *address of an AIL_INT* |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappFree

## Synopsis

Free an application.

## Syntax

```
void MappFree(
    AIL_ID ContextAppId //in
)
```

## Description

This function deallocates an Aurora Imaging Library application previously allocated with MappAlloc().

**MappFree**() must be the last function called; no other function can be executed after a call to this function. Prior to freeing an application, ensure that all systems, buffers, displays, and digitizers allocated in the application are freed, unless M_UNIQUE_ID was specified during allocation.

Note, if you use MappAllocDefault() to allocate the default application, you must use MappFreeDefault() to free the application.

## Parameter

ContextAppId

Specifies the identifier of the application context to free.

## Remark

- If you are creating a DLL that includes a call to **MappFree**(), ensure that the call is not made from the `DllMain()` function, because **MappFree**() might unload any DLL loaded with MappAlloc() and you cannot unload a DLL from `DllMain()`. If necessary, call **MappFree**() from a clean-up function in your DLL instead.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappFreeDefault

## Synopsis

Free default objects.

## Syntax

```
void MappFreeDefault(
    AIL_ID ContextAppId, //in
    AIL_ID SysId,        //in
    AIL_ID DispId,       //in
    AIL_ID DigId,        //in
    AIL_ID ImageBufId    //in
)
```

## Description

**MappFreeDefault**() is implemented as a macro. This macro frees the default Aurora Imaging Library objects that were allocated with the MappAllocDefault() macro.

## Parameters

ContextAppId

Specifies the identifier of the application context to free.

SysId

Specifies the identifier of the system to free.

DispId

Specifies the identifier of the display to free. If set to **M_NULL**, no display is freed.

DigId

Specifies the identifier of the digitizer to free. If set to **M_NULL**, no digitizer is freed.

ImageBufId

Specifies the identifier of the image buffer to free. If set to **M_NULL**, no buffer is freed.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappGetError

## Synopsis

Get error codes and related information.

## Syntax

```
AIL_INT MappGetError(
    AIL_ID ContextAppId, //in
    AIL_INT64 ErrorType, //in
    void *ErrorPtr       //out
)
```

## Description

This function obtains current or global application error information (for example, error codes, error subcodes, error messages, error submessages, and function opcodes). This function allows you to check for errors after each Aurora Imaging Library function call or to get the first error that occurred after a series of Aurora Imaging Library function calls.

A typical use of this function is to check whether a buffer allocation call was successful (MbufAllocColor(), MbufAlloc1d(), and MbufAlloc2d()) or to see if a function generates an error.

In multi-thread environments, a call to **MappGetError**() returns the last error that occurred in the current thread.

This function can also be used to obtain information about a detected error when error-reporting to the screen has been disabled using MappControl().

To clear errors, call MappControl() with M_CLEAR_ERROR.

## Parameters

ContextAppId

Specifies the identifier of the application context to use.

| For specifying the application context | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application context identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

ErrorType

Specifies the type of error information to read. This parameter can be set to one of the values below.

See the Parameter associations section for possible values that can be specified.

ErrorPtr

Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_INT

Specifies the address in which to write the requested information.

The **MappGetError**() function also returns the error/function code; if you don't want to get an error message, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

## Parameter associations

The table below lists possible values for the **ErrorType** parameter and possible values returned to the **ErrorPtr** parameter.

- For specifying the type of error information to read

| For specifying the type of error information to read | |
|---|---|
| ErrorType | Description |
| ErrorPtr<br>- Possible values returned | |
| M_CURRENT + | Retrieves the error code returned by the last function call. The current-error code is reset to **M_NULL_ERROR** before each function call and is set to a specific error code when an error occurs during the execution of a function. |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_CURRENT_OPCODE + | Retrieves the opcode associated with the last function called, when it returns an error.<br><br>To learn which function corresponds to each opcode, refer to *ailfunctioncode.h*. |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_CURRENT_SUB_1 + | Retrieves the first error subcode returned by the last function call.<br><br>Note, when there is no error, the error subcode is set to **M_NULL_ERROR**. |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_CURRENT_SUB_2 + | Retrieves the second error subcode returned by the last function call.<br><br>Note, when there is no error, the error subcode is set to **M_NULL_ERROR**. |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_CURRENT_SUB_3 + | Retrieves the third error subcode returned by the last function call.<br><br>Note, when there is no error, the error subcode is set to **M_NULL_ERROR**. |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

| | |
|---|---|
| M_CURRENT_SUB_NB | Retrieves the number of error subcodes associated with the last function called, when it returns an error. |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_GLOBAL + | Retrieves the error code of the first error that has occurred since the last call to **MappGetError**() (with ErrorType set to M_GLOBAL). This value is reset to **M_NULL_ERROR** before the next call to a function (other than **MappGetError**()). |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_GLOBAL_OPCODE + | Retrieves the opcode associated with the first error-generating function since the last call to **MappGetError**() (with ErrorType set to M_GLOBAL).<br><br>To learn which function corresponds to each opcode, refer to *ailfunctioncode.h*. |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_GLOBAL_SUB_1 + | Retrieves the first error subcode of the first error that has occurred since the call to **MappGetError**() (with ErrorType set to M_GLOBAL).<br><br>Note, when there is no error, the error subcode is set to **M_NULL_ERROR**. |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_GLOBAL_SUB_2 + | Retrieves the second error subcode of the first error that has occurred since the call to **MappGetError**() (with ErrorType set to M_GLOBAL).<br><br>Note, when there is no error, the error subcode is set to **M_NULL_ERROR**. |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_GLOBAL_SUB_3 + | Retrieves the third error subcode of the first error that has occurred since the call to **MappGetError**() (with ErrorType set to M_GLOBAL).<br><br>Note, when there is no error, the error subcode is set to **M_NULL_ERROR**. |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_GLOBAL_SUB_NB | Retrieves the number of error subcodes associated with the first error that occurred since the last call to **MappGetError**() (with ErrorType set to M_GLOBAL). |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

Combination values for M_CURRENT; M_CURRENT_OPCODE; M_CURRENT_SUB_1; M_CURRENT_SUB_2; M_CURRENT_SUB_3; M_GLOBAL; M_GLOBAL_OPCODE; M_GLOBAL_SUB_1; M_GLOBAL_SUB_2; M_GLOBAL_SUB_3.

You can add one of the following values to the above-mentioned values to set whether to return a message or to make the inquire synchronous.

| For error types | |
|---|---|
| ErrorType | Description |

| | |
|---|---|
| ErrorPtr<br>- Possible values returned | |
| M_MESSAGE + | Retrieves the error message associated with the specified error type, or the function name associated with a function opcode, and writes it to the address specified by the ErrorPtr parameter. The error/function opcode is still returned by **MappGetError**(), but it is not written to the address specified by ErrorPtr.<br><br>If the message is truncated, use M_MESSAGE_EXTENDED for the complete message.<br><br>Note that this value cannot be added to M_CURRENT_SUB_NB or M_GLOBAL_SUB_NB. |
| *ErrorPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* |
| M_MESSAGE_EXTENDED + | Retrieves the error message associated with the specified error type, or the function name associated with a function opcode, and writes it to the address specified by the ErrorPtr parameter. The error/function opcode is still returned by **MappGetError**(), but it is not written to the address specified by ErrorPtr.<br><br>Note that this value cannot be added to M_CURRENT_SUB_NB or M_GLOBAL_SUB_NB. |
| *ErrorPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* |
| M_SYNCHRONOUS | Forces the function to wait for all pending calls to complete before continuing. |

Combination value for M_MESSAGE; M_MESSAGE_EXTENDED.

You can add the following value to the above-mentioned values to get the message's length.

| For getting the message size | |
|---|---|
| ErrorType | Description |
| ErrorPtr<br>- Possible values returned | |
| M_STRING_SIZE | Retrieves the length of the string, including the terminating null character ("\0"). |
| *ErrorPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

## Return value

The returned value is the requested error/function code. If the error code is read and it is equal to **M_NULL_ERROR**, no error has occurred.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappGetHookInfo

## Synopsis

Get information about a hooked event.

## Syntax

```
AIL_INT MappGetHookInfo(
    AIL_ID ContextAppId, //in
    AIL_ID EventId,      //in
    AIL_INT64 InfoType,  //in
    void *UserVarPtr     //out
)
```

## Description

This function retrieves information about the event that caused the hook-handler function to be called. This function should only be called within the scope of an application hook-handler function call (see MappHookFunction()).

## Parameters

ContextAppId

Specifies the identifier of the Aurora Imaging Library application context to use.

| For specifying the application context | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application context identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

EventId

Specifies the application event identifier received by the hook-handler function. See MappHookFunction() for more information.

InfoType

Specifies the type of information to get.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE

- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64
- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- a pointer of type void

Specifies the address in which to write the requested information. Note that, when getting parameter values (**MappGetHookInfo**() with M_PARAM_VALUE), the UserVarPtr parameter should be of the same data type as value of the selected parameter.

See the Parameter associations section for possible values that can be returned.

## Parameter associations

The tables below list possible values for the **InfoType** parameter and possible values returned to the **UserVarPtr** parameter.

- For retrieving information about an M_ERROR_CURRENT, M_ERROR_FATAL or M_CALLEE_ERROR_CURRENT event type
- For retrieving information about an M_ERROR_CURRENT, M_ERROR_FATAL, M_CALLEE_ERROR_CURRENT, M_TRACE_START, or M_TRACE_END event type
- For retrieving information about an M_OBJECT_PUBLISH_DAIL event type
- For retrieving information about an M_WEB_CLIENT_CONNECTED or M_WEB_CLIENT_DISCONNECTED event type
- For retrieving information about an M_TRACE_START or M_TRACE_END event type

If the hook-handler function was called due to an M_ERROR_CURRENT, M_ERROR_FATAL, or M_CALLEE_ERROR_CURRENT event type, the InfoType parameter can be set to one of the values below.

| For retrieving information about an M_ERROR_CURRENT, M_ERROR_FATAL or M_CALLEE_ERROR_CURRENT event type | |
|---|---|
| InfoType | Description |
| UserVarPtr - Possible values returned | |
| M_CURRENT + | Retrieves the error code associated with the error event that caused the hook-handler function to be called. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_CURRENT_SUB_1 + | Retrieves the first error subcode associated with the error event that caused the hook-handler function to be called. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_CURRENT_SUB_2 + | Retrieves the second error subcode associated with the error event that caused the hook-handler function to be called. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_CURRENT_SUB_3 + | Retrieves the third error subcode associated with the error event that caused the hook-handler function to be called. |

| | |
|---|---|
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_CURRENT_SUB_NB | Retrieves the number of error subcodes associated with the error event that caused the hook-handler function to be called. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

If the hook-handler function was called due to an M_ERROR_CURRENT, M_ERROR_FATAL, M_TRACE_START, or M_TRACE_END event type, the InfoType parameter can be set to the value below.

| For retrieving information about an M_ERROR_CURRENT, M_ERROR_FATAL, M_CALLEE_ERROR_CURRENT, M_TRACE_START, or M_TRACE_END event type | |
|---|---|
| InfoType | Description |
| UserVarPtr<br>- Possible values returned | |
| M_CURRENT_OPCODE + | Retrieves the function opcode associated with the error event that caused the hook-handler function to be called (for M_ERROR_CURRENT, M_ERROR_FATAL, M_CALLEE_ERROR_CURRENT), just started (for M_TRACE_START), or just ended (for M_TRACE_END).<br><br>To learn which function corresponds to each opcode, refer to *ailfunctioncode.h*. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

Combination values for the values listed in For retrieving information about an M_ERROR_CURRENT, M_ERROR_FATAL, M_CALLEE_ERROR_CURRENT, M_TRACE_START, or M_TRACE_END event type; and for the following values: M_CURRENT; M_CURRENT_SUB_1; M_CURRENT_SUB_2; M_CURRENT_SUB_3.

You can add one of the following values to the above-mentioned values to get the associated message instead of the numeric code.

| For getting the associated message instead of the numeric code | |
|---|---|
| InfoType | Description |
| UserVarPtr<br>- Possible values returned | |
| M_MESSAGE + | Returns the error message instead of the error opcode, or the function name instead of the function opcode.<br><br>If the message is truncated, use M_MESSAGE_EXTENDED for the complete message. |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* |
| M_MESSAGE_EXTENDED + | Returns the error message instead of the error opcode, or the function name instead of the function opcode. |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* |

Combination value for M_MESSAGE; M_MESSAGE_EXTENDED.

You can add the following value to the above-mentioned values to get the message's length.

| For getting the message size | |
|---|---|
| InfoType | Description |
| UserVarPtr<br>- Possible values returned | |
| M_STRING_SIZE | Retrieves the length of the string, including the terminating null character ("\0"). |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

If the hook-handler function was called due to an M_OBJECT_PUBLISH_DAIL event type, the InfoType parameter can be set to the value below.

| For retrieving information about an M_OBJECT_PUBLISH_DAIL event type | |
|---|---|
| InfoType | Description |
| UserVarPtr<br>- Possible values returned | |
| M_OBJECT_ID | Retrieves the identifier of the object whose publication state was modified. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

If the hook-handler function was called due to an M_WEB_CLIENT_CONNECTED or M_WEB_CLIENT_DISCONNECTED event type, the InfoType parameter can be set to the value below.

| For retrieving information about an M_WEB_CLIENT_CONNECTED or M_WEB_CLIENT_DISCONNECTED event type | |
|---|---|
| InfoType | Description |
| UserVarPtr<br>- Possible values returned | |
| M_WEB_CLIENT_INDEX | Retrieves the index that uniquely identifies the instance of your Aurora Imaging Web Library client application that connected or disconnected. This index refers to the same instance of the client application, as long as that instance is connected. If the instance disconnects and then reconnects, it is assigned a new index.<br><br>Indices of clients are not zero- or one-based, nor are they guaranteed to be sequential. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

If the hook-handler function was called due to an M_TRACE_START or M_TRACE_END event type, the InfoType parameter can be set to one of the values below.

| For retrieving information about an M_TRACE_START or M_TRACE_END event type | |
|---|---|
| InfoType | Description |
| UserVarPtr<br>- Possible values returned | |
| M_PARAM_NB | Retrieves the number of parameters associated with the function called. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

| | | |
|---|---|---|
| M_PARAM_SIZE + | | Retrieves the size, in bytes, of the specified parameter. For an array parameter, the size is: *Number of items in the array* X *Size of the item*.<br><br>You must specify a combination value from the following table:<br><br>• To determine the number of the parameter for which to receive the information |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| M_PARAM_TYPE_INFO + | | Retrieves the data type of the specified parameter of the function called.<br><br>You must specify a combination value from the following table:<br><br>• To determine the number of the parameter for which to receive the information |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| | M_PARAM_TYPE_AIL_DOUBLE | Specifies that the parameter is of type *AIL_DOUBLE*. |
| | M_PARAM_TYPE_AIL_FLOAT | Specifies that the parameter is of type *AIL_FLOAT*. |
| | M_PARAM_TYPE_AIL_ID | Specifies that the parameter is of type *AIL_ID*. |
| | M_PARAM_TYPE_AIL_INT | Specifies that the parameter is of type *AIL_INT*. |
| | M_PARAM_TYPE_AIL_INT8 | Specifies that the parameter is of type *AIL_INT8*. |
| | M_PARAM_TYPE_AIL_INT16 | Specifies that the parameter is of type *AIL_INT16*. |
| | M_PARAM_TYPE_AIL_INT32 | Specifies that the parameter is of type *AIL_INT32*. |
| | M_PARAM_TYPE_AIL_INT64 | Specifies that the parameter is of type *AIL_INT64*. |
| | M_PARAM_TYPE_AIL_TEXT | Specifies that the parameter is of type *AIL_TEXT*. This implies that the character encoding scheme of the *AIL_TEXT* parameter is the scheme you are currently using. |
| | M_PARAM_TYPE_AIL_TEXT_ASCII | Specifies that the parameter is of type *AIL_TEXT_ASCII*. This only appears when you are currently using a character encoding scheme other than ASCII, and the parameter you are inquiring is specifically in ASCII. |
| | M_PARAM_TYPE_AIL_TEXT_UNICODE | Specifies that the parameter is of type *AIL_TEXT_UNICODE*. This only appears when you are currently using a character encoding scheme other than unicode, and the parameter you are inquiring is specifically in unicode. |

| | | |
|---|---|---|
| | M_PARAM_TYPE_AIL_UINT | Specifies that the parameter is of type *AIL_UINT*. |
| | M_PARAM_TYPE_AIL_UINT8 | Specifies that the parameter is of type *AIL_UINT8*. |
| | M_PARAM_TYPE_AIL_UINT16 | Specifies that the parameter is of type *AIL_UINT16*. |
| | M_PARAM_TYPE_AIL_UINT32 | Specifies that the parameter is of type *AIL_UINT32*. |
| | M_PARAM_TYPE_AIL_UINT64 | Specifies that the parameter is of type *AIL_UINT64*. |
| | M_PARAM_TYPE_ARRAY_AIL_DOUBLE | Specifies that the parameter is an array of type *AIL_DOUBLE*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_FLOAT | Specifies that the parameter is an array of type *AIL_FLOAT*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_ID | Specifies that the parameter is an array of type *AIL_ID*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_INT | Specifies that the parameter is an array of type *AIL_INT*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_INT8 | Specifies that the parameter is an array of type *AIL_INT8*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_INT16 | Specifies that the parameter is an array of type *AIL_INT16*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_INT32 | Specifies that the parameter is an array of type *AIL_INT32*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_INT64 | Specifies that the parameter is an array of type *AIL_INT64*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_UINT | Specifies that the parameter is an array of type *AIL_UINT*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |

| | | |
|---|---|---|
| | M_PARAM_TYPE_ARRAY_AIL_UINT8 | Specifies that the parameter is an array of type *AIL_UINT8*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_UINT16 | Specifies that the parameter is an array of type *AIL_UINT16*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_UINT32 | Specifies that the parameter is an array of type *AIL_UINT32*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_ARRAY_AIL_UINT64 | Specifies that the parameter is an array of type *AIL_UINT64*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_AIL_TEXT | Specifies that the parameter is a constant of type *AIL_TEXT*. This implies that the character encoding scheme of the *AIL_TEXT* parameter is the scheme you are currently using. |
| | M_PARAM_TYPE_CONST_AIL_TEXT_ASCII | Specifies that the parameter is a constant of type *AIL_TEXT_ASCII*. This only appears when you are currently using a character encoding scheme other than ASCII, and the parameter you are inquiring is specifically in ASCII. |
| | M_PARAM_TYPE_CONST_AIL_TEXT_UNICODE | Specifies that the parameter is a constant of type *AIL_TEXT_UNICODE*. This only appears when you are currently using a character encoding scheme other than unicode, and the parameter you are inquiring is specifically in unicode. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_DOUBLE | Specifies that the parameter is a constant array of type *AIL_DOUBLE*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_FLOAT | Specifies that the parameter is a constant array of type *AIL_FLOAT*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_ID | Specifies that the parameter is a constant array of type *AIL_ID*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_INT | Specifies that the parameter is a constant array of type *AIL_INT*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_INT8 | Specifies that the parameter is a constant array of type *AIL_INT8*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |

| | | |
|---|---|---|
| | M_PARAM_TYPE_CONST_ARRAY_AIL_INT16 | Specifies that the parameter is a constant array of type *AIL_INT16*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_INT32 | Specifies that the parameter is a constant array of type *AIL_INT32*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_INT64 | Specifies that the parameter is a constant array of type *AIL_INT64*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_UINT | Specifies that the parameter is a constant array of type *AIL_UINT*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_UINT8 | Specifies that the parameter is a constant array of type *AIL_UINT8*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_UINT16 | Specifies that the parameter is a constant array of type *AIL_UINT16*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_UINT32 | Specifies that the parameter is a constant array of type *AIL_UINT32*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_ARRAY_AIL_UINT64 | Specifies that the parameter is a constant array of type *AIL_UINT64*. Note that the pointer to the data stored in the array is unavailable after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_CONST_DATA_PTR | Specifies that the parameter is a constant data pointer. Note that the pointer to the data is invalid after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_DATA_PTR | Specifies that the parameter is a data pointer. Note that the pointer to the data is invalid after exiting the hook-handler function. For future use, copy and save the data. |
| | M_PARAM_TYPE_FILENAME | Specifies that the parameter is of type *FILENAME*. This implies that the character encoding scheme of the *FILENAME* parameter is the scheme you are currently using. |
| | M_PARAM_TYPE_FILENAME_ASCII | Specifies that the parameter is of type *FILENAME_ASCII*. This only appears when you are currently using a character encoding scheme other than ASCII, and the parameter you are inquiring is specifically in ASCII. |

| | | |
|---|---|---|
| | M_PARAM_TYPE_FILENAME_UNICODE | Specifies that the parameter is of type *FILENAME_UNICODE*. This only appears when you are currently using a character encoding scheme other than unicode, and the parameter you are inquiring is specifically in unicode. |
| M_PARAM_VALUE + | | Retrieves the value of the specified parameter of the function called.<br><br>You must specify a combination value from the following table:<br><br>• To determine the number of the parameter for which to receive the information |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* **\|** *address of an AIL_ID* **\|** *address of an AIL_INT32* **\|** *address of an AIL_INT64* **\|** *address of an AIL_TEXT_PTR [optionally, in C++: AIL_STRING ]* **\|** *address of an void* |

Combination value for M_PARAM_SIZE; M_PARAM_TYPE_INFO; M_PARAM_VALUE.

You must add the following value to the above-mentioned values to determine the number of the parameter for which to receive the information.

| For specifying the parameter | |
|---|---|
| Combination value | Description |
| 1 <= *Value* <= 16 | Specifies the number of the parameter for which to retrieve the information. This value must be a positive integer, and should not exceed the number of parameters associated with the function call (M_PARAM_NB). |

## Return value

The returned value is **M_NULL** if successful. If the operation fails, a non-null (!**M_NULL**) value is returned.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappHookFunction

## Synopsis

Hook a function to an event.

## Syntax

```
void MappHookFunction(
    AIL_ID ContextAppId,                      //in
    AIL_INT HookType,                         //in
    AIL_APP_HOOK_FUNCTION_PTR HookHandlerPtr, //in
    void *UserDataPtr                         //in-out
)
```

## Description

This function allows you to attach or detach a user-defined function to a specified application event. Once a hook-handler function is defined and hooked to an event, it is automatically called when the event occurs.

You can hook more than one function to an event by making separate calls to **MappHookFunction**() for each function that you want to hook. These hooked functions are automatically chained and kept in an internal list. When a function is hooked, this new function is added to the end of the list. When the event happens, all user-defined functions in the list are executed in the same order that they were hooked to the event. You can also remove any function from the list; in this case, the order of the remaining functions in the list is preserved.

You can obtain information concerning the event, using MappGetHookInfo(), and take appropriate action before returning control to the application.

In multi-thread environments, an **MappHookFunction**() call hooks or unhooks the function specified by HookHandlerPtr to all application threads running Aurora Imaging Library, unless specifically limited to the calling thread.

This function is typically used to trap errors that occur in an application, without checking every function execution with MappGetError(), or to detect the start or end of certain functions.

## Parameters

ContextAppId

Specifies the identifier of the application context to use.

| For specifying the application context | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application context identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

HookType

Specifies the event type. This parameter can be set to one of the following values.

| For specifying the event type | |
|---|---|
| Value | Description |
| M_CALLEE_ERROR_CURRENT + | Calls the hook-handler function each time an error occurs in a user-defined function. Note that hook-handler functions attached to this type of event are called first before the error dialog occurs (if enabled using MappControl() with M_ERROR). It is also called before any error handling code within the user-defined function. |
| M_ERROR_CURRENT + | Calls the hook-handler function each time an error occurs. Note that hook-handler functions attached to this type of event are called first before the error dialog occurs (if enabled using MappControl() with M_ERROR). |
| M_ERROR_FATAL + | Calls the hook-handler function when a user presses the **Cancel** button in an error dialog. For this event to occur, MappControl() with M_ERROR must be set to M_PRINT_ENABLE. |
| M_OBJECT_PUBLISH_DAIL + | Calls the hook-handler function when an object to be used with Distributed Aurora Imaging Library has its publication state modified. This event occurs when the publication state is modified using MobjControl() with M_DAIL_PUBLISH, or when the published object is freed as its publication state was set to M_NO. |
| M_TRACE_END + | Calls the hook-handler function at the end of each function.<br><br>When setting the HookType parameter to M_TRACE_END, the only function that can be called in a hook function is MappGetHookInfo(), otherwise infinite recursion will occur. |
| M_TRACE_START + | Calls the hook-handler function at the start of each function.<br><br>When setting the HookType parameter to M_TRACE_START, the only function that can be called in a hook function is MappGetHookInfo(), otherwise infinite recursion will occur. |
| M_WEB_CLIENT_CONNECTED + | Calls the hook-handler function when an Aurora Imaging Web Library client application connects to your application.<br><br>Before an Aurora Imaging Web Library client can connect to your application, you must enable Aurora Imaging Web Library functionality using MappControl() with M_WEB_CONNECTION and M_ENABLE. |
| M_WEB_CLIENT_DISCONNECTED + | Calls the hook-handler function when an Aurora Imaging Web Library client application disconnects from your application. |

Combination value for any of the possible values of the HookType parameter.

You can add the following value to the above-mentioned values to set whether to detach the hook-handler function.

| For the HookType parameter | |
|---|---|
| Combination value | Description |
| M_UNHOOK | Unhooks the specified function if hooked to the specified event. When you use M_UNHOOK, you must provide the same values for all the parameters as when you originally hooked the function. |

Combination value for any of the possible values of the HookType parameter.

You can add the following value to the above-mentioned values to set whether to limit the hooking or unhooking of the function to the calling thread.

| For hooking or unhooking the function to application threads | |
|---|---|
| Combination value | Description |
| M_THREAD_CURRENT | Limits the hooking or unhooking of the function to the calling thread. For example, to call the hook-handler function only for errors occurring in the current thread, specify M_ERROR_CURRENT + M_THREAD_CURRENT as the HookType parameter. |

HookHandlerPtr

Specifies the address of the function that should be called when an event occurs.

The hook-handler function, pointed to by HookHandlerPtr, must be declared as follows:

```
AIL_INT MFTYPE HookHandler(
    AIL_INT HookType,
    AIL_ID EventId,
    void *UserDataPtr
)
```

Parameters

HookType

Type of event hooked.

EventId

Event identifier to pass to MappGetHookInfo() when inquiring about the hooked event.

UserDataPtr

Specifies the user data pointer passed to **MappHookFunction**().

Upon successful completion, the hook-handler function should return **M_NULL**. Note, MFTYPE and AIL_APP_HOOK_FUNCTION_PTR are reserved predefined types for functions and data pointers, respectively.

UserDataPtr

Specifies the address of the user data that you want to make available to the hook-handler function. This address is passed to the hook-handler function, through its UserDataPtr parameter, when the specified event occurs. Set this parameter to **M_NULL** if user data is not required.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappInquire

## Synopsis

Inquire about an application setting.

## Syntax

```
AIL_INT MappInquire(
    AIL_ID ContextAppId,   //in
    AIL_INT64 InquireType, //in
    void *UserVarPtr       //out
)
```

## Description

This function inquires about the specified application setting.

## Parameters

ContextAppId

Specifies the identifier of the application context to use.

| For specifying the application context | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application context identifier* | Specifies the application context of the current process.<br><br>Typically specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

InquireType

Specifies the type of application setting about which to inquire. This parameter can be set to one of the values below.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_ID
- AIL_INT

Specifies the address in which to write the requested information. Since the **MappInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

## Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about general types of application settings
- For inquiring system fingerprints

The following inquire types allow you to inquire about general types of application settings.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For inquiring about general types of application settings | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType<br>UserVarPtr<br>- Possible values returned | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_AIL_DIRECTORY_CONTEXTS + | Inquires the path to the local contexts directory. The returned value is equivalent to the string constant **M_CONTEXT_PATH**.<br>By default on a Windows platform, this path is *C:\Program Files\Aurora Imaging Library\<version number>\Library\Contexts*.<br>By default on a Linux platform, this path is */opt/aurora_imaging_library/<version>/contexts*. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_AIL_DIRECTORY_EXAMPLES + | Inquires the path to the local examples directory.<br>By default on a Windows platform, this path is *C:\Program Files\Aurora Imaging Library\<version number>\Library\Examples*.<br>By default on a Linux platform, this path is */opt/aurora_imaging_library/<version>/examples*. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_AIL_DIRECTORY_IMAGES + | Inquires the path to the local images directory. The returned value is equivalent to the string constant **M_IMAGE_PATH**.<br>By default on a Windows platform, this path is *C:\Program Files\Aurora Imaging Library\<version number>\Library\Images*.<br>By default on a Linux platform, this path is */opt/aurora_imaging_library/<version>/images*. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_AIL_DIRECTORY_INSTALL + | Inquires the path to the root directory of the local Aurora Imaging Library installation. The returned value is equivalent to the string constant **M_INSTALL_DIR**.<br><br>By default on a Windows platform, this path is *C:\Program Files\Aurora Imaging Library\<version number>\Library*.<br><br>By default on a Linux platform, this path is */opt/aurora_imaging_library/<version>/library*. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_COMPONENT_AUTO_RESELECT + | Inquires whether image buffer components are automatically reselected to 2D displays when they are reallocated.<br><br>This setting does not affect containers shown in 3D displays.<br><br>If M_THREAD_CURRENT was specified, this inquires the setting for the current thread. Otherwise, it inquires the setting of the application environment. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies that the global application setting is used for the current thread. Note that this value is only returned if M_THREAD_CURRENT was specified. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_COMPONENT_AUTO_RESELECT_DISABLE | Specifies not to automatically reselect image buffer components to 2D displays. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_COMPONENT_AUTO_RESELECT_ENABLE | Specifies to automatically reselect image buffer components to 2D displays when they are reallocated. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_COMPONENT_AUTO_RESELECT_ENABLED | Inquires whether M_COMPONENT_AUTO_RESELECT is enabled for the current thread.<br><br>This always returns whether M_COMPONENT_AUTO_RESELECT is enabled for the current thread (regardless of whether it is enabled because it was set for the application environment, or because you enabled it explicitly for the thread using MappControl() with M_COMPONENT_AUTO_RESELECT + M_THREAD_CURRENT).<br><br>To inquire the setting for the application environment (instead of the current thread), use M_COMPONENT_AUTO_RESELECT instead. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_NO | Specifies that M_COMPONENT_AUTO_RESELECT is not enabled in the current thread. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_YES | Specifies that M_COMPONENT_AUTO_RESELECT is enabled in the current thread. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_CURRENT_APPLICATION | Inquires the identifier of the current Aurora Imaging Library application, if any. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_ID* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| 0 | Specifies that no application is allocated. Note that no error is generated. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Application identifier* | Specifies the identifier of the current application. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_DAIL_CONNECTION | Inquires the Distributed Aurora Imaging Library monitoring configuration permission level. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_DAIL_CONTROL | Specifies that objects in the current application can be accessed with either read only or read/write permission, depending on how that object was specified with MobjControl(). | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_DAIL_MONITOR | Specifies that objects in the current application can be accessed only with read only permission, regardless of how that object was specified with MobjControl(). | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_DISABLE | Specifies that Distributed Aurora Imaging Library monitoring configuration is disabled in the current application. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_DAIL_CONNECTION_PORT | Inquires the listening port of a publishing application in the Distributed Aurora Imaging Library monitoring configuration. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Listening port* | Specifies the listening port (an integer between 0 and 65535) that the current application will use to listen for connections from a monitoring application. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_ERROR | Inquires the error printing mode in effect for the current thread.<br><br>Typically, you should not use this setting if ContextAppId specifies an application context allocated on a remote computer, accessed using Distributed Aurora Imaging Library. The result will return information about a thread on the remote computer, rather than the current thread. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_PRINT_DISABLE | Specifies not to print error messages. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_PRINT_ENABLE | Specifies to print error messages. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_THROW_EXCEPTION | Specifies that when an error occurs, a .NET exception will be thrown instead of a message box being displayed.<br><br>> M_THROW_EXCEPTION is only valid when working in a .NET environment. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_ERROR_HOOKS | Inquires whether errors generated in the current thread will trigger the user-defined function that you specified using MappHookFunction() (with M_ERROR_CURRENT or M_ERROR_FATAL). SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_DISABLE | Specifies to not trigger the user-defined function (hooked using MappHookFunction()) when an error event occurs. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_ENABLE | Specifies to trigger the user-defined function when an error event occurs, as specified in MappHookFunction(). | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_GENTL_PRODUCER_COUNT | Inquires the number of different GenTL producers installed during Aurora Imaging Library installation. | | | e | | | | | | | | | | | | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | | | e | | | | | | | | | | | | b |
| *Value* >= 0 | Specifies the number of different GenTL producers installed. | | | e | | | | | | | | | | | | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_GENTL_PRODUCER_DESCRIPTOR + *n* + | Inquires the path and file name of the *n* th GenTL producer, where *n* is a number between 0 and the value returned by M_GENTL_PRODUCER_COUNT. | | | e | | | | | | | | | | | | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | | | e | | | | | | | | | | | | b |
| AIL_TEXT("*PathAndFilename*") 1 | Specifies the path and file name of the specified GenTL producer. | | | e | | | | | | | | | | | | b |
| M_INSTALLED_SYSTEM_COUNT | Inquires the number of different system types (drivers) installed during Aurora Imaging Library installation and the number of Distributed Aurora Imaging Library systems that are registered. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Value* | Specifies the number of different system types installed. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_INSTALLED_SYSTEM_DESCRIPTOR + *n* + | Inquires the descriptor of system type *n*, where *n* is a number between 0 and the value returned by M_INSTALLED_SYSTEM_COUNT. The descriptor can be passed to MsysAlloc() to allocate the system. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_INSTALLED_SYSTEM_DEVICE_COUNT + *n* | Inquires the number of installed devices of the same type as system type n, where n is a number between 0 and the value returned by M_INSTALLED_SYSTEM_COUNT. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_UNKNOWN | Specifies that the number of devices is unknown. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Value* >= 0 | Specifies the number of devices of the specified type. Note that although the driver for a system type is installed, this inquire type returns 0 if a corresponding physical device is not installed. In addition, for a GigE system type and USB3 system type, this inquire type will return 1. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_INSTALLED_SYSTEM_PRINT_NAME + *n* + | Inquires the printable name of system type *n* (for example, Zebra Radient), where *n* is a number between 0 and the value returned by M_INSTALLED_SYSTEM_COUNT. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | *String* | Specifies the printable name of system type *n*. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_INSTALLED_SYSTEM_TYPE + *n* | | Inquires the type of system type *n*, where *n* is a number between 0 and the value returned by M_INSTALLED_SYSTEM_COUNT.<br><br>If a Distributed Aurora Imaging Library remote system has been registered, you can use the following macros to further inquire about the system: SET<br><br>• To determine whether the system is a Distributed Aurora Imaging Library remote system, use the **M_IS_SYSTEM_DISTRIBUTED** macro. This macro returns a non-zero value if the system is a remote system and zero if the system is local.<br><br>• To remove the bit flag for Distributed Aurora Imaging Library remote systems, use the **M_REAL_SYSTEM_TYPE** macro. This macro returns only the system type and not the system type and remote system bit flag. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_SYSTEM_CLARITY_UHD_TYPE + | Specifies an Aurora Imaging Library Clarity UHD system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_SYSTEM_CONCORD_POE_TYPE + | Specifies an Aurora Imaging Library Concord POE system. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_SYSTEM_GENTL_TYPE + | Specifies an Aurora Imaging Library GenTL system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_SYSTEM_GEVIQ_TYPE + | Specifies an Aurora Imaging Library GevIQ system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_SYSTEM_GIGE_VISION_TYPE + | Specifies an Aurora Imaging Library GigE Vision system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_SYSTEM_HOST_TYPE + | Specifies the Host. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_SYSTEM_INDIO_TYPE + | Specifies an Aurora Imaging Library Indio system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_SYSTEM_IRIS_GTX_TYPE + | Specifies an Aurora Imaging Library Iris GTX system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_SYSTEM_RADIENTEVCL_TYPE + | Specifies an Aurora Imaging Library Radient eV-CL system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_SYSTEM_RAPIXOCL_TYPE + | Specifies an Aurora Imaging Library Rapixo Pro CL system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_SYSTEM_RAPIXOCOF_TYPE + | Specifies an Aurora Imaging Library Rapixo CoF system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_SYSTEM_RAPIXOCXP_TYPE + | Specifies an Aurora Imaging Library Rapixo CXP system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_SYSTEM_USB3_VISION_TYPE + | Specifies an Aurora Imaging Library USB3 Vision system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_SYSTEM_V4L2_TYPE + | Specifies an Aurora Imaging Library Video4Linux2 system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_KEY_SERIAL_NUMBER + *n* + | Inquires the serial number of the *n* th Wibu-Systems dongle, where *n* is a number between 0 and the value returned by M_NUMBER_OF_KEYS. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *String* | Specifies the inquired Wibu-Systems dongle serial number. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_LICENSE_MODULES | Inquires the modules for which there is a valid license. The value returned will be a bitwise combination of the following. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_LICENSE_3DCA | Specifies a Camera Calibration module license that supports 3D-based camera calibration modes. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_LICENSE_3DSUP | Specifies a 3D Supplement module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_LICENSE_BEAD | Specifies a Bead module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_LICENSE_BLOB | Specifies a Blob analysis module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_LICENSE_CAL | Specifies a Camera Calibration module license that supports 2D-based camera calibration modes. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_LICENSE_CLASS | Specifies a Classification module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_CODE | Specifies a Code read/write module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_COL | Specifies a Color Analysis module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_COM | Specifies an industrial communication license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_DAIL | Specifies a Distributed Aurora Imaging Library license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_DEBUG | Specifies a development license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_DMR | Specifies a SureDotOCR module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_EDGE | Specifies an Edge Finder module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_IM | Specifies an Image Processing module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_INTERFACE | Specifies a Zebra GigE Vision driver license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_JPEG2000 | Specifies a JPEG2000 compression license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_JPEGSTD | Specifies a JPEG compression license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_LITE | Specifies an Aurora Imaging Library Lite module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_MEAS | Specifies a Measurement module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_MET | Specifies a Metrology module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_MOD | Specifies a Geometric Model Finder module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_OCR | Specifies an OCR module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_PAT | Specifies a Pattern Matching module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | M_LICENSE_REG | Specifies a Registration module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_LICENSE_STR | Specifies a String Reader module license. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_MEMORY | Inquires the memory compensation mode in effect for the current thread. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_COMPENSATION_DISABLE | Specifies that on-board memory compensation is disabled. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_COMPENSATION_ENABLE | Specifies that on-board memory compensation is enabled. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_NON_PAGED_MEMORY_FREE | Inquires the amount of free non-paged memory reserved for Aurora Imaging Library (at installation time or using the Aurora Imaging Configurator utility). | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Value* | Specifies the amount of free non-paged memory, in bytes. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_NON_PAGED_MEMORY_LARGEST_FREE | Inquires the size of the largest contiguous block of free non-paged memory reserved for Aurora Imaging Library (at installation time or using the Aurora Imaging Configurator utility).<br><br>You cannot allocate a buffer larger than this size using an MbufAlloc...() function, even if the total amount of free non-paged memory is sufficient. For more information, see the Large non-paged Aurora Imaging Library buffers subsection of the Advanced memory management section of Chapter 23: Data buffers. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Value* | Specifies the size of the largest contiguous block of free non-paged memory reserved for Aurora Imaging Library. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_NON_PAGED_MEMORY_SIZE | Inquires the amount of the non-paged memory (DMA) reserved for Aurora Imaging Library. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Value* | Specifies the amount of the non-paged memory (DMA) reserved for Aurora Imaging Library, in bytes. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_NON_PAGED_MEMORY_USED | Inquires the amount of non-paged memory being used. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Value* | Specifies the amount of non-paged memory being used, in bytes. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_NUMBER_OF_KEYS | Inquires the number of Wibu-Systems dongles detected. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Value* | Specifies the number of Wibu-Systems dongles detected. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_PARAMETER | Inquires the parameter checking mode in effect for the current thread. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_CHECK_DISABLE | Specifies that checking of parameters is disabled. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_CHECK_ENABLE | Specifies that checking of parameters is enabled. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_PLATFORM_BITNESS | Inquires the bitness for which the application has been compiled. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| 32 | Specifies that the specified application is 32-bit. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| 64 | Specifies that the specified application is 64-bit. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_PLATFORM_OS_TYPE | Inquires the operating system under which the specified application is running. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_OS_LINUX | Specifies that the specified application is running under a Linux operating system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_OS_WINDOWS | Specifies that the specified application is running under a Microsoft Windows operating system. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_PROCESSING | Inquires the processing compensation mode in effect for the current thread. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_COMPENSATION_DISABLE | Specifies that on-board processing compensation is disabled. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_COMPENSATION_ENABLE | Specifies that on-board processing compensation is enabled. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_SELECTED_VERSION | Inquires the selected version of Aurora Imaging Library. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Value* | Specifies the version number of Aurora Imaging Library. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_TRACE + | Inquires the trace setting, which is used to start or stop generating a trace log.<br><br>Note that this inquire type can return M_DEFAULT, a trace setting which can signify either to start or to stop generating a trace log. Use M_TRACE_ACTIVE to determine if a trace log is currently being generated. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies the default trace setting, which either enables or disables generating a trace log, based on whether Aurora Imaging Profiler (or the Interactive log) was used to start generating the log. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_LOG_DISABLE | Specifies the log disable setting, which stops generating the trace log. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_LOG_ENABLE | Specifies the log enable setting, which starts generating the trace log. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_TRACE_ACTIVE | Inquires whether a trace log is currently being generated in the current thread.<br><br>Note that adding the combination constant M_THREAD_CURRENT to this inquire will result in an error. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_NO | Specifies that a trace log is not currently being generated in the current thread. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_YES | Specifies that a trace log is currently being generated in the current thread. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_VERSION | Inquires the version of Aurora Imaging Library. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *Value* | Specifies the version of Aurora Imaging Library. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_VERSION_STRING + | Inquires the version of Aurora Imaging Library, as a string. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| AIL_TEXT("*nn.nn.nnnn*") 1 | Specifies the version of Aurora Imaging Library, as a string. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_WEB_CONNECTION | Inquires whether Aurora Imaging Web Library server functionality is enabled. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_DISABLE | Specifies that Aurora Imaging Web Library server functionality is disabled in the current application. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_ENABLE | Specifies that Aurora Imaging Web Library server functionality is enabled in the current application. | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_WEB_CONNECTION_PORT | Inquires the listening port used for Aurora Imaging Web Library server functionality. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| 0 <= *Value* <=65535 | Specifies the port number. INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| M_XORG_ACCELERATION | [This is only applicable to Linux]<br><br>Inquires whether Aurora Imaging Library can use X11 (Xorg) acceleration for display under Linux. This determines the hardware acceleration display mode. SET | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 0 | Specifies that X11 acceleration cannot be used.<br>INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |
| | 1 | Specifies that X11 acceleration can be used.<br>INFO | c | d | e | f | g | a | h | i | | j | k | l | m | n | b |

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

Combination value for [when InquireType=M_INSTALLED_SYSTEM_TYPE + n: M_SYSTEM_CLARITY_UHD_TYPE; M_SYSTEM_CONCORD_POE_TYPE; M_SYSTEM_GENTL_TYPE; M_SYSTEM_GEVIQ_TYPE; M_SYSTEM_GIGE_VISION_TYPE; M_SYSTEM_HOST_TYPE; M_SYSTEM_INDIO_TYPE; M_SYSTEM_IRIS_GTX_TYPE; M_SYSTEM_RADIENTEVCL_TYPE; M_SYSTEM_RAPIXOCL_TYPE; M_SYSTEM_RAPIXOCOF_TYPE; M_SYSTEM_RAPIXOCXP_TYPE; M_SYSTEM_USB3_VISION_TYPE; M_SYSTEM_V4L2_TYPE].

The following value might be combined with the above-mentioned values to determine if the inquired system type is remote.

By default, an installed systems that is remote will have a distributed flag appended to the returned system type.

| For inquiring if a system is remote | |
|---|---|
| Combination value | Description |
| M_SYSTEM_DISTRIBUTED_FLAG | Specifies that the system is remote. |

Combination value for M_COMPONENT_AUTO_RESELECT; M_TRACE.

You can add the following value to the above-mentioned values to specify that the inquire should be limited to the current thread.

In multi-thread environments, by default, you can inquire about an application setting from any thread.

| For M_TRACE | |
|---|---|
| Combination value | Description |
| M_THREAD_CURRENT | Specifies that data to be returned should relate to the current thread. |

Combination value for M_AIL_DIRECTORY_CONTEXTS; M_AIL_DIRECTORY_EXAMPLES; M_AIL_DIRECTORY_IMAGES; M_AIL_DIRECTORY_INSTALL; M_GENTL_PRODUCER_DESCRIPTOR + n; M_INSTALLED_SYSTEM_DESCRIPTOR + n; M_INSTALLED_SYSTEM_PRINT_NAME + n; M_KEY_SERIAL_NUMBER + n; M_VERSION_STRING.

You can add the following value to the above-mentioned values to get the string's length.

| For inquiring the size of a string | |
|---|---|
| InquireType | Description |
| UserVarPtr<br>- Possible values returned | |

| | |
|---|---|
| M_STRING_SIZE | Inquires the number of characters in the string. This number accounts for every character, including the terminating null character. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

When inquired and present, the requested system fingerprint is returned. If that specific fingerprint is not present, 0 is returned. For more information about system fingerprints, see the Protecting your own Aurora Imaging Library software application using a Zebra hardware fingerprint section of Chapter 69: Distribution and licensing.

| For inquiring system fingerprints | |
|---|---|
| InquireType | Description |
| UserVarPtr<br>- Possible values returned | |
| M_ANY_FINGERPRINT + | Inquires any hardware fingerprint. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| 0 | Specifies that no hardware that has been correctly installed has a fingerprint. |
| *Value* | Specifies the hardware fingerprint. |
| M_CONCORD_GIGE_FINGERPRINT + | Inquires the hardware fingerprint of a Zebra Concord GigE Vision board. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| 0 | Specifies that the board is not present in the computer, or that the board is incorrectly installed. |
| *Value* | Specifies the hardware fingerprint. |
| M_CONCORDPOE_FINGERPRINT + | Inquires the hardware fingerprint of a Zebra Concord PoE board. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| 0 | Specifies that the board is not present in the computer, or that the board is incorrectly installed. |
| *Value* | Specifies the hardware fingerprint. |
| M_ID_KEY_FINGERPRINT + | Inquires the hardware fingerprint of a hardware identification key. The hardware identification key is the device, which when connected to your USB or parallel port, you can use as a hardware fingerprint to generate a software license. |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |

| | | |
|---|---|---|
| | 0 | Specifies that the hardware identification key is not connected to the computer, or that it is not correctly installed. |
| | *Value* | Specifies the hardware fingerprint. |
| M_INDIO_FINGERPRINT + | | Inquires the hardware fingerprint of a Zebra Indio board. |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| | 0 | Specifies that the board is not present in the computer, or that the board is incorrectly installed. |
| | *Value* | Specifies the hardware fingerprint. |
| M_IRISGTX_FINGERPRINT + | | Inquires the hardware fingerprint of a Zebra Iris GTX smart camera. |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| | 0 | Specifies that the current computer is not a Zebra Iris GTX smart camera. |
| | *Value* | Specifies the hardware fingerprint. |
| M_MSERIES_FINGERPRINT + | | Inquires the hardware fingerprint of a Zebra M-Series graphics board. |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| | 0 | Specifies that the board is not present in the computer, or that the board is incorrectly installed. |
| | *Value* | Specifies the hardware fingerprint. |
| M_RADIENT_FINGERPRINT + | | Inquires the hardware fingerprint of a Zebra Radient board. |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| | 0 | Specifies that the board is not present in the computer, or that the board is incorrectly installed. |
| | *Value* | Specifies the hardware fingerprint. |
| M_RAPIXOCXP_FINGERPRINT + | | Inquires the hardware fingerprint of a Zebra Rapixo CXP board. |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_INT* |
| | 0 | Specifies that the board is not present in the computer, or that the board is incorrectly installed. |

| | | |
|---|---|---|
| | *Value* | Specifies the hardware fingerprint. |

Combination value for the values listed in For inquiring system fingerprints.

You can add the following value to the above-mentioned values to specify which Zebra product to take the fingerprint from when there are more than one available.

When multiple fingerprints of the same type are present, all similar fingerprints can be inquired using the following.

| For specifying which Zebra product to take a fingerprint from | |
|---|---|
| Combination value | Description |
| 0 <= *Value* < 16 | Specifies the number of fingerprints of the same type that can be inquired about. |

## Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappInquireConnection

## Synopsis

Inquire about published objects on a Distributed Aurora Imaging Library publishing application.

## Syntax

```
AIL_INT MappInquireConnection(
    AIL_ID ContextAppId,   //in
    AIL_INT64 InquireType, //in
    AIL_INT64 ControlFlag, //in
    AIL_INT64 ExtraFlag,   //in
    void *UserVarPtr       //out
)
```

## Description

This function inquires about the published Aurora Imaging Library objects on a Distributed Aurora Imaging Library publishing application. A monitoring application will use this function to generate a list of the published objects on a publishing application. With this list, you can determine the identifier of a specific object. With this identifier, you can perform any operation on the object that its permission level allows.

## Parameters

ContextAppId

Specifies the identifier of the application context to use.

| For specifying the application context | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application context identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

InquireType

Specifies the type of Distributed Aurora Imaging Library connection setting about which to inquire. This parameter can be set to one of the values below.

> Note that display objects cannot be published, and so do not appear as a possible return value.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies an attribute of the inquire operation to perform.

If this parameter is not required for the inquire operation, it must be set to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

ExtraFlag

Specifies an attribute of the inquire operation to perform.

If this parameter is not required for the inquire operation, it must be set to **M_DEFAULT**.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the parameter.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID> ]
- AIL_ID
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MappInquireConnection**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

## Parameter associations

The table below lists possible values for the **InquireType**, **ControlFlag**, and **ExtraFlag** parameters and possible values returned to the **UserVarPtr** parameter.

- For inquiring about a Distributed Aurora Imaging Library monitoring configuration connection

Set unused parameters to **M_DEFAULT**.

| For inquiring about a Distributed Aurora Imaging Library monitoring configuration connection | |
|---|---|
| InquireType | Description |
| ControlFlag | |
| ExtraFlag | |
| UserVarPtr<br>- Possible values returned | |
| M_DAIL_PUBLISHED_LIST | Inquires the list of identifiers associated with the published objects in a Distributed Aurora Imaging Library publishing application.<br><br>Published objects are those objects set to read or read/write. |
| **ControlFlag** | Specifies to limit the returned list from all objects to all objects of the specified object type.<br><br>> Note that a display object cannot be published, and so is not a possible object type with which to limit the returned list. |

| | | | |
|---|---|---|---|
| | | M_DEFAULT | Specifies to include all object types in the Distributed Aurora Imaging Library publishing application. |
| | | M_3D_DISPLAY | Specifies an Aurora Imaging Library 3D display allocated using M3ddispAlloc(). |
| | | M_3D_DISPLAY_PICKING_AREA_CONTEXT | Specifies an Aurora Imaging Library picking area context allocated using M3ddispAlloc() with M_PICKING_AREA_CONTEXT. |
| | | M_3D_DISPLAY_PICKING_AREA_RESULT | Specifies an Aurora Imaging Library picking area result buffer allocated using M3ddispAllocResult() with M_PICKING_AREA_RESULT. |
| | | M_3D_DISPLAY_PICKING_CONTEXT | Specifies an Aurora Imaging Library picking context allocated using M3ddispAlloc() with M_PICKING_CONTEXT. |
| | | M_3D_DISPLAY_PICKING_RESULT | Specifies an Aurora Imaging Library picking result buffer allocated using M3ddispAllocResult() with M_PICKING_RESULT. |
| | | M_3D_GRAPHIC_LIST | Specifies an Aurora Imaging Library 3D graphics list allocated using M3dgraAlloc(). |
| | | M_3DBLOB_CALCULATE_CONTEXT | Specifies an Aurora Imaging Library 3D blob context that can be used with M3dblobCalculate(), allocated using M3dblobAlloc() with M_CALCULATE_CONTEXT. |
| | | M_3DBLOB_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library 3D blob context that can be used with M3dblobDraw3d(), allocated using M3dblobAlloc() with M_DRAW_3D_CONTEXT. |
| | | M_3DBLOB_SEGMENTATION_CONTEXT | Specifies an Aurora Imaging Library 3D blob context that can be used with M3dblobSegment(), allocated using M3dblobAlloc() with M_SEGMENTATION_CONTEXT. |
| | | M_3DBLOB_SEGMENTATION_RESULT | Specifies an Aurora Imaging Library 3D blob result buffer for segmentation results, allocated using M3dblobAllocResult() with M_SEGMENTATION_RESULT. |
| | | M_3DGEO_GEOMETRY | Specifies an Aurora Imaging Library 3D geometry object allocated using M3dgeoAlloc() with M_GEOMETRY. |
| | | M_3DGEO_TRANSFORMATION_MATRIX | Specifies an Aurora Imaging Library 3D transformation matrix object allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX. |
| | | M_3DIM_CALCULATE_MAP_SIZE_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimCalculateMapSize(), allocated using M3dimAlloc() with M_CALCULATE_MAP_SIZE_CONTEXT. |

| | | | |
|---|---|---|---|
| | | M_3DIM_FILL_GAPS_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimFillGaps(), allocated using M3dimAlloc() with M_FILL_GAPS_CONTEXT. |
| | | M_3DIM_FILTER_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimFilter(), allocated using M3dimAlloc() with M_FILTER_CONTEXT. |
| | | M_3DIM_FIND_TRANSFORMATION_RESULT | Specifies an Aurora Imaging Library 3D image processing result buffer for find transformation results, allocated using M3dimAllocResult() with M_FIND_TRANSFORMATION_RESULT. |
| | | M_3DIM_LATTICE_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimLattice(), allocated using M3dimAlloc() with M_LATTICE_CONTEXT. |
| | | M_3DIM_LATTICE_RESULT | Specifies an Aurora Imaging Library 3D image processing result buffer for lattice results, allocated using M3dimAllocResult() with M_LATTICE_RESULT. |
| | | M_3DIM_MESH_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimMesh(), allocated using M3dimAlloc() with M_MESH_CONTEXT. |
| | | M_3DIM_NORMALS_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimNormals(), allocated using M3dimAlloc() with M_NORMALS_CONTEXT. |
| | | M_3DIM_OUTLIERS_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimOutliers(), allocated using M3dimAlloc() with M_OUTLIERS_CONTEXT. |
| | | M_3DIM_PROFILE_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimProfileEx(), allocated using M3dimAlloc() with M_PROFILE_CONTEXT. |
| | | M_3DIM_PROFILE_RESULT | Specifies an Aurora Imaging Library 3D image processing result buffer for profile results, allocated using M3dimAllocResult() with M_PROFILE_RESULT. |
| | | M_3DIM_PROJECT_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimProjectEx(), allocated using M3dimAlloc() with M_PROJECT_CONTEXT. |
| | | M_3DIM_REMAP_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimRemapDepthMap(), allocated using M3dimAlloc() with M_REMAP_CONTEXT. |

| | | | |
|---|---|---|---|
| | | M_3DIM_STATISTICS_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimStat(), allocated using M3dimAlloc() with M_STATISTICS_CONTEXT. |
| | | M_3DIM_STATISTICS_RESULT | Specifies an Aurora Imaging Library 3D image processing result buffer used to store M3dimStat() results, and is allocated using M3dimAllocResult() with M_STATISTICS_RESULT. |
| | | M_3DIM_STITCH_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimMerge(), allocated using M3dimAlloc() with M_STITCH_CONTEXT. |
| | | M_3DIM_SUBSAMPLE_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimSample(), allocated using M3dimAlloc() with M_SUBSAMPLE_CONTEXT. |
| | | M_3DIM_SURFACE_SAMPLE_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimSample(), allocated using M3dimAlloc() with M_SURFACE_SAMPLE_CONTEXT. |
| | | M_3DMAP_ALIGN_CONTEXT | Specifies an Aurora Imaging Library 3D align context allocated using M3dmapAlloc() with M_ALIGN_CONTEXT. |
| | | M_3DMAP_ALIGN_RESULT | Specifies an Aurora Imaging Library 3D align result buffer allocated using M3dmapAllocResult() with M_ALIGN_RESULT. |
| | | M_3DMAP_DEPTH_CORRECTED_DATA | Specifies an Aurora Imaging Library 3D reconstruction result buffer allocated using M3dmapAllocResult() with M_DEPTH_CORRECTED_DATA. |
| | | M_3DMAP_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library 3D draw context allocated using M3dmapAlloc() with M_DRAW_3D_CONTEXT. |
| | | M_3DMAP_LASER_CALIBRATION_DATA | Specifies an Aurora Imaging Library 3D reconstruction result buffer allocated using M3dmapAllocResult() with M_LASER_CALIBRATION_DATA. |
| | | M_3DMAP_LASER_CONTEXT | Specifies an Aurora Imaging Library 3D reconstruction context allocated using M3dmapAlloc() with M_LASER. |
| | | M_3DMAP_POINT_CLOUD_RESULT | Specifies an Aurora Imaging Library 3D reconstruction result buffer allocated using M3dmapAllocResult() with M_POINT_CLOUD_RESULT. |
| | | M_3DMEAS_DRAW_3D_PATH_CONTEXT | Specifies an Aurora Imaging Library draw path 3D measurement context allocated using M3dmeasAlloc() with M_DRAW_3D_PATH_CONTEXT. |

| | | | |
|---|---|---|---|
| | | M_3DMEAS_DRAW_3D_PROFILE_CONTEXT | Specifies an Aurora Imaging Library draw profile 3D measurement context allocated using M3dmeasAlloc() with M_DRAW_3D_PROFILE_CONTEXT. |
| | | M_3DMEAS_DRAW_3D_TEMPLATE_CONTEXT | Specifies an Aurora Imaging Library draw template 3D measurement context allocated using M3dmeasAlloc() with M_DRAW_3D_TEMPLATE_CONTEXT. |
| | | M_3DMEAS_FIND_MARKER_PATH_CONTEXT | Specifies an Aurora Imaging Library path 3D measurement context allocated using M3dmeasAlloc() with M_FIND_MARKER_PATH_CONTEXT. |
| | | M_3DMEAS_FIND_MARKER_PATH_RESULT | Specifies an Aurora Imaging Library path 3D measurement result allocated using M3dmeasAllocResult() with M_FIND_MARKER_PATH_RESULT. |
| | | M_3DMEAS_FIND_MARKER_PROFILE_CONTEXT | Specifies an Aurora Imaging Library profile 3D measurement context allocated using M3dmeasAlloc() with M_FIND_MARKER_PROFILE_CONTEXT. |
| | | M_3DMEAS_FIND_MARKER_PROFILE_RESULT | Specifies an Aurora Imaging Library profile 3D measurement result allocated using M3dmeasAllocResult() with M_FIND_MARKER_PROFILE_RESULT. |
| | | M_3DMEAS_FIND_MARKER_TEMPLATE_CONTEXT | Specifies an Aurora Imaging Library template 3D measurement context allocated using M3dmeasAlloc() with M_FIND_MARKER_TEMPLATE_CONTEXT. |
| | | M_3DMEAS_FIND_MARKER_TEMPLATE_RESULT | Specifies an Aurora Imaging Library template 3D measurement result allocated using M3dmeasAllocResult() with M_FIND_MARKER_TEMPLATE_RESULT. |
| | | M_3DMEAS_FIT_CONTEXT | Specifies an Aurora Imaging Library fit 3D measurement context allocated using M3dmeasAlloc() with M_FIT_CONTEXT. |
| | | M_3DMET_CALCULATE_RESULT | Specifies an Aurora Imaging Library calculate 3D metrology result buffer allocated using M3dmetAllocResult() with M_CALCULATE_RESULT. |
| | | M_3DMET_DISTANCE_CONTEXT | Specifies an Aurora Imaging Library distance 3D metrology context allocated using M3dmetAlloc() with M_DISTANCE_CONTEXT. |
| | | M_3DMET_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library draw 3D metrology context allocated using M3dmetAlloc() with M_DRAW_3D_CONTEXT. |
| | | M_3DMET_FIT_CONTEXT | Specifies an Aurora Imaging Library fit 3D metrology context allocated using M3dmetAlloc() with M_FIT_CONTEXT. |

| | | | |
|---|---|---|---|
| | | M_3DMET_FIT_RESULT | Specifies an Aurora Imaging Library fit 3D metrology result buffer allocated using M3dmetAllocResult() with M_FIT_RESULT. |
| | | M_3DMET_STATISTICS_CONTEXT | Specifies an Aurora Imaging Library statistics 3D metrology context allocated using M3dmetAlloc() with M_STATISTICS_CONTEXT. |
| | | M_3DMET_STATISTICS_RESULT | Specifies an Aurora Imaging Library statistics 3D metrology result buffer allocated using M3dmetAllocResult() with M_STATISTICS_RESULT. |
| | | M_3DMET_VOLUME_CONTEXT | Specifies an Aurora Imaging Library calculate volume 3D metrology context allocated using M3dmetAlloc() with M_VOLUME_CONTEXT. |
| | | M_3DMOD_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library draw 3D model finder context that can be allocated using M3dmodAlloc() with M_DRAW_3D_GEOMETRIC_CONTEXT (to draw occurrences of geometric models). |
| | | M_3DMOD_DRAW_3D_SURFACE_CONTEXT | Specifies an Aurora Imaging Library draw 3D model finder context that can be allocated using M3dmodAlloc() with M_DRAW_3D_SURFACE_CONTEXT (to draw occurrences of surface models). |
| | | M_3DMOD_FIND_BOX_CONTEXT | Specifies an Aurora Imaging Library box 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_BOX_CONTEXT. |
| | | M_3DMOD_FIND_BOX_RESULT | Specifies an Aurora Imaging Library box 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_BOX_RESULT. |
| | | M_3DMOD_FIND_CYLINDER_CONTEXT | Specifies an Aurora Imaging Library cylinder 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_CYLINDER_CONTEXT. |
| | | M_3DMOD_FIND_CYLINDER_RESULT | Specifies an Aurora Imaging Library cylinder 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_CYLINDER_RESULT. |
| | | M_3DMOD_FIND_PLANAR_SURFACE_CONTEXT | Specifies an Aurora Imaging Library planar surface 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_PLANAR_SURFACE_CONTEXT. |
| | | M_3DMOD_FIND_PLANAR_SURFACE_RESULT | Specifies an Aurora Imaging Library planar surface 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_PLANAR_SURFACE_RESULT. |

| | | | |
|---|---|---|---|
| | | M_3DMOD_FIND_RECTANGULAR_PLANE_CONTEXT | Specifies an Aurora Imaging Library rectangular plane 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_RECTANGULAR_PLANE_CONTEXT. |
| | | M_3DMOD_FIND_RECTANGULAR_PLANE_RESULT | Specifies an Aurora Imaging Library rectangular plane 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_RECTANGULAR_PLANE_RESULT. |
| | | M_3DMOD_FIND_SPHERE_CONTEXT | Specifies an Aurora Imaging Library sphere 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_SPHERE_CONTEXT. |
| | | M_3DMOD_FIND_SPHERE_RESULT | Specifies an Aurora Imaging Library sphere 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_SPHERE_RESULT. |
| | | M_3DMOD_FIND_SURFACE_CONTEXT | Specifies an Aurora Imaging Library surface 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_SURFACE_CONTEXT. |
| | | M_3DMOD_FIND_SURFACE_RESULT | Specifies an Aurora Imaging Library surface 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_SURFACE_RESULT. |
| | | M_3DREG_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library draw 3D registration context that can be used with M3dregDraw3d(), allocated using M3dregAlloc() with M_DRAW_3D_CONTEXT. |
| | | M_3DREG_PAIRWISE_REGISTRATION_CONTEXT | Specifies an Aurora Imaging Library pairwise 3D registration context allocated using M3dregAlloc() with M_PAIRWISE_REGISTRATION_CONTEXT. |
| | | M_3DREG_PAIRWISE_REGISTRATION_RESULT | Specifies an Aurora Imaging Library pairwise 3D registration result buffer allocated using M3dregAllocResult() with M_PAIRWISE_REGISTRATION_RESULT. |
| | | M_AGM_EDGE_BASED_FIND_CONTEXT | Specifies an Aurora Imaging Library Advanced Geometric Matching context allocated using MagmAlloc() with M_GLOBAL_EDGE_BASED_FIND. |
| | | M_AGM_EDGE_BASED_FIND_RESULT | Specifies an Aurora Imaging Library Advanced Geometric Matching result buffer allocated using MagmAllocResult() with M_GLOBAL_EDGE_BASED_FIND_RESULT. |
| | | M_AGM_EDGE_BASED_TRAIN_CONTEXT | Specifies an Aurora Imaging Library Advanced Geometric Matching context allocated using MagmAlloc() with M_GLOBAL_EDGE_BASED_TRAIN. |

| | | | |
|---|---|---|---|
| | | M_AGM_EDGE_BASED_TRAIN_RESULT | Specifies an Aurora Imaging Library Advanced Geometric Matching result buffer allocated using MagmAllocResult() with M_GLOBAL_EDGE_BASED_TRAIN_RESULT. |
| | | M_APPLICATION | Specifies an Aurora Imaging Library application context allocated using MappAlloc(). |
| | | M_ARRAY | Specifies an Aurora Imaging Library array buffer allocated using MbufAlloc2d() with M_ARRAY. |
| | | M_AUGMENTATION_RESULT | Specifies an Aurora Imaging Library augmentation result buffer allocated using MimAllocResult() with M_AUGMENTATION_RESULT. |
| | | M_BEAD_CONTEXT | Specifies an Aurora Imaging Library bead context allocated using MbeadAlloc(). |
| | | M_BEAD_RESULT | Specifies an Aurora Imaging Library bead result buffer allocated using MbeadAllocResult(). |
| | | M_BLOB_CONTEXT | Specifies an Aurora Imaging Library blob analysis context allocated using MblobAlloc(). |
| | | M_BLOB_RESULT | Specifies an Aurora Imaging Library blob analysis result buffer allocated using MblobAllocResult(). |
| | | M_CAL_CALCULATE_HAND_EYE_CONTEXT | Specifies an Aurora Imaging Library calculate hand-eye context (hand-eye calibration) allocated using McalAlloc() with M_CALCULATE_HAND_EYE_CONTEXT. |
| | | M_CAL_CALCULATE_HAND_EYE_RESULT | Specifies an Aurora Imaging Library calculate hand-eye result buffer allocated using McalAllocResult() with M_CALCULATE_HAND_EYE_RESULT. |
| | | M_CAL_CONTEXT | Specifies an Aurora Imaging Library camera calibration context allocated using McalAlloc(). |
| | | M_CAL_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library 3D draw calibration context allocated using McalAlloc() with M_DRAW_3D_CONTEXT. |
| | | M_CAL_FIXTURING_OFFSET | Specifies an Aurora Imaging Library fixturing offset object allocated using McalAlloc() with M_FIXTURING_OFFSET. |
| | | M_CLASS_CLASSIFIER_ANO_PREDEFINED_CONTEXT | Specifies an Aurora Imaging Library predefined anomaly detection classifier context, allocated using MclassAlloc() with M_CLASSIFIER_ANO_PREDEFINED. |

| | | | |
|---|---|---|---|
| | | M_CLASS_CLASSIFIER_CNN_PREDEFINED_CONTEXT | Specifies an Aurora Imaging Library predefined CNN classifier context, allocated using MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED. |
| | | M_CLASS_CLASSIFIER_DET_PREDEFINED_CONTEXT | Specifies an Aurora Imaging Library predefined object detection classifier context, allocated using MclassAlloc() with M_CLASSIFIER_DET_PREDEFINED. |
| | | M_CLASS_CLASSIFIER_ONNX_CONTEXT | Specifies an Aurora Imaging Library ONNX classifier context, allocated using MclassAlloc() with M_CLASSIFIER_ONNX. |
| | | M_CLASS_CLASSIFIER_SEG_PREDEFINED_CONTEXT | Specifies an Aurora Imaging Library predefined segmentation classifier context, allocated using MclassAlloc() with M_CLASSIFIER_SEG_PREDEFINED. |
| | | M_CLASS_CLASSIFIER_TREE_ENSEMBLE_CONTEXT | Specifies an Aurora Imaging Library tree ensemble classifier context, allocated using MclassAlloc() with M_CLASSIFIER_TREE_ENSEMBLE. |
| | | M_CLASS_DATASET_FEATURES | Specifies an Aurora Imaging Library features dataset context, allocated using MclassAlloc() with M_DATASET_FEATURES. |
| | | M_CLASS_DATASET_IMAGES | Specifies an Aurora Imaging Library images dataset context, allocated using MclassAlloc() with M_DATASET_IMAGES. |
| | | M_CLASS_PREDICT_ANO_RESULT | Specifies an Aurora Imaging Library anomaly detection prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_ANO_RESULT. |
| | | M_CLASS_PREDICT_CNN_RESULT | Specifies an Aurora Imaging Library CNN prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_CNN_RESULT. |
| | | M_CLASS_PREDICT_DET_RESULT | Specifies an Aurora Imaging Library object detection prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_DET_RESULT. |
| | | M_CLASS_PREDICT_ONNX_RESULT | Specifies an ONNX prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_ONNX_RESULT. |
| | | M_CLASS_PREDICT_SEG_RESULT | Specifies an Aurora Imaging Library segmentation prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_SEG_RESULT. |
| | | M_CLASS_PREDICT_TREE_ENSEMBLE_RESULT | Specifies an Aurora Imaging Library tree ensemble prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_TREE_ENSEMBLE_RESULT. |
| | | M_CLASS_PREPARE_IMAGES_CNN | Specifies an Aurora Imaging Library CNN data preparation context, allocated using MclassAlloc() with M_PREPARE_IMAGES_CNN. |

| | | | |
|---|---|---|---|
| | | M_CLASS_PREPARE_IMAGES_DET | Specifies an Aurora Imaging Library object detection data preparation context, allocated using MclassAlloc() with M_PREPARE_IMAGES_DET. |
| | | M_CLASS_PREPARE_IMAGES_SEG | Specifies an Aurora Imaging Library segmentation data preparation context, allocated using MclassAlloc() with M_PREPARE_IMAGES_SEG. |
| | | M_CLASS_STAT_ANO_CONTEXT | Specifies an Aurora Imaging Library statistics classification context for anomaly detection, allocated using MclassAlloc() with M_STAT_ANO. |
| | | M_CLASS_STAT_ANO_RESULT | Specifies an Aurora Imaging Library statistics classification result buffer for anomaly detection, allocated using MclassAllocResult() with M_STAT_ANO_RESULT. |
| | | M_CLASS_STAT_CNN_CONTEXT | Specifies an Aurora Imaging Library statistics classification context for image classification, allocated using MclassAlloc() with M_STAT_CNN. |
| | | M_CLASS_STAT_CNN_RESULT | Specifies an Aurora Imaging Library statistics classification result buffer for image classification, allocated using MclassAllocResult() with M_STAT_CNN_RESULT. |
| | | M_CLASS_STAT_DET_CONTEXT | Specifies an Aurora Imaging Library statistics classification context for object detection, allocated using MclassAlloc() with M_STAT_DET. |
| | | M_CLASS_STAT_DET_RESULT | Specifies an Aurora Imaging Library statistics classification result buffer for object detection, allocated using MclassAllocResult() with M_STAT_DET_RESULT. |
| | | M_CLASS_STAT_SEG_CONTEXT | Specifies an Aurora Imaging Library statistics classification context for segmentation, allocated using MclassAlloc() with M_STAT_SEG. |
| | | M_CLASS_STAT_SEG_RESULT | Specifies an Aurora Imaging Library statistics classification result buffer for segmentation, allocated using MclassAllocResult() with M_STAT_SEG_RESULT. |
| | | M_CLASS_STAT_TREE_ENSEMBLE_CONTEXT | Specifies an Aurora Imaging Library statistics classification context for feature classification, allocated using MclassAlloc() with M_STAT_TREE_ENSEMBLE. |
| | | M_CLASS_STAT_TREE_ENSEMBLE_RESULT | Specifies an Aurora Imaging Library statistics classification result buffer for feature classification, allocated using MclassAllocResult() with M_STAT_TREE_ENSEMBLE_RESULT. |
| | | M_CLASS_TRAIN_ANO_CONTEXT | Specifies an Aurora Imaging Library anomaly detection training context, allocated using MclassAlloc() with M_TRAIN_ANO. |

| | | | |
|---|---|---|---|
| | | M_CLASS_TRAIN_ANO_RESULT | Specifies an Aurora Imaging Library anomaly detection training result buffer, allocated using MclassAllocResult() with M_TRAIN_ANO_RESULT. |
| | | M_CLASS_TRAIN_CNN_CONTEXT | Specifies an Aurora Imaging Library CNN training context, allocated using MclassAlloc() with M_TRAIN_CNN. |
| | | M_CLASS_TRAIN_CNN_RESULT | Specifies an Aurora Imaging Library CNN training result buffer, allocated using MclassAllocResult() with M_TRAIN_CNN_RESULT. |
| | | M_CLASS_TRAIN_DET_CONTEXT | Specifies an Aurora Imaging Library object detection training context, allocated using MclassAlloc() with M_TRAIN_DET. |
| | | M_CLASS_TRAIN_DET_RESULT | Specifies an Aurora Imaging Library object detection training result buffer, allocated using MclassAllocResult() with M_TRAIN_DET_RESULT. |
| | | M_CLASS_TRAIN_SEG_CONTEXT | Specifies an Aurora Imaging Library segmentation training context, allocated using MclassAlloc() with M_TRAIN_SEG. |
| | | M_CLASS_TRAIN_SEG_RESULT | Specifies an Aurora Imaging Library segmentation training result buffer, allocated using MclassAllocResult() with M_TRAIN_CNN_RESULT. |
| | | M_CLASS_TRAIN_TREE_ENSEMBLE_CONTEXT | Specifies an Aurora Imaging Library tree ensemble training context, allocated using MclassAlloc() with M_TRAIN_TREE_ENSEMBLE. |
| | | M_CLASS_TRAIN_TREE_ENSEMBLE_RESULT | Specifies an Aurora Imaging Library tree ensemble training result buffer, allocated using MclassAllocResult() with M_TRAIN_TREE_ENSEMBLE_RESULT. |
| | | M_CODE_CONTEXT | Specifies an Aurora Imaging Library code context allocated using McolAlloc(). |
| | | M_CODE_DETECT_RESULT | Specifies an Aurora Imaging Library code detect result buffer allocated using McodeAllocResult(). |
| | | M_CODE_GRADE_RESULT | Specifies an Aurora Imaging Library code grade result buffer allocated using McodeAllocResult(). |
| | | M_CODE_MODEL | Specifies an Aurora Imaging Library code model allocated using McodeModel() with M_ADD. |
| | | M_CODE_READ_RESULT | Specifies an Aurora Imaging Library code read result buffer allocated using McodeAllocResult(). |

| | | | |
|---|---|---|---|
| | | M_CODE_TRAIN_RESULT | Specifies an Aurora Imaging Library code train result buffer allocated using McodeAllocResult(). |
| | | M_CODE_WRITE_RESULT | Specifies an Aurora Imaging Library code write result buffer allocated using McodeAllocResult(). |
| | | M_COL_MATCH_CONTEXT | Specifies an Aurora Imaging Library color analysis context (for matching) allocated using McolAlloc() with M_COLOR_MATCHING. |
| | | M_COL_MATCH_RESULT | Specifies an Aurora Imaging Library color analysis result buffer (for matching) allocated using McolAllocResult() with M_COLOR_MATCHING_RESULT. |
| | | M_COL_RELATIVE_CALIBRATION_CONTEXT | Specifies an Aurora Imaging Library relative color calibration context allocated using McolAlloc() with M_COLOR_CALIBRATION_RELATIVE. |
| | | M_COM_CONTEXT | Specifies an Aurora Imaging Library industrial communication context allocated using McomAlloc(). |
| | | M_CONTAINER | Specifies an Aurora Imaging Library container buffer allocated using MbufAllocContainer(). |
| | | M_COUNT_LIST | Specifies an Aurora Imaging Library count list result buffer allocated using MimAllocResult() with M_COUNT_LIST. |
| | | M_DIGITIZER | Specifies an Aurora Imaging Library digitizer allocated using MdigAlloc(). |
| | | M_DLOCR_DATASET | Specifies an Aurora Imaging Library Deep Learning OCR dataset allocated using MdlocrAllocDataset(). |
| | | M_DLOCR_FINETUNE_CONTEXT | Specifies an Aurora Imaging Library Deep Learning OCR fine-tuning context allocated using MdlocrAlloc(). |
| | | M_DLOCR_FINETUNE_RESULT | Specifies an Aurora Imaging Library Deep Learning OCR fine-tuning result buffer allocated using MdlocrAllocResult(). |
| | | M_DLOCR_READ_CONTEXT | Specifies an Aurora Imaging Library Deep Learning OCR read context allocated using MdlocrAlloc(). |
| | | M_DLOCR_READ_RESULT | Specifies an Aurora Imaging Library Deep Learning OCR read result buffer allocated using MdlocrAllocResult(). |
| | | M_DMR_CONTEXT | Specifies an Aurora Imaging Library SureDotOCR context allocated using MdmrAlloc(). |

| | | | |
|---|---|---|---|
| | | M_DMR_RESULT | Specifies an Aurora Imaging Library SureDotOCR result buffer allocated using MdmrAllocResult(). |
| | | M_EDGE_CONTOUR | Specifies an Aurora Imaging Library edge contour context allocated using MedgeAlloc() with M_CONTOUR. |
| | | M_EDGE_CREST | Specifies an Aurora Imaging Library edge crest context allocated using MedgeAlloc() with M_CREST. |
| | | M_EDGE_RESULT | Specifies an Aurora Imaging Library edge result buffer allocated using MedgeAllocResult(). |
| | | M_EVENT | Specifies an Aurora Imaging Library event allocated using MthrAlloc() with M_EVENT. |
| | | M_EVENT_LIST | Specifies an Aurora Imaging Library event list result buffer allocated using MimAllocResult() with M_EVENT_LIST. |
| | | M_EXTREME_LIST | Specifies an Aurora Imaging Library extreme list result buffer allocated using MimAllocResult() with M_EXTREME_LIST. |
| | | M_FIND_ORIENTATION_LIST | Specifies an Aurora Imaging Library find orientation list result buffer allocated using MimAllocResult() with M_FIND_ORIENTATION_LIST. |
| | | M_GRAPHIC_CONTEXT | Specifies an Aurora Imaging Library 2D graphics context allocated using MgraAlloc(). |
| | | M_GRAPHIC_LIST | Specifies an Aurora Imaging Library 2D graphics list allocated using MgraAllocList(). |
| | | M_HIST_LIST | Specifies an Aurora Imaging Library histogram list result buffer allocated using MimAllocResult() with M_HIST_LIST. |
| | | M_HTTP_SERVER | Specifies an HTTP server allocated using MobjAlloc() with M_HTTP_SERVER. |
| | | M_IM_CONTEXT | Specifies an Aurora Imaging Library image processing context allocated using MimAlloc(). |
| | | M_IMAGE | Specifies an Aurora Imaging Library image buffer allocated using MbufAllocColor() with M_IMAGE. |
| | | M_KERNEL | Specifies an Aurora Imaging Library kernel buffer allocated using MbufAlloc2d() with M_KERNEL. |

| | | | |
|---|---|---|---|
| | | M_LOCATE_DIFFERENCE_RESULT | Specifies an Aurora Imaging Library locate difference result buffer allocated using MimAllocResult() with M_LOCATE_DIFFERENCE_RESULT. |
| | | M_LOCATE_PEAK_1D_RESULT | Specifies an Aurora Imaging Library locate 1D peak result buffer allocated using MimAllocResult() with M_LOCATE_PEAK_1D_RESULT. |
| | | M_LUT | Specifies an Aurora Imaging Library LUT buffer allocated using MbufAlloc2d() with M_LUT. |
| | | M_MEAS_CONTEXT | Specifies an Aurora Imaging Library measurement context allocated using MmeasAllocContext(). |
| | | M_MEAS_MARKER | Specifies an Aurora Imaging Library measurement marker allocated using MmeasAllocMarker(). |
| | | M_MEAS_RESULT | Specifies an Aurora Imaging Library measurement result buffer allocated using MmeasAllocResult(). |
| | | M_MESSAGE_MAILBOX | Specifies an Aurora Imaging Library message mailbox allocated using MobjAlloc(). |
| | | M_MET_CONTEXT | Specifies an Aurora Imaging Library metrology context allocated using MmetAlloc() with M_CONTEXT. |
| | | M_MET_DERIVED_GEOMETRY_REGION | Specifies an Aurora Imaging Library metrology derived geometry region object allocated using MmetAlloc() with M_DERIVED_GEOMETRY_REGION. |
| | | M_MET_RESULT | Specifies an Aurora Imaging Library metrology result buffer allocated using MmetAllocResult(). |
| | | M_MOD_GEOMETRIC | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_GEOMETRIC. |
| | | M_MOD_GEOMETRIC_CONTROLLED | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_GEOMETRIC_CONTROLLED. |
| | | M_MOD_RESULT | Specifies an Aurora Imaging Library Model Finder result buffer allocated using MmodAllocResult() with M_DEFAULT. |
| | | M_MOD_SHAPE_CIRCLE_CONTEXT | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_CIRCLE. |
| | | M_MOD_SHAPE_ELLIPSE_CONTEXT | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_ELLIPSE. |

| | | | |
|---|---|---|---|
| | | M_MOD_SHAPE_RECTANGLE_CONTEXT | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_RECTANGLE. |
| | | M_MOD_SHAPE_RESULT | Specifies an Aurora Imaging Library Model Finder result buffer allocated using MmodAllocResult() with M_SHAPE_.... |
| | | M_MOD_SHAPE_SEGMENT_CONTEXT | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_SEGMENT. |
| | | M_MUTEX | Specifies an Aurora Imaging Library mutex allocated using MthrAlloc() with M_MUTEX. |
| | | M_OCR_FONT | Specifies an Aurora Imaging Library character recognition font context allocated using MocrAllocFont(). |
| | | M_OCR_RESULT | Specifies an Aurora Imaging Library character recognition result buffer allocated using MocrAllocResult(). |
| | | M_PAT_CONTEXT | Specifies an Aurora Imaging Library pattern matching context allocated using MpatAlloc(). |
| | | M_PAT_RESULT | Specifies an Aurora Imaging Library pattern matching result buffer allocated using MpatAllocResult(). |
| | | M_PROJ_LIST | Specifies an Aurora Imaging Library project list result buffer allocated using MimAllocResult() with M_PROJ_LIST. |
| | | M_REG_DFF_CONTEXT | Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_DEPTH_FROM_FOCUS. |
| | | M_REG_DFF_RESULT | Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_DEPTH_FROM_FOCUS_RESULT. |
| | | M_REG_EDOF_CONTEXT | Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_EXTENDED_DEPTH_OF_FIELD. |
| | | M_REG_EDOF_RESULT | Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_EXTENDED_DEPTH_OF_FIELD_RESULT. |
| | | M_REG_HDR_CONTEXT | Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_HIGH_DYNAMIC_RANGE. |
| | | M_REG_HDR_RESULT | Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_HIGH_DYNAMIC_RANGE_RESULT. |

| | | | |
|---|---|---|---|
| | | M_REG_PHOTOMETRIC_STEREO_CONTEXT | Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_PHOTOMETRIC_STEREO. |
| | | M_REG_PHOTOMETRIC_STEREO_RESULT | Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_PHOTOMETRIC_STEREO_RESULT. |
| | | M_REG_STITCHING_CONTEXT | Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_STITCHING. |
| | | M_REG_STITCHING_RESULT | Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_STITCHING_RESULT. |
| | | M_SELECTABLE_THREAD | Specifies an Aurora Imaging Library selectable thread allocated using MthrAlloc() with M_SELECTABLE_THREAD. |
| | | M_SEQUENCE_CONTEXT | Specifies an Aurora Imaging Library sequence context allocated using MseqAlloc(). |
| | | M_STATISTICS_RESULT | Specifies an Aurora Imaging Library MimStatCalculate() result buffer allocated using MimAllocResult() with M_STATISTICS_RESULT. |
| | | M_STR_CONTEXT | Specifies an Aurora Imaging Library string context allocated using MstrAlloc(). |
| | | M_STR_RESULT | Specifies an Aurora Imaging Library string result buffer allocated using MstrAllocResult(). |
| | | M_STRUCT_ELEMENT | Specifies an Aurora Imaging Library structuring element buffer allocated using MbufAlloc2d() with M_STRUCT_ELEMENT. |
| | | M_SYS_IO_CONTEXT | Specifies an Aurora Imaging Library system I/O context allocated using MsysIoAlloc(). |
| | | M_SYSTEM | Specifies an Aurora Imaging Library system context allocated using MsysAlloc(). |
| | | M_SYSTEM_THREAD | Specifies an Aurora Imaging Library thread allocated using MthrAlloc() with M_THREAD. |
| | | M_THREAD | Specifies an Aurora Imaging Library thread context object allocated using MthrAlloc() with M_THREAD. |

| | | | |
|---|---|---|---|
| | | M_USER_OBJECT_1 + | Specifies an Aurora Imaging Library object from group one of the user-defined object types allocated using MfuncAllocId() with M_USER_OBJECT_1.<br><br>You must specify a combination value from the following table:<br><br>• To set the offset |
| | | M_USER_OBJECT_2 + | Specifies an Aurora Imaging Library object from group two of the user-defined object types allocated using MfuncAllocId() with M_USER_OBJECT_2.<br><br>You must specify a combination value from the following table:<br><br>• To set the offset |
| | | M_WAVELET_TRANSFORM_RESULT | Specifies an Aurora Imaging Library wavelet transform result buffer allocated using MimAllocResult() with M_WAVELET_TRANSFORM_RESULT. |
| | **ExtraFlag** | | Specifies the size of the array.<br><br>If the current list of published objects in the publishing application is smaller than the array size, the array will be populated with the identifiers associated with all the objects in the list, and the remaining elements will be populated with **M_NULL**.<br><br>If the current list of published objects in the publishing application is larger than the array size, the array will be populated with the identifiers associated with the first objects it receives, and the remaining identifiers on the list will be ignored. |
| | **UserVarPtr** | | Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID> ]* |
| M_DAIL_PUBLISHED_LIST_SIZE | | | Inquires the current number of objects published in a Distributed Aurora Imaging Library publishing application. |
| | **ControlFlag** | | Specifies to limit the count from all objects to all objects of the specified object type.<br><br>> Note that a display object cannot be published, and so is not a possible object type with which to limit the size of returned list. |
| | | M_DEFAULT | Specifies to count all object types in the Distributed Aurora Imaging Library publishing application. |
| | | M_3D_DISPLAY | Specifies an Aurora Imaging Library 3D display allocated using M3ddispAlloc(). |
| | | M_3D_DISPLAY_PICKING_AREA_CONTEXT | Specifies an Aurora Imaging Library picking area context allocated using M3ddispAlloc() with M_PICKING_AREA_CONTEXT. |

| | | | |
|---|---|---|---|
| | | M_3D_DISPLAY_PICKING_AREA_RESULT | Specifies an Aurora Imaging Library picking area result buffer allocated using M3ddispAllocResult() with M_PICKING_AREA_RESULT. |
| | | M_3D_DISPLAY_PICKING_CONTEXT | Specifies an Aurora Imaging Library picking context allocated using M3ddispAlloc() with M_PICKING_CONTEXT. |
| | | M_3D_DISPLAY_PICKING_RESULT | Specifies an Aurora Imaging Library picking result buffer allocated using M3ddispAllocResult() with M_PICKING_RESULT. |
| | | M_3D_GRAPHIC_LIST | Specifies an Aurora Imaging Library 3D graphics list allocated using M3dgraAlloc(). |
| | | M_3DBLOB_CALCULATE_CONTEXT | Specifies an Aurora Imaging Library 3D blob context that can be used with M3dblobCalculate(), allocated using M3dblobAlloc() with M_CALCULATE_CONTEXT. |
| | | M_3DBLOB_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library 3D blob context that can be used with M3dblobDraw3d(), allocated using M3dblobAlloc() with M_DRAW_3D_CONTEXT. |
| | | M_3DBLOB_SEGMENTATION_CONTEXT | Specifies an Aurora Imaging Library 3D blob context that can be used with M3dblobSegment(), allocated using M3dblobAlloc() with M_SEGMENTATION_CONTEXT. |
| | | M_3DBLOB_SEGMENTATION_RESULT | Specifies an Aurora Imaging Library 3D blob result buffer for segmentation results, allocated using M3dblobAllocResult() with M_SEGMENTATION_RESULT. |
| | | M_3DGEO_GEOMETRY | Specifies an Aurora Imaging Library 3D geometry object allocated using M3dgeoAlloc() with M_GEOMETRY. |
| | | M_3DGEO_TRANSFORMATION_MATRIX | Specifies an Aurora Imaging Library 3D transformation matrix object allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX. |
| | | M_3DIM_CALCULATE_MAP_SIZE_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimCalculateMapSize(), allocated using M3dimAlloc() with M_CALCULATE_MAP_SIZE_CONTEXT. |
| | | M_3DIM_FILL_GAPS_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimFillGaps(), allocated using M3dimAlloc() with M_FILL_GAPS_CONTEXT. |
| | | M_3DIM_FILTER_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimFilter(), allocated using M3dimAlloc() with M_FILTER_CONTEXT. |

| | | | |
|---|---|---|---|
| | | M_3DIM_FIND_TRANSFORMATION_RESULT | Specifies an Aurora Imaging Library 3D image processing result buffer for find transformation results, allocated using M3dimAllocResult() with M_FIND_TRANSFORMATION_RESULT. |
| | | M_3DIM_LATTICE_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimLattice(), allocated using M3dimAlloc() with M_LATTICE_CONTEXT. |
| | | M_3DIM_LATTICE_RESULT | Specifies an Aurora Imaging Library 3D image processing result buffer for lattice results, allocated using M3dimAllocResult() with M_LATTICE_RESULT. |
| | | M_3DIM_MESH_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimMesh(), allocated using M3dimAlloc() with M_MESH_CONTEXT. |
| | | M_3DIM_NORMALS_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimNormals(), allocated using M3dimAlloc() with M_NORMALS_CONTEXT. |
| | | M_3DIM_OUTLIERS_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimOutliers(), allocated using M3dimAlloc() with M_OUTLIERS_CONTEXT. |
| | | M_3DIM_PROFILE_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimProfileEx(), allocated using M3dimAlloc() with M_PROFILE_CONTEXT. |
| | | M_3DIM_PROFILE_RESULT | Specifies an Aurora Imaging Library 3D image processing result buffer for profile results, allocated using M3dimAllocResult() with M_PROFILE_RESULT. |
| | | M_3DIM_PROJECT_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimProjectEx(), allocated using M3dimAlloc() with M_PROJECT_CONTEXT. |
| | | M_3DIM_REMAP_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimRemapDepthMap(), allocated using M3dimAlloc() with M_REMAP_CONTEXT. |
| | | M_3DIM_STATISTICS_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimStat(), allocated using M3dimAlloc() with M_STATISTICS_CONTEXT. |
| | | M_3DIM_STATISTICS_RESULT | Specifies an Aurora Imaging Library 3D image processing result buffer used to store M3dimStat() results, and is allocated using M3dimAllocResult() with M_STATISTICS_RESULT. |

| | | | |
|---|---|---|---|
| | | M_3DIM_STITCH_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimMerge(), allocated using M3dimAlloc() with M_STITCH_CONTEXT. |
| | | M_3DIM_SUBSAMPLE_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimSample(), allocated using M3dimAlloc() with M_SUBSAMPLE_CONTEXT. |
| | | M_3DIM_SURFACE_SAMPLE_CONTEXT | Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimSample(), allocated using M3dimAlloc() with M_SURFACE_SAMPLE_CONTEXT. |
| | | M_3DMAP_ALIGN_CONTEXT | Specifies an Aurora Imaging Library 3D align context allocated using M3dmapAlloc() with M_ALIGN_CONTEXT. |
| | | M_3DMAP_ALIGN_RESULT | Specifies an Aurora Imaging Library 3D align result buffer allocated using M3dmapAllocResult() with M_ALIGN_RESULT. |
| | | M_3DMAP_DEPTH_CORRECTED_DATA | Specifies an Aurora Imaging Library 3D reconstruction result buffer allocated using M3dmapAllocResult() with M_DEPTH_CORRECTED_DATA. |
| | | M_3DMAP_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library 3D draw context allocated using M3dmapAlloc() with M_DRAW_3D_CONTEXT. |
| | | M_3DMAP_LASER_CALIBRATION_DATA | Specifies an Aurora Imaging Library 3D reconstruction result buffer allocated using M3dmapAllocResult() with M_LASER_CALIBRATION_DATA. |
| | | M_3DMAP_LASER_CONTEXT | Specifies an Aurora Imaging Library 3D reconstruction context allocated using M3dmapAlloc() with M_LASER. |
| | | M_3DMAP_POINT_CLOUD_RESULT | Specifies an Aurora Imaging Library 3D reconstruction result buffer allocated using M3dmapAllocResult() with M_POINT_CLOUD_RESULT. |
| | | M_3DMEAS_DRAW_3D_PATH_CONTEXT | Specifies an Aurora Imaging Library draw path 3D measurement context allocated using M3dmeasAlloc() with M_DRAW_3D_PATH_CONTEXT. |
| | | M_3DMEAS_DRAW_3D_PROFILE_CONTEXT | Specifies an Aurora Imaging Library draw profile 3D measurement context allocated using M3dmeasAlloc() with M_DRAW_3D_PROFILE_CONTEXT. |
| | | M_3DMEAS_DRAW_3D_TEMPLATE_CONTEXT | Specifies an Aurora Imaging Library draw template 3D measurement context allocated using M3dmeasAlloc() with M_DRAW_3D_TEMPLATE_CONTEXT. |

| | | | |
|---|---|---|---|
| | | M_3DMEAS_FIND_MARKER_PATH_CONTEXT | Specifies an Aurora Imaging Library path 3D measurement context allocated using M3dmeasAlloc() with M_FIND_MARKER_PATH_CONTEXT. |
| | | M_3DMEAS_FIND_MARKER_PATH_RESULT | Specifies an Aurora Imaging Library path 3D measurement result allocated using M3dmeasAllocResult() with M_FIND_MARKER_PATH_RESULT. |
| | | M_3DMEAS_FIND_MARKER_PROFILE_CONTEXT | Specifies an Aurora Imaging Library profile 3D measurement context allocated using M3dmeasAlloc() with M_FIND_MARKER_PROFILE_CONTEXT. |
| | | M_3DMEAS_FIND_MARKER_PROFILE_RESULT | Specifies an Aurora Imaging Library profile 3D measurement result allocated using M3dmeasAllocResult() with M_FIND_MARKER_PROFILE_RESULT. |
| | | M_3DMEAS_FIND_MARKER_TEMPLATE_CONTEXT | Specifies an Aurora Imaging Library template 3D measurement context allocated using M3dmeasAlloc() with M_FIND_MARKER_TEMPLATE_CONTEXT. |
| | | M_3DMEAS_FIND_MARKER_TEMPLATE_RESULT | Specifies an Aurora Imaging Library template 3D measurement result allocated using M3dmeasAllocResult() with M_FIND_MARKER_TEMPLATE_RESULT. |
| | | M_3DMEAS_FIT_CONTEXT | Specifies an Aurora Imaging Library fit 3D measurement context allocated using M3dmeasAlloc() with M_FIT_CONTEXT. |
| | | M_3DMET_CALCULATE_RESULT | Specifies an Aurora Imaging Library calculate 3D metrology result buffer allocated using M3dmetAllocResult() with M_CALCULATE_RESULT. |
| | | M_3DMET_DISTANCE_CONTEXT | Specifies an Aurora Imaging Library distance 3D metrology context allocated using M3dmetAlloc() with M_DISTANCE_CONTEXT. |
| | | M_3DMET_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library draw 3D metrology context allocated using M3dmetAlloc() with M_DRAW_3D_CONTEXT. |
| | | M_3DMET_FIT_CONTEXT | Specifies an Aurora Imaging Library fit 3D metrology context allocated using M3dmetAlloc() with M_FIT_CONTEXT. |
| | | M_3DMET_FIT_RESULT | Specifies an Aurora Imaging Library fit 3D metrology result buffer allocated using M3dmetAllocResult() with M_FIT_RESULT. |
| | | M_3DMET_STATISTICS_CONTEXT | Specifies an Aurora Imaging Library statistics 3D metrology context allocated using M3dmetAlloc() with M_STATISTICS_CONTEXT. |

| | | | |
|---|---|---|---|
| | | M_3DMET_STATISTICS_RESULT | Specifies an Aurora Imaging Library statistics 3D metrology result buffer allocated using M3dmetAllocResult() with M_STATISTICS_RESULT. |
| | | M_3DMET_VOLUME_CONTEXT | Specifies an Aurora Imaging Library calculate volume 3D metrology context allocated using M3dmetAlloc() with M_VOLUME_CONTEXT. |
| | | M_3DMOD_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library draw 3D model finder context that can be allocated using M3dmodAlloc() with M_DRAW_3D_GEOMETRIC_CONTEXT (to draw occurrences of geometric models). |
| | | M_3DMOD_DRAW_3D_SURFACE_CONTEXT | Specifies an Aurora Imaging Library draw 3D model finder context that can be allocated using M3dmodAlloc() with M_DRAW_3D_SURFACE_CONTEXT (to draw occurrences of surface models). |
| | | M_3DMOD_FIND_BOX_CONTEXT | Specifies an Aurora Imaging Library box 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_BOX_CONTEXT. |
| | | M_3DMOD_FIND_BOX_RESULT | Specifies an Aurora Imaging Library box 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_BOX_RESULT. |
| | | M_3DMOD_FIND_CYLINDER_CONTEXT | Specifies an Aurora Imaging Library cylinder 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_CYLINDER_CONTEXT. |
| | | M_3DMOD_FIND_CYLINDER_RESULT | Specifies an Aurora Imaging Library cylinder 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_CYLINDER_RESULT. |
| | | M_3DMOD_FIND_PLANAR_SURFACE_CONTEXT | Specifies an Aurora Imaging Library planar surface 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_PLANAR_SURFACE_CONTEXT. |
| | | M_3DMOD_FIND_PLANAR_SURFACE_RESULT | Specifies an Aurora Imaging Library planar surface 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_PLANAR_SURFACE_RESULT. |
| | | M_3DMOD_FIND_RECTANGULAR_PLANE_CONTEXT | Specifies an Aurora Imaging Library rectangular plane 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_RECTANGULAR_PLANE_CONTEXT. |
| | | M_3DMOD_FIND_RECTANGULAR_PLANE_RESULT | Specifies an Aurora Imaging Library rectangular plane 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_RECTANGULAR_PLANE_RESULT. |

| | | | |
|---|---|---|---|
| | | M_3DMOD_FIND_SPHERE_CONTEXT | Specifies an Aurora Imaging Library sphere 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_SPHERE_CONTEXT. |
| | | M_3DMOD_FIND_SPHERE_RESULT | Specifies an Aurora Imaging Library sphere 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_SPHERE_RESULT. |
| | | M_3DMOD_FIND_SURFACE_CONTEXT | Specifies an Aurora Imaging Library surface 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_SURFACE_CONTEXT. |
| | | M_3DMOD_FIND_SURFACE_RESULT | Specifies an Aurora Imaging Library surface 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_SURFACE_RESULT. |
| | | M_3DREG_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library draw 3D registration context that can be used with M3dregDraw3d(), allocated using M3dregAlloc() with M_DRAW_3D_CONTEXT. |
| | | M_3DREG_PAIRWISE_REGISTRATION_CONTEXT | Specifies an Aurora Imaging Library pairwise 3D registration context allocated using M3dregAlloc() with M_PAIRWISE_REGISTRATION_CONTEXT. |
| | | M_3DREG_PAIRWISE_REGISTRATION_RESULT | Specifies an Aurora Imaging Library pairwise 3D registration result buffer allocated using M3dregAllocResult() with M_PAIRWISE_REGISTRATION_RESULT. |
| | | M_AGM_EDGE_BASED_FIND_CONTEXT | Specifies an Aurora Imaging Library Advanced Geometric Matching context allocated using MagmAlloc() with M_GLOBAL_EDGE_BASED_FIND. |
| | | M_AGM_EDGE_BASED_FIND_RESULT | Specifies an Aurora Imaging Library Advanced Geometric Matching result buffer allocated using MagmAllocResult() with M_GLOBAL_EDGE_BASED_FIND_RESULT. |
| | | M_AGM_EDGE_BASED_TRAIN_CONTEXT | Specifies an Aurora Imaging Library Advanced Geometric Matching context allocated using MagmAlloc() with M_GLOBAL_EDGE_BASED_TRAIN. |
| | | M_AGM_EDGE_BASED_TRAIN_RESULT | Specifies an Aurora Imaging Library Advanced Geometric Matching result buffer allocated using MagmAllocResult() with M_GLOBAL_EDGE_BASED_TRAIN_RESULT. |
| | | M_APPLICATION | Specifies an Aurora Imaging Library application context allocated using MappAlloc(). |

| | | | |
|---|---|---|---|
| | | M_ARRAY | Specifies an Aurora Imaging Library array buffer allocated using MbufAlloc2d() with M_ARRAY. |
| | | M_AUGMENTATION_RESULT | Specifies an Aurora Imaging Library augmentation result buffer allocated using MimAllocResult() with M_AUGMENTATION_RESULT. |
| | | M_BEAD_CONTEXT | Specifies an Aurora Imaging Library bead context allocated using MbeadAlloc(). |
| | | M_BEAD_RESULT | Specifies an Aurora Imaging Library bead result buffer allocated using MbeadAllocResult(). |
| | | M_BLOB_CONTEXT | Specifies an Aurora Imaging Library blob analysis context allocated using MblobAlloc(). |
| | | M_BLOB_RESULT | Specifies an Aurora Imaging Library blob analysis result buffer allocated using MblobAllocResult(). |
| | | M_CAL_CALCULATE_HAND_EYE_CONTEXT | Specifies an Aurora Imaging Library calculate hand-eye context (hand-eye calibration) allocated using McalAlloc() with M_CALCULATE_HAND_EYE_CONTEXT. |
| | | M_CAL_CALCULATE_HAND_EYE_RESULT | Specifies an Aurora Imaging Library calculate hand-eye result buffer allocated using McalAllocResult() with M_CALCULATE_HAND_EYE_RESULT. |
| | | M_CAL_CONTEXT | Specifies an Aurora Imaging Library camera calibration context allocated using McalAlloc(). |
| | | M_CAL_DRAW_3D_CONTEXT | Specifies an Aurora Imaging Library 3D draw calibration context allocated using McalAlloc() with M_DRAW_3D_CONTEXT. |
| | | M_CAL_FIXTURING_OFFSET | Specifies an Aurora Imaging Library fixturing offset object allocated using McalAlloc() with M_FIXTURING_OFFSET. |
| | | M_CLASS_CLASSIFIER_ANO_PREDEFINED_CONTEXT | Specifies an Aurora Imaging Library predefined anomaly detection classifier context, allocated using MclassAlloc() with M_CLASSIFIER_ANO_PREDEFINED. |
| | | M_CLASS_CLASSIFIER_CNN_PREDEFINED_CONTEXT | Specifies an Aurora Imaging Library predefined CNN classifier context, allocated using MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED. |
| | | M_CLASS_CLASSIFIER_DET_PREDEFINED_CONTEXT | Specifies an Aurora Imaging Library predefined object detection classifier context, allocated using MclassAlloc() with M_CLASSIFIER_DET_PREDEFINED. |

| | | | |
|---|---|---|---|
| | | M_CLASS_CLASSIFIER_ONNX_CONTEXT | Specifies an Aurora Imaging Library ONNX classifier context, allocated using MclassAlloc() with M_CLASSIFIER_ONNX. |
| | | M_CLASS_CLASSIFIER_SEG_PREDEFINED_CONTEXT | Specifies an Aurora Imaging Library predefined segmentation classifier context, allocated using MclassAlloc() with M_CLASSIFIER_SEG_PREDEFINED. |
| | | M_CLASS_CLASSIFIER_TREE_ENSEMBLE_CONTEXT | Specifies an Aurora Imaging Library tree ensemble classifier context, allocated using MclassAlloc() with M_CLASSIFIER_TREE_ENSEMBLE. |
| | | M_CLASS_DATASET_FEATURES | Specifies an Aurora Imaging Library features dataset context, allocated using MclassAlloc() with M_DATASET_FEATURES. |
| | | M_CLASS_DATASET_IMAGES | Specifies an Aurora Imaging Library images dataset context, allocated using MclassAlloc() with M_DATASET_IMAGES. |
| | | M_CLASS_PREDICT_ANO_RESULT | Specifies an Aurora Imaging Library anomaly detection prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_ANO_RESULT. |
| | | M_CLASS_PREDICT_CNN_RESULT | Specifies an Aurora Imaging Library CNN prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_CNN_RESULT. |
| | | M_CLASS_PREDICT_DET_RESULT | Specifies an Aurora Imaging Library object detection prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_DET_RESULT. |
| | | M_CLASS_PREDICT_ONNX_RESULT | Specifies an ONNX prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_ONNX_RESULT. |
| | | M_CLASS_PREDICT_SEG_RESULT | Specifies an Aurora Imaging Library segmentation prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_SEG_RESULT. |
| | | M_CLASS_PREDICT_TREE_ENSEMBLE_RESULT | Specifies an Aurora Imaging Library tree ensemble prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_TREE_ENSEMBLE_RESULT. |
| | | M_CLASS_PREPARE_IMAGES_CNN | Specifies an Aurora Imaging Library CNN data preparation context, allocated using MclassAlloc() with M_PREPARE_IMAGES_CNN. |
| | | M_CLASS_PREPARE_IMAGES_DET | Specifies an Aurora Imaging Library object detection data preparation context, allocated using MclassAlloc() with M_PREPARE_IMAGES_DET. |
| | | M_CLASS_PREPARE_IMAGES_SEG | Specifies an Aurora Imaging Library segmentation data preparation context, allocated using MclassAlloc() with M_PREPARE_IMAGES_SEG. |

| | | | |
|---|---|---|---|
| | | M_CLASS_STAT_ANO_CONTEXT | Specifies an Aurora Imaging Library statistics classification context for anomaly detection, allocated using MclassAlloc() with M_STAT_ANO. |
| | | M_CLASS_STAT_ANO_RESULT | Specifies an Aurora Imaging Library statistics classification result buffer for anomaly detection, allocated using MclassAllocResult() with M_STAT_ANO_RESULT. |
| | | M_CLASS_STAT_CNN_CONTEXT | Specifies an Aurora Imaging Library statistics classification context for image classification, allocated using MclassAlloc() with M_STAT_CNN. |
| | | M_CLASS_STAT_CNN_RESULT | Specifies an Aurora Imaging Library statistics classification result buffer for image classification, allocated using MclassAllocResult() with M_STAT_CNN_RESULT. |
| | | M_CLASS_STAT_DET_CONTEXT | Specifies an Aurora Imaging Library statistics classification context for object detection, allocated using MclassAlloc() with M_STAT_DET. |
| | | M_CLASS_STAT_DET_RESULT | Specifies an Aurora Imaging Library statistics classification result buffer for object detection, allocated using MclassAllocResult() with M_STAT_DET_RESULT. |
| | | M_CLASS_STAT_SEG_CONTEXT | Specifies an Aurora Imaging Library statistics classification context for segmentation, allocated using MclassAlloc() with M_STAT_SEG. |
| | | M_CLASS_STAT_SEG_RESULT | Specifies an Aurora Imaging Library statistics classification result buffer for segmentation, allocated using MclassAllocResult() with M_STAT_SEG_RESULT. |
| | | M_CLASS_STAT_TREE_ENSEMBLE_CONTEXT | Specifies an Aurora Imaging Library statistics classification context for feature classification, allocated using MclassAlloc() with M_STAT_TREE_ENSEMBLE. |
| | | M_CLASS_STAT_TREE_ENSEMBLE_RESULT | Specifies an Aurora Imaging Library statistics classification result buffer for feature classification, allocated using MclassAllocResult() with M_STAT_TREE_ENSEMBLE_RESULT. |
| | | M_CLASS_TRAIN_ANO_CONTEXT | Specifies an Aurora Imaging Library anomaly detection training context, allocated using MclassAlloc() with M_TRAIN_ANO. |
| | | M_CLASS_TRAIN_ANO_RESULT | Specifies an Aurora Imaging Library anomaly detection training result buffer, allocated using MclassAllocResult() with M_TRAIN_ANO_RESULT. |
| | | M_CLASS_TRAIN_CNN_CONTEXT | Specifies an Aurora Imaging Library CNN training context, allocated using MclassAlloc() with M_TRAIN_CNN. |

| | | | |
|---|---|---|---|
| | | M_CLASS_TRAIN_CNN_RESULT | Specifies an Aurora Imaging Library CNN training result buffer, allocated using MclassAllocResult() with M_TRAIN_CNN_RESULT. |
| | | M_CLASS_TRAIN_DET_CONTEXT | Specifies an Aurora Imaging Library object detection training context, allocated using MclassAlloc() with M_TRAIN_DET. |
| | | M_CLASS_TRAIN_DET_RESULT | Specifies an Aurora Imaging Library object detection training result buffer, allocated using MclassAllocResult() with M_TRAIN_DET_RESULT. |
| | | M_CLASS_TRAIN_SEG_CONTEXT | Specifies an Aurora Imaging Library segmentation training context, allocated using MclassAlloc() with M_TRAIN_SEG. |
| | | M_CLASS_TRAIN_SEG_RESULT | Specifies an Aurora Imaging Library segmentation training result buffer, allocated using MclassAllocResult() with M_TRAIN_CNN_RESULT. |
| | | M_CLASS_TRAIN_TREE_ENSEMBLE_CONTEXT | Specifies an Aurora Imaging Library tree ensemble training context, allocated using MclassAlloc() with M_TRAIN_TREE_ENSEMBLE. |
| | | M_CLASS_TRAIN_TREE_ENSEMBLE_RESULT | Specifies an Aurora Imaging Library tree ensemble training result buffer, allocated using MclassAllocResult() with M_TRAIN_TREE_ENSEMBLE_RESULT. |
| | | M_CODE_CONTEXT | Specifies an Aurora Imaging Library code context allocated using McolAlloc(). |
| | | M_CODE_DETECT_RESULT | Specifies an Aurora Imaging Library code detect result buffer allocated using McodeAllocResult(). |
| | | M_CODE_GRADE_RESULT | Specifies an Aurora Imaging Library code grade result buffer allocated using McodeAllocResult(). |
| | | M_CODE_MODEL | Specifies an Aurora Imaging Library code model allocated using McodeModel() with M_ADD. |
| | | M_CODE_READ_RESULT | Specifies an Aurora Imaging Library code read result buffer allocated using McodeAllocResult(). |
| | | M_CODE_TRAIN_RESULT | Specifies an Aurora Imaging Library code train result buffer allocated using McodeAllocResult(). |
| | | M_CODE_WRITE_RESULT | Specifies an Aurora Imaging Library code write result buffer allocated using McodeAllocResult(). |
| | | M_COL_MATCH_CONTEXT | Specifies an Aurora Imaging Library color analysis context (for matching) allocated using McolAlloc() with M_COLOR_MATCHING. |

| | | | |
|---|---|---|---|
| | | M_COL_MATCH_RESULT | Specifies an Aurora Imaging Library color analysis result buffer (for matching) allocated using McolAllocResult() with M_COLOR_MATCHING_RESULT. |
| | | M_COL_RELATIVE_CALIBRATION_CONTEXT | Specifies an Aurora Imaging Library relative color calibration context allocated using McolAlloc() with M_COLOR_CALIBRATION_RELATIVE. |
| | | M_COM_CONTEXT | Specifies an Aurora Imaging Library industrial communication context allocated using McomAlloc(). |
| | | M_CONTAINER | Specifies an Aurora Imaging Library container buffer allocated using MbufAllocContainer(). |
| | | M_COUNT_LIST | Specifies an Aurora Imaging Library count list result buffer allocated using MimAllocResult() with M_COUNT_LIST. |
| | | M_DIGITIZER | Specifies an Aurora Imaging Library digitizer allocated using MdigAlloc(). |
| | | M_DLOCR_DATASET | Specifies an Aurora Imaging Library Deep Learning OCR dataset allocated using MdlocrAllocDataset(). |
| | | M_DLOCR_FINETUNE_CONTEXT | Specifies an Aurora Imaging Library Deep Learning OCR fine-tuning context allocated using MdlocrAlloc(). |
| | | M_DLOCR_FINETUNE_RESULT | Specifies an Aurora Imaging Library Deep Learning OCR fine-tuning result buffer allocated using MdlocrAllocResult(). |
| | | M_DLOCR_READ_CONTEXT | Specifies an Aurora Imaging Library Deep Learning OCR read context allocated using MdlocrAlloc(). |
| | | M_DLOCR_READ_RESULT | Specifies an Aurora Imaging Library Deep Learning OCR read result buffer allocated using MdlocrAllocResult(). |
| | | M_DMR_CONTEXT | Specifies an Aurora Imaging Library SureDotOCR context allocated using MdmrAlloc(). |
| | | M_DMR_RESULT | Specifies an Aurora Imaging Library SureDotOCR result buffer allocated using MdmrAllocResult(). |
| | | M_EDGE_CONTOUR | Specifies an Aurora Imaging Library edge contour context allocated using MedgeAlloc() with M_CONTOUR. |
| | | M_EDGE_CREST | Specifies an Aurora Imaging Library edge crest context allocated using MedgeAlloc() with M_CREST. |

| | | | |
|---|---|---|---|
| | | M_EDGE_RESULT | Specifies an Aurora Imaging Library edge result buffer allocated using MedgeAllocResult(). |
| | | M_EVENT | Specifies an Aurora Imaging Library event allocated using MthrAlloc() with M_EVENT. |
| | | M_EVENT_LIST | Specifies an Aurora Imaging Library event list result buffer allocated using MimAllocResult() with M_EVENT_LIST. |
| | | M_EXTREME_LIST | Specifies an Aurora Imaging Library extreme list result buffer allocated using MimAllocResult() with M_EXTREME_LIST. |
| | | M_FIND_ORIENTATION_LIST | Specifies an Aurora Imaging Library find orientation list result buffer allocated using MimAllocResult() with M_FIND_ORIENTATION_LIST. |
| | | M_GRAPHIC_CONTEXT | Specifies an Aurora Imaging Library 2D graphics context allocated using MgraAlloc(). |
| | | M_GRAPHIC_LIST | Specifies an Aurora Imaging Library 2D graphics list allocated using MgraAllocList(). |
| | | M_HIST_LIST | Specifies an Aurora Imaging Library histogram list result buffer allocated using MimAllocResult() with M_HIST_LIST. |
| | | M_HTTP_SERVER | Specifies an HTTP server allocated using MobjAlloc() with M_HTTP_SERVER. |
| | | M_IM_CONTEXT | Specifies an Aurora Imaging Library image processing context allocated using MimAlloc(). |
| | | M_IMAGE | Specifies an Aurora Imaging Library image buffer allocated using MbufAllocColor() with M_IMAGE. |
| | | M_KERNEL | Specifies an Aurora Imaging Library kernel buffer allocated using MbufAlloc2d() with M_KERNEL. |
| | | M_LOCATE_DIFFERENCE_RESULT | Specifies an Aurora Imaging Library locate difference result buffer allocated using MimAllocResult() with M_LOCATE_DIFFERENCE_RESULT. |
| | | M_LOCATE_PEAK_1D_RESULT | Specifies an Aurora Imaging Library locate 1D peak result buffer allocated using MimAllocResult() with M_LOCATE_PEAK_1D_RESULT. |
| | | M_LUT | Specifies an Aurora Imaging Library LUT buffer allocated using MbufAlloc2d() with M_LUT. |

| | | | |
|---|---|---|---|
| | | M_MEAS_CONTEXT | Specifies an Aurora Imaging Library measurement context allocated using MmeasAllocContext(). |
| | | M_MEAS_MARKER | Specifies an Aurora Imaging Library measurement marker allocated using MmeasAllocMarker(). |
| | | M_MEAS_RESULT | Specifies an Aurora Imaging Library measurement result buffer allocated using MmeasAllocResult(). |
| | | M_MESSAGE_MAILBOX | Specifies an Aurora Imaging Library message mailbox allocated using MobjAlloc(). |
| | | M_MET_CONTEXT | Specifies an Aurora Imaging Library metrology context allocated using MmetAlloc() with M_CONTEXT. |
| | | M_MET_DERIVED_GEOMETRY_REGION | Specifies an Aurora Imaging Library metrology derived geometry region object allocated using MmetAlloc() with M_DERIVED_GEOMETRY_REGION. |
| | | M_MET_RESULT | Specifies an Aurora Imaging Library metrology result buffer allocated using MmetAllocResult(). |
| | | M_MOD_GEOMETRIC | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_GEOMETRIC. |
| | | M_MOD_GEOMETRIC_CONTROLLED | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_GEOMETRIC_CONTROLLED. |
| | | M_MOD_RESULT | Specifies an Aurora Imaging Library Model Finder result buffer allocated using MmodAllocResult() with M_DEFAULT. |
| | | M_MOD_SHAPE_CIRCLE_CONTEXT | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_CIRCLE. |
| | | M_MOD_SHAPE_ELLIPSE_CONTEXT | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_ELLIPSE. |
| | | M_MOD_SHAPE_RECTANGLE_CONTEXT | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_RECTANGLE. |
| | | M_MOD_SHAPE_RESULT | Specifies an Aurora Imaging Library Model Finder result buffer allocated using MmodAllocResult() with M_SHAPE_.... |
| | | M_MOD_SHAPE_SEGMENT_CONTEXT | Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_SEGMENT. |

| | | | |
|---|---|---|---|
| | | M_MUTEX | Specifies an Aurora Imaging Library mutex allocated using MthrAlloc() with M_MUTEX. |
| | | M_OCR_FONT | Specifies an Aurora Imaging Library character recognition font context allocated using MocrAllocFont(). |
| | | M_OCR_RESULT | Specifies an Aurora Imaging Library character recognition result buffer allocated using MocrAllocResult(). |
| | | M_PAT_CONTEXT | Specifies an Aurora Imaging Library pattern matching context allocated using MpatAlloc(). |
| | | M_PAT_RESULT | Specifies an Aurora Imaging Library pattern matching result buffer allocated using MpatAllocResult(). |
| | | M_PROJ_LIST | Specifies an Aurora Imaging Library project list result buffer allocated using MimAllocResult() with M_PROJ_LIST. |
| | | M_REG_DFF_CONTEXT | Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_DEPTH_FROM_FOCUS. |
| | | M_REG_DFF_RESULT | Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_DEPTH_FROM_FOCUS_RESULT. |
| | | M_REG_EDOF_CONTEXT | Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_EXTENDED_DEPTH_OF_FIELD. |
| | | M_REG_EDOF_RESULT | Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_EXTENDED_DEPTH_OF_FIELD_RESULT. |
| | | M_REG_HDR_CONTEXT | Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_HIGH_DYNAMIC_RANGE. |
| | | M_REG_HDR_RESULT | Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_HIGH_DYNAMIC_RANGE_RESULT. |
| | | M_REG_PHOTOMETRIC_STEREO_CONTEXT | Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_PHOTOMETRIC_STEREO. |
| | | M_REG_PHOTOMETRIC_STEREO_RESULT | Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_PHOTOMETRIC_STEREO_RESULT. |

| | | | |
|---|---|---|---|
| | | M_REG_STITCHING_CONTEXT | Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_STITCHING. |
| | | M_REG_STITCHING_RESULT | Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_STITCHING_RESULT. |
| | | M_SELECTABLE_THREAD | Specifies an Aurora Imaging Library selectable thread allocated using MthrAlloc() with M_SELECTABLE_THREAD. |
| | | M_SEQUENCE_CONTEXT | Specifies an Aurora Imaging Library sequence context allocated using MseqAlloc(). |
| | | M_STATISTICS_RESULT | Specifies an Aurora Imaging Library MimStatCalculate() result buffer allocated using MimAllocResult() with M_STATISTICS_RESULT. |
| | | M_STR_CONTEXT | Specifies an Aurora Imaging Library string context allocated using MstrAlloc(). |
| | | M_STR_RESULT | Specifies an Aurora Imaging Library string result buffer allocated using MstrAllocResult(). |
| | | M_STRUCT_ELEMENT | Specifies an Aurora Imaging Library structuring element buffer allocated using MbufAlloc2d() with M_STRUCT_ELEMENT. |
| | | M_SYS_IO_CONTEXT | Specifies an Aurora Imaging Library system I/O context allocated using MsysIoAlloc(). |
| | | M_SYSTEM | Specifies an Aurora Imaging Library system context allocated using MsysAlloc(). |
| | | M_SYSTEM_THREAD | Specifies an Aurora Imaging Library thread allocated using MthrAlloc() with M_THREAD. |
| | | M_THREAD | Specifies an Aurora Imaging Library thread context object allocated using MthrAlloc() with M_THREAD. |
| | | M_USER_OBJECT_1 + | Specifies an Aurora Imaging Library object from group one of the user-defined object types allocated using MfuncAllocId() with M_USER_OBJECT_1.<br><br>You must specify a combination value from the following table:<br><br>• To set the offset |

| | | |
|---|---|---|
| | M_USER_OBJECT_2 + | Specifies an Aurora Imaging Library object from group two of the user-defined object types allocated using MfuncAllocId() with M_USER_OBJECT_2.<br><br>You must specify a combination value from the following table:<br>• To set the offset |
| | M_WAVELET_TRANSFORM_RESULT | Specifies an Aurora Imaging Library wavelet transform result buffer allocated using MimAllocResult() with M_WAVELET_TRANSFORM_RESULT. |
| **UserVarPtr** | | Data type info: Data type: *address of an AIL_INT64* |
| M_DAIL_PUBLISHED_NAME | | Inquires the object's identifier from the object's name. |
| **ControlFlag** | | Specifies the following. |
| | AIL_TEXT("*ObjectName*") 1 | Specifies the name of the object.<br><br>Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*ObjectName*")). |
| **UserVarPtr** | | Data type info: Data type: *address of an AIL_ID* |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Combination value for [when InquireType=M_DAIL_PUBLISHED_LIST: M_USER_OBJECT_1; M_USER_OBJECT_2;], [when InquireType=M_DAIL_PUBLISHED_LIST_SIZE: M_USER_OBJECT_1; M_USER_OBJECT_2].

You must add the following value to the above-mentioned values to set the offset.

The offset allows you to distinguish between the different object types of the same group (for example, M_USER_OBJECT_1 + 0x0001).

| For distinguishing between the different object types | |
|---|---|
| Combination value | Description |
| *Value* | Specifies the offset within the selected object type group. The value must have only one of its 16 least significant bits set. |

## Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |

| | |
|---|---|
| **DLL** | Requires ail11.dll. |

# MappInquireMp

## Synopsis

Inquire about a multi-core processing application environment setting.

## Syntax

```
AIL_INT MappInquireMp(
    AIL_ID ContextAppId,   //in
    AIL_INT64 InquireType, //in
    AIL_INT64 TypeFlag,    //in
    AIL_INT64 TypeValue,   //in
    void *UserVarPtr       //out
)
```

## Description

This function inquires about a setting of an Aurora Imaging Library multi-core processing application environment. It can return whether multi-core processing can be used to execute certain parts of functions. It can also return how multi-core processing is performed.

In multi-thread environments, an **MappInquireMp**() call returns information that applies to all application threads running Aurora Imaging Library, unless the specific setting was overridden for a specific thread using MthrControlMp(); in which case, you should use MthrInquireMp() to obtain information about that thread.

## Parameters

ContextAppId

Specifies the identifier of the application context to use.

| For specifying the application context | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application context identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

InquireType

Specifies the type of multi-core processing application environment setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

TypeFlag

Specifies additional information regarding the inquired setting. This parameter must be set to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

TypeValue

Specifies additional information regarding the inquired setting. This parameter must be set to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_UINT64 [optionally, in C++: a reference to a std::vector<AIL_UINT64> ]
- AIL_INT

Specifies the address in which to write the requested information. Since the **MappInquireMp**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

## Parameter associations

The table below lists possible values for the **InquireType**, **TypeFlag**, and **TypeValue** parameters and possible values returned to the **UserVarPtr** parameter.

- For inquiring about multi-core processing application environment setting

Note that TypeValue and TypeFlag should be set to **M_DEFAULT** if they are not used.

| For inquiring about multi-core processing application environment setting | | | |
|---|---|---|---|
| InquireType | | | Description |
| | TypeFlag | | |
| | TypeValue | | |
| | UserVarPtr<br>- Possible values returned | | |
| M_CORE_AFFINITY_MASK | | | Inquires about the core affinity bit-mask, which indicates the CPU cores that can be used to run the multi-core processing part of functions.<br><br>The first element of the array represents the first 64 CPU cores. The least-significant bit of the first element represents CPU core 0. The most-significant bit of the first element represents CPU core 63. The least-significant bit of the second element represents CPU core 64 and so on. A CPU core is available if its corresponding bit is enabled (1). If its corresponding bit is disabled (0), the CPU core is not available. CPU cores always have the same indices, as long as the hardware in your computer and the operating system does not change. SET |
| | **UserVarPtr** | | Data type info: Data type: *array of type AIL_UINT64 [optionally, in C++: a reference to a std::vector<AIL_UINT64> ]*<br><br>Returns one of the following. |
| | | *Non-zero initialized bit-mask array* | Specifies a non-zero initialized bit-mask array. INFO |

| | | |
|---|---|---|
| | *Zero-initialized bit-mask* | Specifies that all CPU cores that are available to the process running your application can be used (M_CORE_AFFINITY_MASK is set to M_ALL). |
| M_CORE_AFFINITY_MASK_ARRAY_SIZE | | Inquires the size of the array required to store the core affinity bit-mask. See M_CORE_AFFINITY_MASK in MappControlMp() for more details.<br><br>The size of the array might represent a larger number of CPU cores than there are available in your computer. This is because the CPU cores might not be indexed by the operating system successively and/or are less than a multiple of 64. |
| | **UserVarPtr** | Data type info: Data type: *address of an AIL_INT*<br><br>Returns the following. |
| | *Value* >= 1 | Specifies the size of the core affinity mask array. |
| M_CORE_AFFINITY_MASK_PROCESS | | Inquires about the core affinity bit-mask that the operating system assigned to the process running your application. This mask indicates all CPU cores available to the process at the time when the application was allocated using MappAlloc().<br><br>The first element of the array represents the first 64 CPU cores. The least-significant bit of the first element represents CPU core 0. The most-significant bit of the first element represents CPU core 63. The least-significant bit of the second element represents CPU core 64 and so on. A CPU core is available if its corresponding bit is enabled (1). If its corresponding bit is disabled (0), the CPU core is not available. CPU cores always have the same indices, as long as the hardware in your computer and the operating system does not change. |
| | **UserVarPtr** | Data type info: Data type: *array of type AIL_UINT64 [optionally, in C++: a reference to a std::vector<AIL_UINT64> ]*<br><br>Returns the core affinity bit-mask. |
| M_CORE_MAX | | Inquires the maximum number of CPU cores to use to process the multi-core processing part of each function. SET |
| | **UserVarPtr** | Data type info: Data type: *address of an AIL_INT*<br><br>Returns the following. |
| | 1 <= *Value* <= 65535 | Specifies the maximum number of CPU cores to use. INFO |
| M_CORE_MAX_FOR_COPY | | Inquires the maximum number of CPU cores to use to process the multi-core processing part of copy type functions. SET |
| | **UserVarPtr** | Data type info: Data type: *address of an AIL_INT*<br><br>Returns one of the following. |
| | M_FOLLOW_CORE_MAX | Specifies to use the current value of M_CORE_MAX. |

| | | | |
|---|---|---|---|
| | | 1 <= *Value* <= 65535 | Specifies the maximum number of CPU cores to use. INFO |
| M_CORE_MEMORY_BANK | | | Inquires the index of a memory bank local to the specified CPU core. |
| | **TypeValue** | | Specifies the CPU core. |
| | | *Value* >= 0 | Specifies the CPU core to inquire. |
| | **UserVarPtr** | | Data type info: Data type: *address of an AIL_INT*<br><br>Returns one of the following. |
| | | -1 | Specifies that no memory bank is local to the specified CPU core or the CPU core is not available to the process running your application. |
| | | *Value* >= 0 | Specifies the memory bank index. |
| M_CORE_NUM_PROCESS | | | Inquires the number of CPU cores available to the process running your application. Note that if the operating system restricts the available CPU cores, the number of CPU cores returned differs from the number of CPU cores present in your computer. Use M_CORE_AFFINITY_MASK_PROCESS to establish which CPU cores are available to the process running your application. |
| | **UserVarPtr** | | Data type info: Data type: *address of an AIL_INT*<br><br>Returns the following. |
| | | *Value* >= 1 | Specifies the number of CPU cores. |
| M_CORE_SHARING | | | Inquires whether multi-core processing can use multiple logical cores per physical CPU core, when hyper-threading is enabled and supported. SET |
| | **UserVarPtr** | | Data type info: Data type: *address of an AIL_INT*<br><br>Returns one of the following. |
| | | M_DISABLE | Specifies to use only one logical core per physical CPU core, if hyper-threading is enabled. |
| | | M_ENABLE | Specifies to use all the logical cores of a physical CPU core, if hyper-threading is enabled. |

| | | | |
|---|---|---|---|
| M_MEMORY_BANK_AFFINITY_MASK | | | Inquires about a memory bank affinity bit-mask. This mask identifies the memory banks local to the CPU cores available to the process running your application. The operating system establishes the CPU cores available to the process.<br><br>The first element of the array represents the first 64 memory banks. The least-significant bit of the first element represents memory bank 0. The most-significant bit of the first element represents memory bank 63. The least-significant bit of the second element represents memory bank 64 and so on. A memory bank is local to the CPU cores available to the process if the memory bank's corresponding bit is enabled (1). If its corresponding bit is disabled (0), the memory bank is not local to the CPU cores available to the process or is not present in your computer. |
| | **TypeFlag** | | Specifies the following. |
| | | M_LOCAL | Specifies that M_MEMORY_BANK_AFFINITY_MASK returns an affinity bit-mask that identifies local memory banks. |
| | **UserVarPtr** | | Data type info: Data type: *array of type AIL_UINT64 [optionally, in C++: a reference to a std::vector<AIL_UINT64> ]*<br><br>Returns the memory bank affinity bit-mask. |
| M_MEMORY_BANK_AFFINITY_MASK_ARRAY_SIZE | | | Inquires the size of the array required to store the memory bank affinity bit-mask.<br><br>The size of the array might represent a larger number of memory banks than available in your computer. This is because the memory banks might not be indexed by the operating system successively and/or are less than a multiple of 64. |
| | **UserVarPtr** | | Data type info: Data type: *address of an AIL_INT*<br><br>Returns the following. |
| | | *Value* >= 1 | Specifies the size of the memory bank affinity array. |
| M_MEMORY_BANK_CORE_AFFINITY_MASK | | | Inquires about the core affinity bit-mask representing the CPU cores local to the specified memory bank.<br><br>The first element of the array represents the first 64 CPU cores. The least-significant bit of the first element represents CPU core 0. The most-significant bit of the first element represents CPU core 63. The least-significant bit of the second element represents CPU core 64 and so on. A CPU core is available if its corresponding bit is enabled (1). If its corresponding bit is disabled (0), the CPU core is not available. CPU cores always have the same indices, as long as the hardware in your computer and the operating system does not change. |
| | **TypeValue** | | Specifies the memory bank. |
| | **UserVarPtr** | | Data type info: Data type: *array of type AIL_UINT64 [optionally, in C++: a reference to a std::vector<AIL_UINT64> ]*<br><br>Returns the core affinity bit-mask. |

| | |
|---|---|
| M_MEMORY_BANK_NUM | Inquires the number of memory banks local to the CPU cores available to the process running your application. The operating system establishes the CPU cores available to the process. |
| **UserVarPtr** | Data type info: Data type: *address of an AIL_INT*<br>Returns the following. |
| *Value* >= 0 | Specifies the number of memory banks. |
| M_MP_FORCED_DISABLE | Inquires whether Aurora Imaging Library applications can use multi-core processing to execute certain parts of functions. This is set using the Aurora Imaging Configurator utility.<br>This setting takes precedence over M_MP_USE. |
| **UserVarPtr** | Data type info: Data type: *address of an AIL_INT*<br>Returns one of the following. |
| M_NO | Specifies that multi-core processing works normally and all control types are in effect. |
| M_YES | Specifies that multi-core processing is disabled regardless of any other control type. |
| M_MP_NB_PERFORMANCE_LEVEL | Inquires the number of core performance levels.<br>For example, if your CPU has a heterogeneous architecture consisting two core types, this inquire will return 2. |
| **UserVarPtr** | Data type info: Data type: *address of an AIL_INT*<br>Returns the following. |
| *Value* >= 0 | Specifies the number of core performance levels. |
| M_MP_PRIORITY | Inquires the processing priority of the multi-core processing part of functions. SET |
| **UserVarPtr** | Data type info: Data type: *address of an AIL_INT*<br>Returns one of the following. |
| M_ABOVE_NORMAL | Specifies that the multi-core processing part of functions will be executed with above normal priority. |
| M_BELOW_NORMAL | Specifies that the multi-core processing part of functions will be executed with below normal priority. |
| M_HIGHEST | Specifies that the multi-core processing part of functions will be executed with high priority. INFO |

| | | | |
|---|---|---|---|
| | | M_IDLE | Specifies that the multi-core processing part of functions will be executed with idle priority. INFO |
| | | M_LOWEST | Specifies that the multi-core processing part of functions will be executed with low priority. |
| | | M_NORMAL | Specifies that the multi-core processing part of functions will be executed with normal priority. |
| | | M_TIME_CRITICAL | Specifies that the multi-core processing part of functions will be executed with time critical priority. INFO |
| M_MP_USE | | | Inquires whether multi-core processing can be used to execute certain parts of functions.<br>M_MP_FORCED_DISABLE takes precedence over this setting. SET |
| | **UserVarPtr** | | Data type info: Data type: *address of an AIL_INT*<br>Returns one of the following. |
| | | M_DISABLE | Specifies that multi-core processing cannot be used. |
| | | M_ENABLE | Specifies that multi-core processing can be used. |
| M_MP_USE_PERFORMANCE_LEVEL | | | Inquires which cores a multi-core processing application can use on a heterogeneous architecture, according to their performance level. SET |
| | **TypeFlag** | | Specifies the following. |
| | | 1 <= *Value* <= 5 | Specifies the performance level.<br>> Note that Level 1 corresponds to the fastest cores. |
| | **UserVarPtr** | | Data type info: Data type: *address of an AIL_INT*<br>Returns one of the following. |
| | | M_DISABLE | Specifies that multi-core processing cannot be used on cores with the specified performance level. |
| | | M_ENABLE | Specifies that multi-core processing can be used on cores with the specified performance level. |

## Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information. The return value is undefined for M_CORE_AFFINITY_MASK, M_CORE_AFFINITY_MASK_PROCESS, M_MEMORY_BANK_AFFINITY_MASK, and

M_MEMORY_BANK_CORE_AFFINITY_MASK.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll, ailmp11.dll. |

# MappOpenConnection

## Synopsis

Connect to a Distributed Aurora Imaging Library publishing application.

## Syntax

```
void MappOpenConnection(
    AIL_CONST_TEXT_PTR ConnectionDescriptor, //in
    AIL_INT64 InitFlag,                      //in
    AIL_INT64 ControlFlag,                   //in
    AIL_ID *RemoteContextAppIdPtr            //out
)
```

## Description

This function opens a connection to a local or remote computer that is running a publishing application, through the listening port of the publishing application. This function should be run from the monitoring application and it gives access to the objects that the publishing application publishes (read-only or read/write). Once it opens the connection, this function returns the application context identifier of the publishing application.

You must specify the monitoring application's connection port if the default connection port, set with the Aurora Imaging Configurator utility, is different from the publishing application's listening port. Furthermore, if you have set up a passkey for publishing applications on the remote computer (using the Aurora Imaging Configurator utility on the remote computer), you must specify the remote computer's passkey when establishing a connection from the monitoring application. A passkey permits a secure connection between the publishing and monitoring applications. The passkey is a case-sensitive alphanumeric string of up to 16 characters in length.

> Note that the publishing application must have called MappControl() with M_DAIL_CONNECTION set to either M_DAIL_CONTROL or M_DAIL_MONITOR before the connection can take place.

Prior to connecting to a Distributed Aurora Imaging Library publishing application, certain conditions must be met. For more information, see Chapter 63: Distributed Aurora Imaging Library.

## Parameters

ConnectionDescriptor

Specifies the computer to which to connect, and if necessary the listening port of the publishing application. A publishing application must have already enabled publishing using M_DAIL_CONNECTION.

| For specifying the remote application | |
|---|---|
| Value | Description |

| | |
|---|---|
| AIL_TEXT("*dailshm://[Passkey:]localhost[:Port]*") 1 | Opens a connection to a Distributed Aurora Imaging Library publishing application on the local computer using the DAIL SHM protocol. To connect to a publishing application on the local computer using the DAIL SHM protocol, the local computer must have a valid Aurora Imaging Library installation.<br><br>When specifying the string that indicates local computer:<br><br>1. If required, replace *[Passkey]* with the passkey for publishing applications on the local computer. The passkey is an alphanumeric string of up to 16 characters in length. If a passkey was set up for the publishing application, you must specify the passkey when establishing a connection from the monitoring application to the publishing application. If a passkey was not set for publishing applications on the local computer, omit the passkey and the ":" that follows it.<br>2. Following "localhost:", enter the port that the monitoring application should access to initiate a new connection with the publishing application, unless the default connection port is the listening port of the publishing application; in which case, omit the port number and the ":" that follows it.<br><br>> The DAIL SHM protocol only supports connections to localhost. |

| | |
|---|---|
| AIL_TEXT("*dailtcp://[Passkey:]RemoteComputerName[:Port]*") [1] | Opens a connection to a Distributed Aurora Imaging Library publishing application on a remote or local computer. To connect to a Distributed Aurora Imaging Library remote or local computer, that computer must have a valid Aurora Imaging Library installation.<br><br>When specifying the string that indicates the remote computer (or local computer for a publishing application also running locally):<br><br>1. If required, replace *[Passkey]* with the passkey for publishing applications on the remote computer. The passkey is an alphanumeric string of up to 16 characters in length. If a passkey was set up for publishing applications on the remote computer, you must specify the passkey when establishing a connection from the monitoring application to the publishing application. If a passkey was not set for publishing applications on the remote computer, omit the passkey and the ":" that follows it. If you are attempting to connect to a local computer, specify the passkey for publishing applications on the local computer.<br><br>2. Replace *RemoteComputerName* with the remote computer's name or IP address; both IPv4 and IPv6 addresses are supported. A typical IPv4 string has the format n.n.n.n, where *n* is a number between 0 and 255. A typical IPv6 string has the format x:x:x:x:x:x:x:x, where *x* is a hexadecimal number between 0000 and FFFF. If you are supplying an IPv6 address, you must use square brackets to separate the address from the port. If you are attempting to connect to a local computer, set this to *localhost*.<br><br>3. If required, enter the port that the monitoring application should access on the remote computer to initiate a new connection, unless the default connection port is the listening port of the publishing application on the remote computer; in which case, omit the port number and the ":" that follows it. If you are attempting to connect to a local computer, specify the listening port of the publishing application on the local computer. |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

InitFlag

Reserved for future expansion. Set to **M_DEFAULT**.

ControlFlag

Reserved for future expansion. Set to **M_DEFAULT**.

RemoteContextAppIdPtr

Specifies the address in which to write the remote application context identifier returned by this function.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |

| | |
|---|---|
| **DLL** | Requires ail11.dll. |

# MappTimer

## Synopsis

Control or inquire a timer's setting or value.

## Syntax

```
AIL_DOUBLE MappTimer(
    AIL_ID ContextAppId,   //out
    AIL_INT64 ControlType, //in
    AIL_DOUBLE *TimePtr    //in-out
)
```

## Description

This function controls or inquires an Aurora Imaging Library timer's setting or value. There is a different timer for each thread of the application, and one global timer for the application. There is also a separate non-resettable trace timer used by Aurora Imaging Profiler. By default, the timer that is accessed is the timer of the current thread.

This function is useful for benchmarking operations in an application. The timer resolution varies according to the hardware and operating system used.

> Note that functions run at a slower speed during debug mode. Therefore, once you are done debugging your application, you should compile in release mode to achieve maximum processing speed.

## Parameters

ContextAppId

Specifies the identifier of the application context to use.

| For specifying the application context | |
|---|---|
| Value | Description |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application context identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

ControlType

Specifies the control to exert on the timer. It can be set to one of the following:

| For timers | |
|---|---|
| Value | Description |

| | |
|---|---|
| M_TIMER_READ + | Reads the time, in sec, of the timer since the last reset. Note that the time read takes into account (subtracts) the µsec (microseconds) needed to execute M_TIMER_READ.<br><br>In this case, the function is always called synchronously. |
| M_TIMER_RESET + | Resets the timer to zero. |
| M_TIMER_RESOLUTION + | Reads the timer resolution, in seconds. |
| M_TIMER_WAIT + | Waits for the specified period of time, in sec, before returning.<br><br>In this case, the function is always called synchronously. |

Combination value for M_TIMER_READ; M_TIMER_RESET.

You can add the following value to the above-mentioned values to set whether the function will be called synchronously.

| For all ControlType parameters | |
|---|---|
| Combination value | Description |
| M_SYNCHRONOUS | Forces the function to wait for all pending calls of the current thread to complete before continuing. |

Combination values for the values listed in For timers.

You can add one of the following values to the above-mentioned values to specify to access the global timer or the trace timer, instead of the timer of the current thread.

| For controlling and accessing the global or trace timer | |
|---|---|
| Combination value | Description |
| M_GLOBAL | Executes the requested timer operation using the global timer. In an application, there is only one global timer that is accessible by all threads.<br><br>If used with M_SYNCHRONOUS, the global timer will wait for all pending calls of the current thread to complete before continuing. |
| M_TRACE | Executes the requested timer operation using the trace timer. This timer is accessible by all threads in the current application.<br><br>Note that this value can only be combined with M_TIMER_READ or M_TIMER_RESOLUTION. |

TimePtr

Specifies the address of the variable in which to store the timer information produced by M_TIMER_READ or M_TIMER_RESOLUTION. Since **MappTimer**() also returns the requested information when M_TIMER_READ or M_TIMER_RESOLUTION are selected, you can set this parameter to **M_NULL**.

For M_TIMER_RESET, set TimePtr to **M_NULL**.

For M_TIMER_WAIT, TimePtr specifies the address of the variable from which to read the timer information.

## Return value

The returned value, for M_TIMER_READ and M_TIMER_RESOLUTION, is the requested information cast to an *AIL_DOUBLE*. For M_TIMER_RESET and M_TIMER_WAIT, this function returns 0.0.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |

# MappTrace

## Synopsis

Create a user trace marker or section in a trace log.

## Syntax

```
void MappTrace(
    AIL_ID ContextAppId,           //out
    AIL_INT64 TraceType,           //in
    AIL_INT64 TraceTag,            //in
    AIL_INT64 TraceValue,          //in
    AIL_CONST_TEXT_PTR TraceString //in
)
```

## Description

This function creates an Aurora Imaging Library trace marker or section that allows runtime events to be easily identified in a trace log. Add a trace marker or section at places in your code that need attention, such as before entering or after exiting a block of code or before and after an external event. You can create a trace log using Aurora Imaging Profiler, or by calling MappControl() with M_TRACE set to M_LOG_ENABLE. See the Aurora Imaging Profiler and trace logs section of Chapter 65: Development and debugging tools and techniques for more information on creating a trace log.

When creating a trace marker, only a single function call to **MappTrace**() is needed (TraceType set to M_TRACE_MARKER). When creating a trace section, two calls to **MappTrace**() are needed, one function call to start the section and another call to end the section (TraceType set to M_TRACE_SECTION_START and M_TRACE_SECTION_END, respectively).

When creating a trace marker or section, you can assign it a trace tag (TraceTag), a trace value (TraceValue), and/or a trace string (TraceString). The trace tag is a fixed user-defined value that identifies the trace marker or section. The trace value and trace string are used to display information relevant to the current placement of the marker; typically, these are variables that depend on runtime events, such as the calculated or return value of a user-defined function. When reading a trace log with Aurora Imaging Profiler, you can directly search for your trace markers or sections and display the relevant information. Note that when creating a trace section, both calls to **MappTrace**() must have the same TraceTag value.

Optionally, this function can set additional trace tag information using TraceType set to M_TRACE_SET_TAG_INFORMATION. Once this additional tag information is set, all trace markers or sections that have the same TraceTag value will use the trace tag's color and name. When viewing an *.mtrace* file in Aurora Imaging Profiler, trace markers and sections display this new tag information, along with the original trace values and trace strings assigned when the trace marker or section was created. Together, these pieces of information add an extra level of convenience when searching for trace markers or sections.

> Note that **MappTrace**() does nothing if you are not generating a trace log.

## Parameters

ContextAppId

Specifies the identifier of the application context to use.

| |
|---|
| For specifying the application context |

| Value | Description |
| --- | --- |
| M_DEFAULT | Specifies the application context of the current process. |
| *Application context identifier* | Specifies the identifier of an application context.<br><br>Typically, specifying an application context identifier is used to specify a remote application on a remote computer. However, it can be used for a remote application on the local computer, or the current process. When you explicitly specify the identifier of the current process, it is equivalent to specifying M_DEFAULT. |

TraceType

Specifies the type of trace marker to create.

| For specifying the trace type | |
| --- | --- |
| Value | Description |
| M_DEFAULT | Same as M_TRACE_MARKER. |
| M_TRACE_MARKER | Specifies to create a trace marker that identifies a single point in the code. |
| M_TRACE_SECTION_END | Specifies to end a user trace section. It must come after M_TRACE_SECTION_START, and must have the same TraceTag value as the corresponding start. |
| M_TRACE_SECTION_START | Specifies to start a user trace section. It must be followed by a call to **MappTrace**() with M_TRACE_SECTION_END. |
| M_TRACE_SET_TAG_INFORMATION | Specifies to associate a name and/or color with the specified trace tag. Trace markers or sections that have the same tag will display this color an/or name in Aurora Imaging Profiler.<br><br>Associating a color allows the user to quickly identify different trace markers or sections instead of seeing the same color for all user traces in Aurora Imaging Profiler. Associating a name to a marker or section can be helpful in searching for the marker's or section's name rather than its numerical TraceTag value.<br><br>> Note that when generating a trace log in memory, the color and name associated with a trace tag might not be displayed, dependent on if the trace element associated with this call is still in memory or not. |

TraceTag

Specifies a value that identifies this trace marker or section.

| For specifying the trace tag | |
| --- | --- |
| Value | Description |
| M_NULL | Specifies that no trace tag is used for this trace marker or section. |

| | |
|---|---|
| 1 <= *Value* <= 255 | Specifies a user-defined value. When using Aurora Imaging Profiler, you can search for a marker or section based on its TraceTag value.<br><br>When creating a trace section, both the start function call and the end function call must specify the same TraceTag value.<br><br>When TraceType is set to M_TRACE_SET_TAG_INFORMATION, this value specifies the trace tag to apply the tag information to. All trace markers and sections with the same trace tag display the tag's color and/or name. |

TraceValue

Specifies a value relevant to the trace marker or section, or relevant to the surrounding code.

| For specifying the trace value | |
|---|---|
| Value | Description |
| M_NULL | Specifies that no trace value is logged for this trace marker or section. |
| *Value* | Specifies a user-defined value. |

When TraceType is set to M_TRACE_SET_TAG_INFORMATION, you can specify a color associated with the trace tag.

| For specifying the trace color | | |
|---|---|---|
| Value | Description | |
| M_DEFAULT | Specifies the default color. | |
| M_RGB888(<br>AIL_INT Red,<br>AIL_INT Green,<br>AIL_INT Blue<br>) | Specifies an RGB value used to clear an 8-bit 3-band buffer to an RGB color. Each value (red, green or blue) must not exceed 8 bits. | |
| | | **Parameters** |
| | | Red<br><br>Specifies the red band, as a value between 0 and 255. |
| | | Green<br><br>Specifies the green band, as a value between 0 and 255. |
| | | Blue<br><br>Specifies the blue band, as a value between 0 and 255. |

TraceString

Specifies a string relevant to the trace marker or section, or relevant to the surrounding code.

| For specifying the trace string | |
|---|---|
| Value | Description |

| | |
|---|---|
| M_NULL | Specifies that no trace string is logged for this trace marker or section. |
| AIL_TEXT("*TraceString*") [1] | Specifies a user-defined string. |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

When TraceType is set to M_TRACE_SET_TAG_INFORMATION, you can specify a name associated with the trace tag.

| For specifying the trace name | |
|---|---|
| Value | Description |
| AIL_TEXT("M_DEFAULT") [1] | Specifies the default name, which is equivalent to the value of the TraceTag parameter. |
| AIL_TEXT("*TraceTagName*") [1] | Specifies a user-defined name for the trace tag. |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib. |
| **DLL** | Requires ail11.dll. |